QYAI-开源项目文档

第一阶段：环境准备与基础知识

1. sdk 工程操作说明

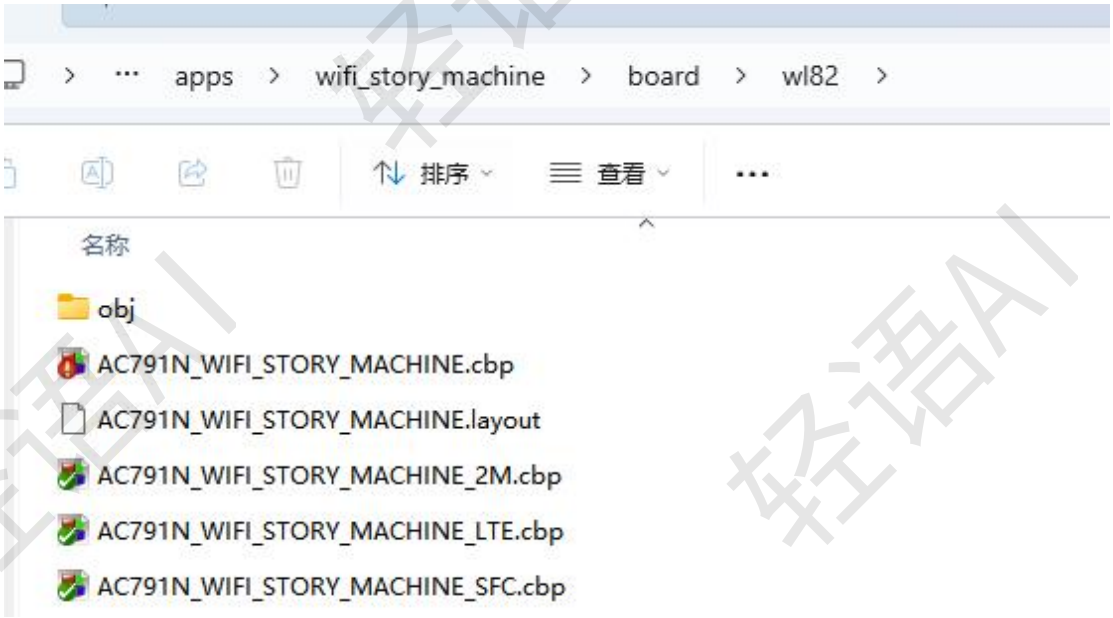
2. sdk 应用开发说明

3. QYAI-AC-A100-V1.1 原理图分析

第二阶段：项目架构与规范

1. wifi_story_machine 工程操作说明

a. 打开工程：




```
apps/wifi_story_machine/board/wl82/AC791N_WIFI_STORY_MACHINE.cbp
```

b. 打开工程后 SDK 中选择开发板对应的板级（选择开发板对应型号）：

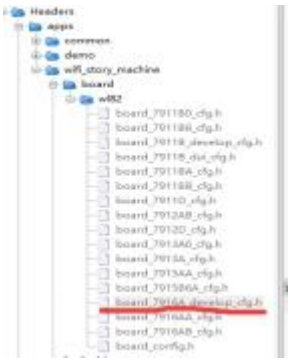
```
Headers/apps/wifi_story_machine/board/wl82/board_config.h
```

c. 在 board_config.h 文件中选择对应的板级宏打开



d. 在 board_7916A_develop_cfg.h 文件中选择具体开发板和需要使用的功能和平台将其宏打开。





e. 以 MXZN-开发板为例，分析其中的功能宏定义

```
C++
//************************************************************
**********************//
//                                MXZN-开发板
//
```

```
//*************************************************************
*********************//
#if SXY_MXZN_BOARD
//#define RF_FCC_DATA_TRIM_ENABLE //使用RF_FCC校准频偏，校准
PA-gian
//#define RF_FCC_TEST_ENABLE  //使能RF_FCC测试，详细配置见
"apps/common/rf_fcc_tool/include/rf_fcc_main.h"
#define TCFG_EQ_ENABLE                          0       //0关闭EQ
调试功能
//#define PRODUCTION_UUID_ENABLE                                //开
启量产ID
//#define PRODUCTION_TEST_ENABLE                                //模
组产测模式使能
//#define PRODUCTION_ALL_TEST_ENABLE
//开启量产测试所有功能
//#define PRODUCTION_ALL_TEST_PORT
IO_PORTA_10//IO_PORTB_01     //触发量产测试所有功能的IO口
//#define PRODUCTION_ALL_TEST_WIFI_SSID       "WIFI-PD-TEST"
//量产测试WiFi名称
//#define PRODUCTION_ALL_TEST_WIFI_PWD        "12345678"  //量
产测试WiFi密码
//#define PRODUCTION_ALL_TEST_BT_NAME         "PD_TEST"   //量
产测试蓝牙名称（暂时不能开启）蓝牙测试太慢
//看产品具体需要选择对应模式进行测试
//#define PRODUCTION_WIFI_TEST_ENABLE         //开启WiFi测试
//#define PRODUCTION_NET_TEST_ENABLE          //开启网络测试
//#define PRODUCTION_BT_TEST_ENABLE           //开启蓝牙测试
//#define PRODUCTION_KEY_TEST_ENABLE          //开启按键测试
//#define PRODUCTION_IR_TEST_ENABLE           //开启红外测试
//#define PRODUCTION_MIC_TEST_ENABLE          //开启麦克风测试
//#define PRODUCTION_DAC_TEST_ENABLE          //开启喇叭测试
//#define PRODUCTION_LED_TEST_ENABLE          //开启灯带测试
//#define PRODUCTION_AUX_TEST_ENABLE          //开启线路输入播放
测试
//#define PRODUCTION_SD_TEST_ENABLE           //开启SD卡播放测
试
//#define PRODUCTION_UDISK_TEST_ENABLE        //开启U盘播放测试
#define CONFIG_BT_MUSIC_MODE_ENABLE                     //蓝牙
播放模式使能
//#define CONFIG_SD_MUSIC_MODE_ENABLE                   //SD
卡播放模式使能
```

```
//#define CONFIG_SD_AUTO_ENABLE                                    //开启SD卡插入自动切换模式，不自动切换可以使用按键进行切换
//#define CONFIG_USB_DISK_MUSIC_MODE_ENABLE
//line in线路输入模式使能
//#define CONFIG_USB_DISK_AUTO_ENABLE
//开启U盘插入自动切换模式
//#define CONFIG_AUX_MUSIC_MODE_ENABLE
//line in线路输入模式使能
//#define CONFIG_AUX_AUTO_ENABLE                                   //开启SD卡插入自动切换模式
//#define TCFG_AUX_DET_PORT                    IO_PORTA_07
//AUX检测引脚
//语音检测的灵敏度相关参数
#define CONFIG_AISP_MIC_ADC_GAIN            90  //本地空闲下mic增益，远场拾音加大80以上，进场拾音40以下
#define SPEECH_ENERGY_MIN                   (84)   //语音检测最小值能量值
#define DIGITAL_VOL_AGC                     1      //AEC后的音频数字增益
#define TCFG_DEBUG_PORT                     IO_PORTC_05 //打印调试IO
#define TCFG_VCC33_CTRL_PORT    IO_PORTB_03 //3.3V电原控制IO
#define TCFG_IOVDD_CTRL_PORT    IO_PORTB_00 //VDDIO电原控制IO,控制低功耗的使能引脚
//#define TCFG_VCC33_CTRL_PORT    IO_PORTA_09 //3.3V电原控制IO
//#define TCFG_IOVDD_CTRL_PORT    IO_PORTA_08 //VDDIO电原控制IO
//#define CONFIG_SPECTRUM_FFT_EFFECT_ENABLE                        //开启FFT频谱显示
//#define USED_WS2812B_SHOWN
//WS2812-rgb灯带-PH1
//#define SPI2_CLOCK_HZ   2500000
//#define SPI2_ATTR_SET   SPI_SCLK_L_UPL_SMPH | SPI_UNIDIR_MODE //上升沿更新数据，下降沿采样数据，单向单线
//#define USED_TM1629_SHOWN                                        //使用TM1629显示
//#define TCFG_TM1629_SPI_CS                   IO_PORTH_02
//SPI2A_CS
//#define CONFIG_SPECTRUM_FFT_EFFECT_ENABLE                        //开启FFT频谱显示
//#define USED_TM1629_EAY                                          //开
```

```
启使用 TM1629 的 LED 显示
//客户原理图上的 VBAT 检测直接接芯片 VBAT 引脚，可以直接使能 TCFG_VBAT_CHECK_EN；
//客户原理图上的 VBAT 检测接芯片的其他引脚，根据具体的 io 口配置 TCFG_VBAT_CHECK_AD_PORT 和具体电阻分压配置
TCFG_VBAT_CHECK_AD_PORT_PPS
#define TCFG_VBAT_CHECK_EN                  1           //使能电量检测，有电池时打开
//#define TCFG_VBAT_CHECK_AD_PORT             IO_PORTA_10 //AD IO 检测 VBAT 电池电量 IO
//#define TCFG_VBAT_CHECK_AD_PORT_PPS         0.6         //AD IO 检测 VBAT 电池的分压占比
//#define TCFG_AUTO_SLEEP_CHECK_EN            1           //使能特定时间待机检测空闲后进入关机
//#define TCFG_AUTO_SLEEP_CHECK_ONLY_SPEECH   1           //使能只检测语音对话作为是否空闲
//#define TCFG_AUTO_DEV_LOW_PWER_TIME_SEC     30          //使能 30 秒进入外设低功耗(未关机，只是把耗电外设降低功耗)
//#define AUTO_SLEEP_TIME_MIN                 5           //5 分钟自动关闭
//#define UART0_BUAD                          9600        //串口 0 的波特率
//#define TIMER_UART_CALIBRATION_ENABLE       0           //使用串口 0 作为时钟校准
//#define TCFG_LED_PWM0_PORT_SHORT_PRESS_AUTO 1
//PWM0 的 IO 短按自动：开灯-70-30-关灯
#define TCFG_LED_STATUES_PORT               IO_PORTH_04 //LED 的 IO-慢闪状态灯
//#define LED_EYA_OPEN                        1           //高电平睁眼-LED
//#define TCFG_LED_EYA_R_PORT                 IO_PORTH_04
//LED 眼睛：对话眨眼
//#define TCFG_LED_EYA_L_PORT                 IO_PORTH_04
//LED 眼睛：对话眨眼
//#define TCFG_LED_PWM0_PORT                  IO_PORTA_06
//PWM0 的 IO-LED1 背景灯
//#define TCFG_LED_PWMCH_PORT                 PWMCH1_L //-LED1 背景灯
//#define TCFG_PWM1_PORT                      IO_PORTA_08 //地球仪灯光的 PWM1 的 IO
//#define TCFG_PWM1_FREQ                      165000 //地球仪灯
```

```
光PWM1的频率
//#define TCFG_PWM1_FREQ_DUTY                50 //地球仪灯光PWM1的频率占空比
//#define AI_UART_CTROL_ENABLE                0    //使用串口0作为AI串口命令（客户自行开发）
//#define UART0_BUAD                          9600 //串口0波特率
#ifdef CONFIG_LTE_PHY_ENABLE
#define LTE_POWER_ONOFF_PORT               IO_PORTA_10 //LTE-4G开关IO
#endif
//一个IO口对应单个按键使能IOKEY,对应多个按键使能ADKEY
//#define TCFG_ADKEY_ENABLE                   1          //AD按键
//#define TCFG_ADKEY_PORT      IO_PORTB_01 //ADC按键IO
#define TCFG_IOKEY_PORT                    IO_PORTB_01 //IO按键的IO
//#define TCFG_IOKEY_PORT                     IO_PORTA_05 //IO按键的IO
#define TCFG_IOKEY_ENABLE                  1           //IO按键
//#define TCFG_POWER_KEY_LONG_PRESS_VAD_EN    1           //使用按键长按触发VAD
//#define TCFG_KEY_VAD_PORT                   IO_PORTH_04 //独立按键触发VAD
//#define TCFG_TOUCH_KEY_PORT                 IO_PORTH_07 //触摸的IO
//#define CONFIG_PRESS_LONG_KEY_POWERON                   //长按开关
#define TCFG_DAC_MUTE_PORT                 IO_PORTA_07 //功放静音IO
#define TCFG_DAC_MUTE_VALUE                0           //功放静音IO电平值
#define TCFG_DAC_AUTO_MUTE_ENABLE          0
//模组有四个输出通道（左右通道，单端的情况下可以四个通道，差分情况下两个差分通道）
#define TCFG_DAC_DIFF_OUT_ENABLE           1
#define TCFG_DAC_HW_CHANNEL_BIT
BIT(0)|BIT(1)|BIT(2)|BIT(3)   //BIT(0):DACFL、 BIT(1):DACFR、 BIT(2):DACRL、 BIT(3):DACRR
#define TCFG_DAC_SOFT_CHANNEL_NUM          2  //差分两个通道,
```

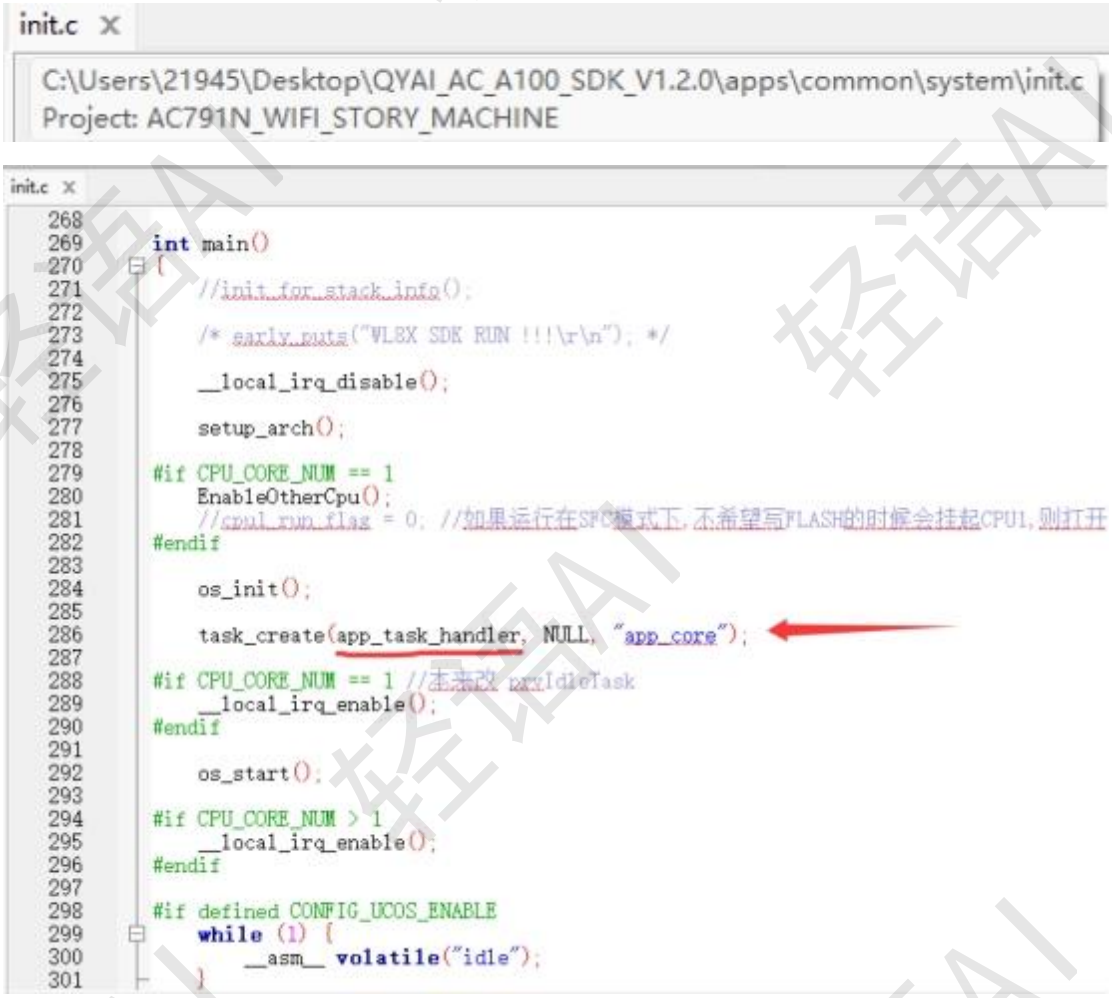
```
单端情况下，这里设置4
#define TCFG_SD0_ENABLE                         0  //使能SD 卡，默
认 SD0
#define TCFG_SD_PORTS                           'D'  //SD0/SD1 的
ABCD 组(默认为开发板 SD0-D),注意:IO 占用问题
#define TCFG_SD_DAT_WIDTH                       1   //1:单线模式,
4:四线模式
#define TCFG_SD_DET_MODE                        SD_CMD_DECT //检测
模式:命令检测，时钟检测，IO 检测
#define TCFG_SD_DET_IO                          -1
//SD_DET_MODE 为SD_IO_DECT 时有效
#define TCFG_SD_DET_IO_LEVEL                    0   //IO 检卡上线的
电平(0/1),SD_DET_MODE 为SD_IO_DECT 时有效
#define TCFG_SD_CLK                             24000000 //SDIO 时
钟
#if TCFG_VBAT_CHECK_EN
#define CONFIG_ASR_POWER_OFF_ENABLE             //语音在线关机功能
#endif
#endif
```

f. board_7916A.c 文件中配置设备信息。board_init()函数也在 board_7916A.c 文件中。



2. wifi_story_machine 工程代码流程重点讲解

a. 在系统公共文件 init.c 文件中进入 main()函数，创建 app_task_handler()函数



b. app_task_handler()函数中初始化系统相关的任务（系统定时器），然后初始化板级相关的信息，先执行 __do_initcall(late_initcall)函数，然后执行 app_main()函数。

```
static void app_task_handler(void *p)
{
#if (!defined CONFIG_SFC_ENABLE && defined PRODUCTION_TEST_ENABLE)
    wdt_reset_enable();//跑 sdram，量产时候看门狗直接复位系统防止看门狗异常中断了卡死
#endif
    sys_timer_init();
    sys_timer_task_init();
#ifdef RTOS_STACK_CHECK_ENABLE
    sys_timer_add(NULL, rtos_stack_check_func, 60 * 1000);
#endif
#ifdef MEM_LEAK_CHECK_ENABLE
    malloc_debug_start();
    sys_timer_add(NULL, malloc_debug_dump, 60 * 1000);
#endif
    board_early_init();
    __do_initcall(early_initcall);
#ifdef CONFIG_SDFILE_EXT_ENABLE
    sdfile_ext_mount_init();
#endif
    __do_initcall(platform_initcall);
    board_init();
    __do_initcall(initcall);
    __do_initcall(module_initcall);
    app_core_init();
#ifdef RF_FCC_TEST_ENABLE
    u8 rf_fcc_test_init(void);
    if (rf_fcc_test_init()) {
#ifndef RF_FCC_IN_NORNAL_WORK
        while (1) {
            os_time_dly(10);
        }
#endif
    }
#endif
    __do_initcall(late_initcall);
    app_main();
#ifdef CONFIG_FINSH_ENABLE
    extern int finsh_system_init(void);
```

c. “app_main.c”：app_main()入口：使能主工程，创建 app_music 的 app_state_machine_test_task 任务。

```
C++
void app_main()
{
    puts("------------ wifi_story_machine app main ----------
---\n");
```

```
    //检查Flash是否被篡改或者移植
    extern int flash_protect_check_match(void);
    while(!flash_protect_check_match()){//校验，防止复制flash放在另外flash可以启动
        os_time_dly(50);
    }
    //产测代码，在量产时打开
#if (defined PRODUCTION_TEST_ENABLE || defined PRODUCTION_ALL_TEST_ENABLE)
    extern int production_test_io_get(void);
    extern void product_function_app_start(void);
    if(production_io_is_enter() && (system_reset_reason_get() != SYS_RST_ALM_WKUP
                                    &&
system_reset_reason_get() != SYS_RST_PORT_WKUP)){
        product_test_task_init();
        return;
    }
#endif
    //电池检测，当电池电压过低时进入关机状态
#if (defined TCFG_VBAT_CHECK_EN && TCFG_VBAT_CHECK_EN == 1)
    extern int vbat_check_percent(void);
 extern int user_is_soft_power_off_read(void);
#if (!defined TCFG_SOFT_POWER_OFF_ALL_ELECTRY || TCFG_SOFT_POWER_OFF_ALL_ELECTRY == 0)//没有彻底断电才需要检查
    //带电池：正常软关机后重启
    if(user_is_soft_power_off_read() &&
system_reset_reason_get() != SYS_RST_ALM_WKUP &&//闹钟

system_reset_reason_get() != SYS_RST_PORT_WKUP &&//按键

system_reset_reason_get() != SYS_RST_LONG_PRESS &&//长按

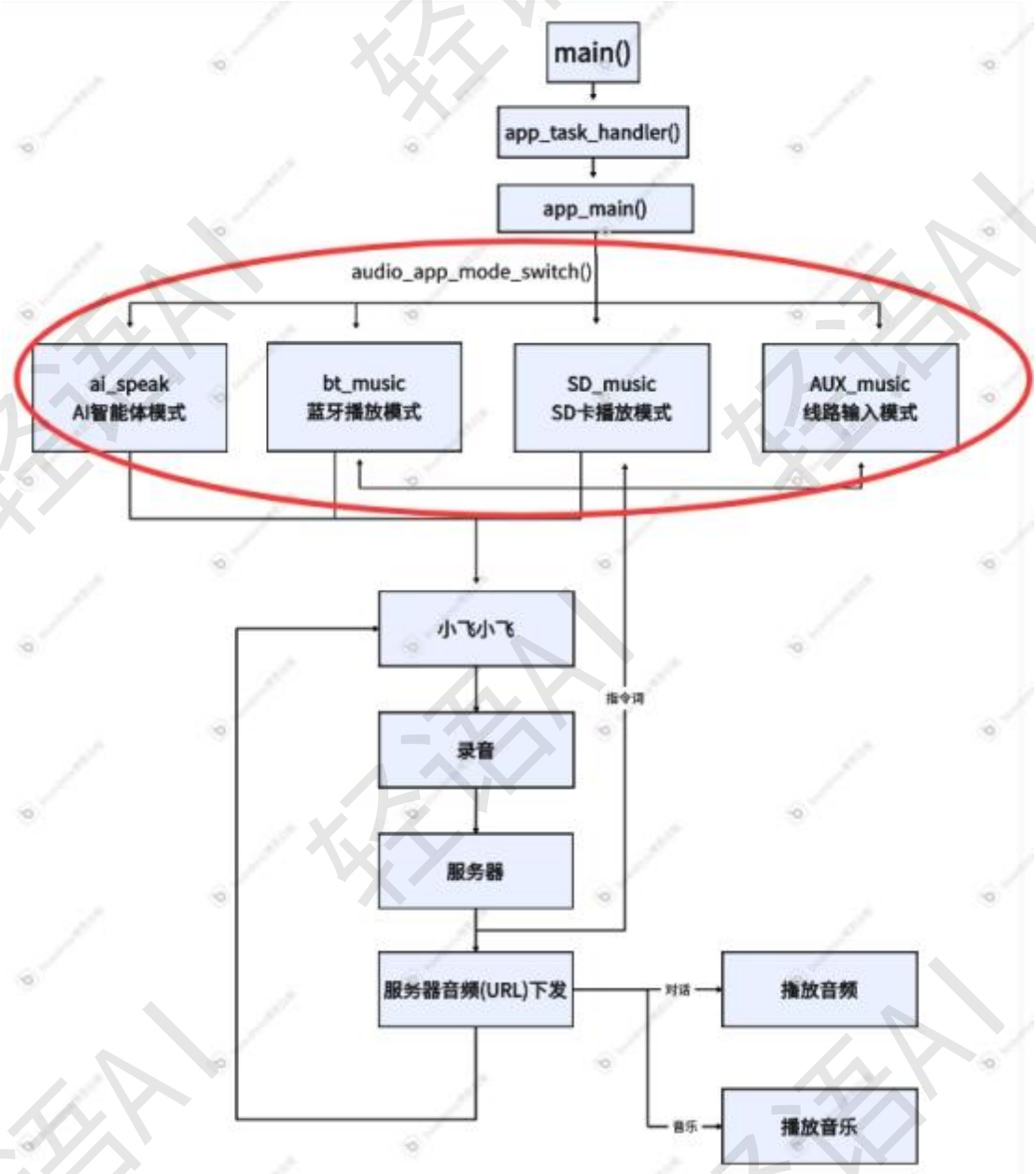
system_reset_reason_get() != SYS_RST_SOFT){//软件重启
        sys_power_poweroff();//异常重启进入关机
    }
#endif
    int vbat_perc = vbat_check_percent();
    if(vbat_perc <= 20){//电压过低
        os_time_dly(10);
        vbat_perc = vbat_check_percent();
        if(vbat_perc <= 20){//电压过低
            os_time_dly(50);
```

```
            sys_power_poweroff();//电压过低强制关机
        }
    }
#endif
    //UI 测试
#ifdef CONFIG_UI_ENABLE
    extern void lvgl_test_demo(void);
    lvgl_test_demo();
#endif
    //闹钟唤醒，有音频播放
    if(system_reset_reason_get() == SYS_RST_ALM_WKUP){
        if(music_play_alarm(0)){
            sys_timeout_add(NULL, music_play_alarm, 1000);
        }
    }
    //屏幕显示
#ifdef  USED_TM1629_SHOWN
    extern int tm_time_init(void);
    tm_time_init();
#endif
#ifdef  USED_WS2812B_SHOWN
    extern int ws_shown_init(void);
    ws_shown_init();
#endif
    //wifi 开启
    int net_ch = 0;
#if (defined CONFIG_WIFI_ENABLE && defined
CONFIG_LTE_PHY_ENABLE)
    int sys_net_channel_read(void);
    net_ch = sys_net_channel_read();
    if(net_ch < 0 || (net_ch != 0 && net_ch != 1)){
        sys_net_channel_write(0);
        net_ch = sys_net_channel_read();
    }
#elif (defined CONFIG_LTE_PHY_ENABLE)
    net_ch = 1;
#endif
#ifdef CONFIG_WIFI_ENABLE
    extern int wireless_net_init(void);
    if(net_ch == 0){
#if (defined CONFIG_WIFI_ENABLE && defined
CONFIG_LTE_PHY_ENABLE)
        extern u8 IPV4_ADDR_CONFLICT_DETECT;
        IPV4_ADDR_CONFLICT_DETECT = 1;//WiFi 开启静态 ip 冲突检测
```

```
#endif
        wireless_net_init();
    }
#endif
    //4G 初始化在 multiple_ethernetif.c 协议栈里面==>具体的内容原厂
封了，只能调用一些应用层接口
#ifdef CONFIG_LTE_PHY_ENABLE
    extern int lte_net_init(void);
    if(net_ch == 1){
#if (defined CONFIG_WIFI_ENABLE && defined
CONFIG_LTE_PHY_ENABLE)
        extern u8 IPV4_ADDR_CONFLICT_DETECT;
        IPV4_ADDR_CONFLICT_DETECT = 0;//4G 关闭静态 ip 冲突检测
#endif
        lte_net_init();
    }else{
        lte_power_cntrol(0);
    }
#endif
    //EQ 开启
#if TCFG_EQ_ENABLE && !defined EQ_CORE_V1
    extern void app_eq_init(void);
    app_eq_init();
#endif // TCFG_EQ_ENABLE
    //状态机模式切换，mode.c 文件，默认进入 AI 智能体模式。
    extern int audio_app_mode_switch(char *name);
audio_app_mode_switch(NULL);
    //LED 初始化
#if (defined TCFG_LED_STATUES_PORT && TCFG_LED_STATUES_PORT !=
-1)
    led_init();
#endif
    //未开启 wifi 时，使能蓝牙功能
#if defined CONFIG_BT_ENABLE && !defined CONFIG_WIFI_ENABLE
    extern void bt_ble_module_init(void);
    bt_ble_module_init();
#else
#if defined CONFIG_BT_ENABLE
    if(net_ch == 1){
        extern void bt_ble_module_init(void);
        bt_ble_module_init();
    }
#endif
#endif
```

```
}
```

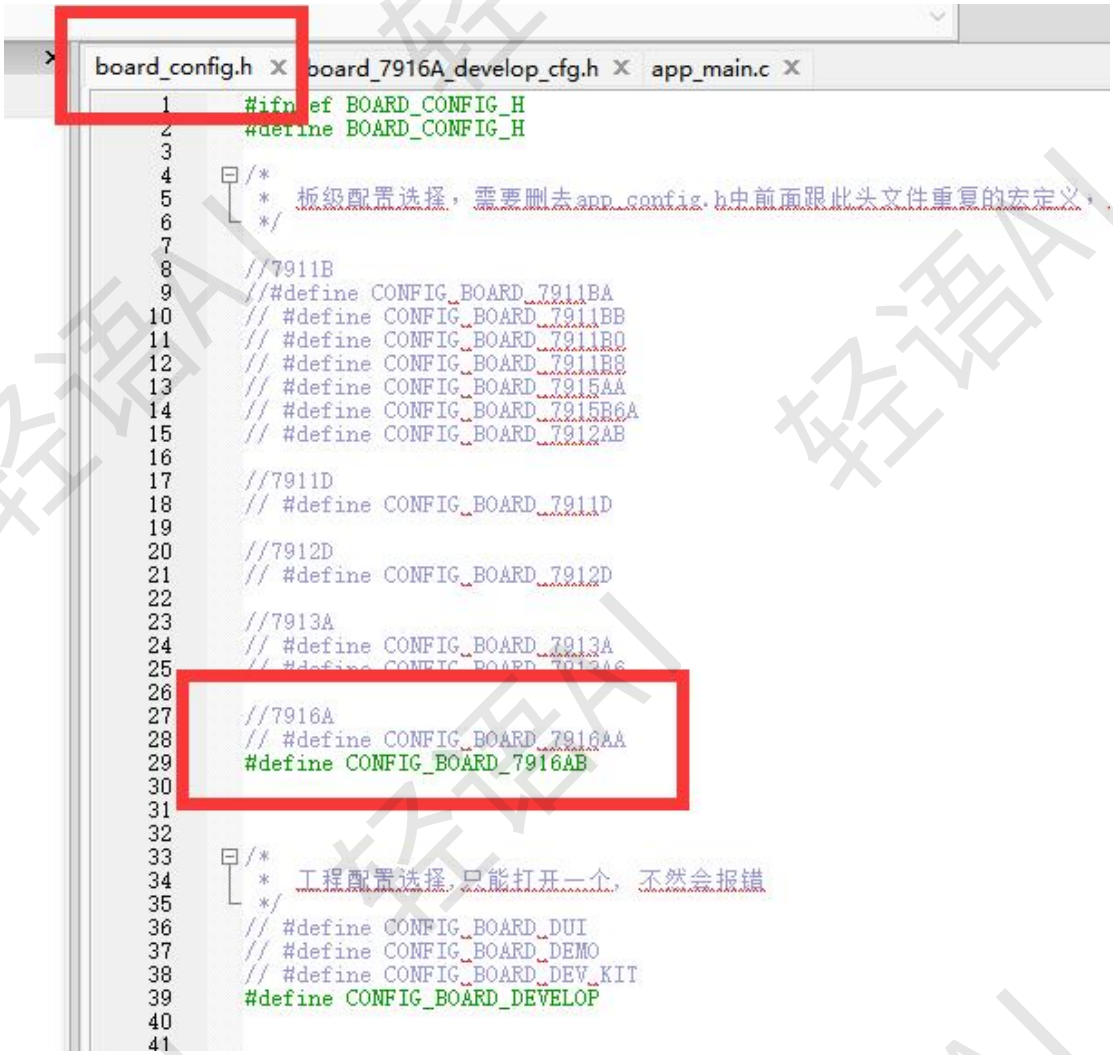
d. “mode.c”：audio_app_mode_switch(NULL);状态机模式切换说明：



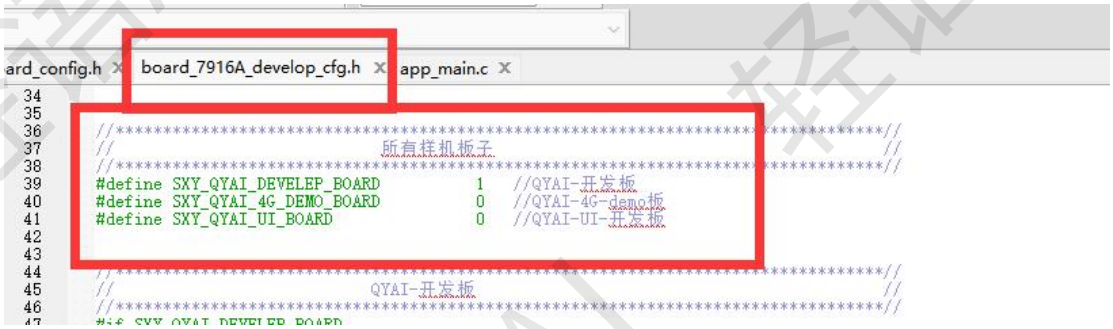
第三阶段：模块例程

1. 正常对话功能——检测开发板硬件是否有问题

1.1 配置正确的板级



1.2 选择正确的开发板



a. 注意：几个基础引脚配置是否正确

```
#define TCFG_DEBUG_PORT                         IO_PORTA_10 //打印
#define TCFG_VCC33_CTRL_PORT                    IO_PORTB_03 //3.3V电源控制IO
#define TCFG_IOVDD_CTRL_PORT                    IO_PORTB_00 //VDDIO电源控制IO
//#define TCFG_VCC33_CTRL_PORT                  IO_PORTA_09 //3.3V电源控制IO
//#define TCFG_IOVDD_CTRL_PORT                  IO_PORTA_08 //VDDIO电源控制IO

#define TCFG_DAC_MUTE_PORT                      IO_PORTH_01 //功放静音IO
#define TCFG_DAC_MUTE_VALUE                     0           //功放静音IO电平值
#define TCFG_DAC_AUTO_MUTE_ENABLE               0
```

- 差分喇叭配置：

```
#define TCFG_DAC_DIFF_OUT_ENABLE            1
#define TCFG_DAC_HW_CHANNEL_BIT             BIT(0)|BIT(1)|BIT(2)|BIT(3)   //BIT(0):DACFL、 BIT(1):DACFR、 BIT(2):DACRL、 BIT(3):DACRR
#define TCFG_DAC_SOFT_CHANNEL_NUM           2                               //差分两个通道
```

- 单端喇叭配置：

```
#define TCFG_DAC_DIFF_OUT_ENABLE            0
#define TCFG_DAC_HW_CHANNEL_BIT             BIT(0)|BIT(1)|BIT(2)|BIT(3)   //BIT(0):DACFL、 BIT(1):DACFR、 BIT(2):DACRL、 BIT(3):DACRR
#define TCFG_DAC_SOFT_CHANNEL_NUM           4                               //差分两个通道
```

- 双麦配置：

```
//*********************************************************************************//
//                                 AUDIO_ADC应用的通道配置                          //
//*********************************************************************************//
#define CONFIG_AUDIO_ENC_SAMPLE_SOURCE      AUDIO_ENC_SAMPLE_SOURCE_MIC
#define TCFG_MIC_CHANNEL_MAP                LADC_CH_MIC1_P_N | LADC_CH_MIC2_P_N | LADC_CH_MIC0_P_N

#define MIC0                                1   //mic0通道
#define MIC1                                2   //mic1通道
#define MIC2_AEC                            0   //回采AEC回声消除通道

#define TCFG_MIC_CHANNEL_NUM                3

#ifdef CONFIG_AUX_MUSIC_MODE_ENABLE
#define TCFG_LINEIN_CHANNEL_MAP             LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             2
#else
#define TCFG_LINEIN_CHANNEL_MAP             -1//LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             0
#endif

#define CONFIG_ASR_CLOUD_ADC_CHANNEL        MIC0        //云端识别mic通道
#define CONFIG_VOICE_NET_CFG_ADC_CHANNEL    MIC0        //声波配网mic通道
#define CONFIG_AISP_MIC0_ADC_CHANNEL        MIC0        //本地唤醒左mic通道
#define CONFIG_AISP_MIC1_ADC_CHANNEL        MIC1        //本地唤醒右mic通道
#define CONFIG_REVERB_ADC_CHANNEL           MIC0        //混响mic通道
#define CONFIG_PHONE_CALL_ADC_CHANNEL       MIC0        //通话mic通道
#define CONFIG_UAC_MIC_ADC_CHANNEL          MIC0        //UAC mic通道

#define AUDIO_RECORD_MIC_COUNT          2       //1单麦,2双麦
```

- 单麦配置：

```
#define MIC0                                1   //mic0通道
#define MIC1                                2   //mic1通道
#define MIC2_AEC                            0   //回采AEC回声消除通道

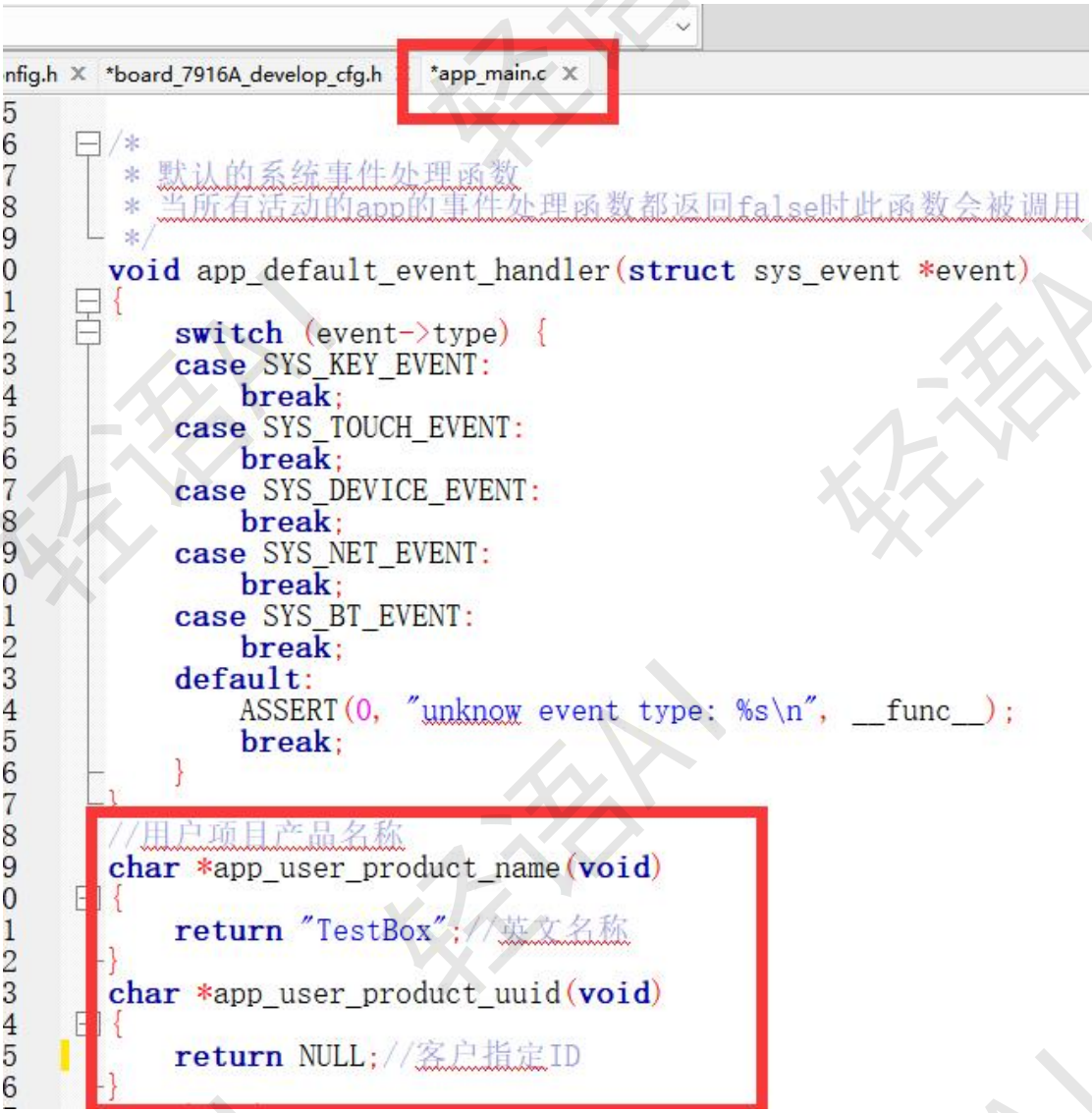
#define TCFG_MIC_CHANNEL_NUM                2

#ifdef CONFIG_AUX_MUSIC_MODE_ENABLE
#define TCFG_LINEIN_CHANNEL_MAP             LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             2
#else
#define TCFG_LINEIN_CHANNEL_MAP             -1//LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             0
#endif

#define CONFIG_ASR_CLOUD_ADC_CHANNEL        MIC0        //云端识别mic通道
#define CONFIG_VOICE_NET_CFG_ADC_CHANNEL    MIC0        //声波配网mic通道
#define CONFIG_AISP_MIC0_ADC_CHANNEL        MIC0        //本地唤醒左mic通道
#define CONFIG_AISP_MIC1_ADC_CHANNEL        MIC1        //本地唤醒右mic通道
#define CONFIG_REVERB_ADC_CHANNEL           MIC0        //混响mic通道
#define CONFIG_PHONE_CALL_ADC_CHANNEL       MIC0        //通话mic通道
#define CONFIG_UAC_MIC_ADC_CHANNEL          MIC0        //UAC mic通道

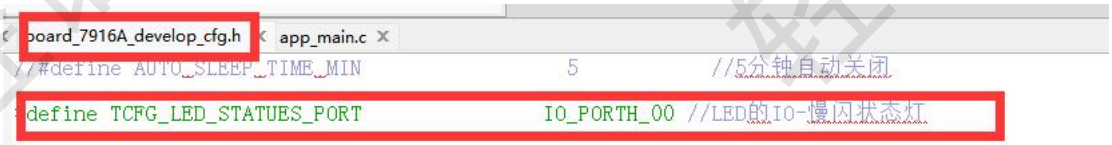
#define AUDIO_RECORD_MIC_COUNT          1       //1单麦,2双麦
```

b. 在 app_main.c 文件中设置正确的用户产品名称和 uuid（用户产品名称和 uuid 不正确会导致开发板一直进入重启状态或者无法绑定账号）



2. 点亮一个 LED 灯——简单应用

a. 对引脚进行配置，打开使能宏



```
onfig.h × board_7916A_develop_cfg.h × app_main.c ×
#endif

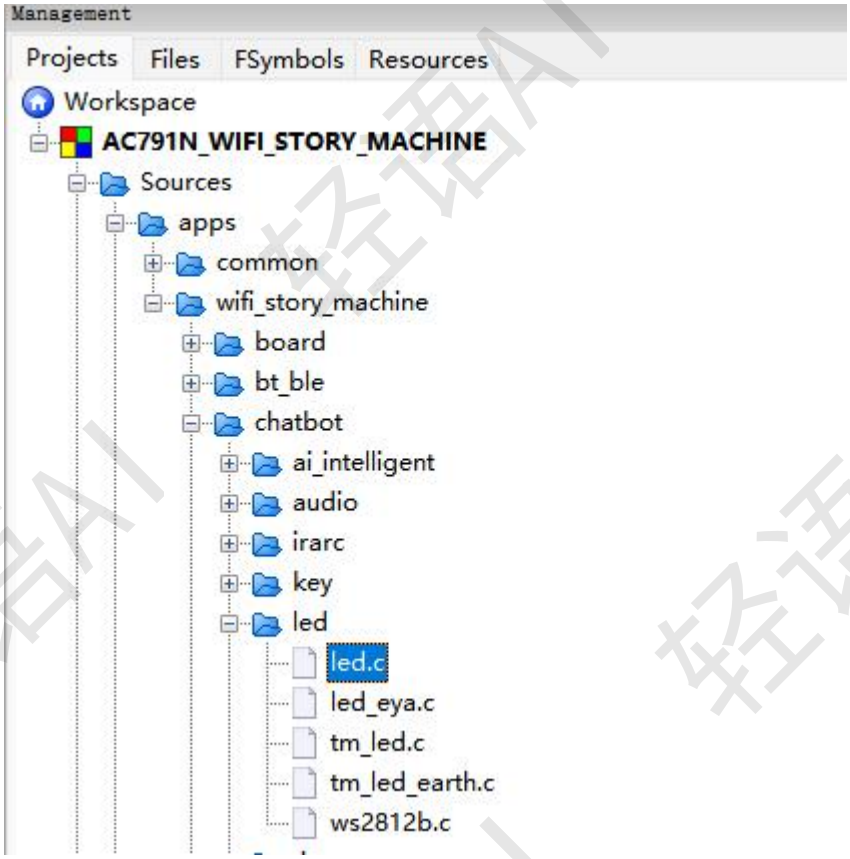

#ifdef CONFIG_LTE_PHY_ENABLE
    extern int lte_net_init(void);
    if(net_ch == 1){
#if (defined CONFIG_WIFI_ENABLE && defined CONFIG_LTE_PHY_ENABLE)
        extern u8 IPV4_ADDR_CONFLICT_DETECT;
        IPV4_ADDR_CONFLICT_DETECT = 0;//4G关闭静态ip冲突检测
#endif
        lte_net_init();
    }else{
        lte_power_cntrol(0);
    }
#endif

#if TCFG_EQ_ENABLE && !defined EQ_CORE_V1
    extern void app_eq_init(void);
    app_eq_init();
#endif // TCFG_EQ_ENABLE

    extern int audio_app_mode_switch(char *name);
    audio_app_mode_switch(NULL);

#if (defined TCFG_LED_STATUES_PORT && TCFG_LED_STATUES_PORT != -1)
    led_init();
#endif
```

b. 可在相应的.c 文件中找到对应函数进行修改，led_init () 函数默认实现的是每 1 秒闪烁一次的流水灯效果。



```
board_7916A_develop_cfg.h | app_main.c | led.c

}
void led_pwm_close(void)
{
    if(pwm_dev_handler){
        dev_ioctl(pwm_dev_handler, PWM_STOP, (u32)&led_pwm);//PWM停止
        dev_close(pwm_dev_handler);
        pwm_dev_handler = NULL;
    }
}
int led_pwm_write(int duty)
{
    if(pwm_dev_handler){
        led_pwm.duty = duty;
        dev_write(pwm_dev_handler, (void *)&led_pwm, 0);//dev_write()²¿¿É¿ØÓÃÓÚ¿ØÖÆ±È
    }
    return pwm_dev_handler ? 0 : -1;
}

static void led_shown(void)
{
    static char led_status;
    if(++led_status & 0x1){
        gpio_direction_output(TCFG_LED_STATUES_PORT, 0);
    }else{
        gpio_direction_output(TCFG_LED_STATUES_PORT, 1);
    }
}

void led_init(void)
{
#if (!defined TCFG_LED_STATUES_VBAT_NET_EN || TCFG_LED_STATUES_VBAT_NET_EN == 0)
    sys_timer_add_to_task("sys_timer", NULL, led_shown, 1000);
#endif
}
```

3. 呼吸灯——pwm 操作

a. 配置对应 led 引脚

```
board_7916A.c | board_7916A_develop_cfg.h | led.c | user_private_cmd.c

//#define USED_TM1629_FAY                       //开启使用TM1629的LED显示

//#define TCFG_VBAT_CHECK_EN                    1           //使能电量检测
//#define TCFG_VBAT_CHECK_AD_PORT               IO_PORTA_10 //AD IO检测VBAT电池电量IO
//#define TCFG_VBAT_CHECK_AD_PORT_PPS           0.6         //AD IO检测VBAT电池的分压占比

//#define TCFG_AUTO_SLEEP_CHECK_EN              1           //使能特定时间待机检测空闲后进入关机
//#define TCFG_AUTO_SLEEP_CHECK_ONLY_SPEECH     1           //使能只检测语音是否处于播放状态
//#define TCFG_AUTO_DEV_LOW_PWER_TIME_SEC       30          //使能30秒进入外设低功耗(未关机，只是把耗电
//#define AUTO_SLEEP_TIME_MIN                   5           //5分钟自动关机

//#define UART0_BUAD                            9600        //串口0的波特率
//#define TIMER_UART_CALIBRATION_ENABLE         0           //使用串口0作为时钟校准
                                                            配置对应引脚
//#define TCFG_LED_PWM0_PORT_SHORT_PRESS_AUTO   1           //PWM0的IO短按自动：开灯-70-30-关灯

#define TCFG_LED_STATUES_PORT                 IO_PORTH_00 //LED的IO:慢闪状态灯
#ifdef TCFG_VBAT_CHECK_EN
#define TCFG_LED_STATUES_VBAT_NET_EN          1           //LED的IO:显示网络状态、电池状态、正常状态
#define TCFG_LED_STATUES_NORMAL_BLINK         1           //LED的IO:正常状态闪烁
#endif
```

b. 配置 pwm 相关信息

```
board_7916A.c | board_7916A_develop_cfg.h | led.c | user_private_cmd.c

    led_cfg.double_led_fast_freq    = 3,    //1 ~ 4, value越大, LED交替快闪速度越慢, value * 100ms闪烁一次
LED_PLATFORM_DATA_END()
                                            配置PWM相关信息
#ifdef TCFG_LED_STATUES_PORT
PWM_PLATFORM_DATA_BEGIN(pwm_data0)
    .port    = TCFG_LED_STATUES_PORT,
    .pwm_ch = PWMCH3_H,//初始化可使多通道,如:PWMCH0_H | PWMCH0_L | PWMCH1_H ... | PWMCH7_H | PWMCH7_L | PWM_TIMER2_OPCH2 | PWM_TIMER3_OPCH3 ,
    .freq    = 10000,//频率
    .duty    = 100,//占空比
    .point_bit = 0,//根据point_bit值调节占空比小数点精度位  0<freq<=4K,point_bit=2;4K<freq<=40K,point_bit=1; freq>40K,point_bit=0;
PWM_PLATFORM_DATA_END()
#endif
```

c. 选择正确的 PWM 通道：通道支持多选，但需注意每组中仅能选择一个通道使用

2.11.1. PWM硬件IO说明

1.MCPWM是有多通道的PWM硬件模块，总共8组PWM通道，每组PWM包含2通道（即H和L通道），MCPWM包含死区时间设置、反向电平设置等，专用与驱动电机。

MCPWM为： - PWMCH0_H、PWMCH0_L：硬件IO为PA3、PA4

- PWMCH1_H、PWMCH1_L：硬件IO为PA7、PA8
- PWMCH2_H、PWMCH2_L：硬件IO为PC7、PC8
- PWMCH3_H、PWMCH3_L：硬件IO为PH0、PH1
- PWMCH4_H、PWMCH4_L：硬件IO为PC0、PC2
- PWMCH5_H、PWMCH5_L：硬件IO为PH3、PH7
- PWMCH6_H、PWMCH6_L：硬件IO为PB2、PB3
- PWMCH7_H、PWMCH7_L：硬件IO为PB6、PB7

d. 打开设备 pwm 通道

board_7916A.c | board_7916A_develop_cfg.h | led.c | user_private_cmd.c

```
#endif

#ifdef CONFIG_DMSDX_ENABLE
static struct dmsdx_platform_data dmsdx_data0 = {
    .block_offset_start = CONFIG_SDNAND_HFS_LEN / 512,
    .block_offset_end = (CONFIG_SDNAND_HFS_LEN + CONFIG_SDNAND_HFAT_LEN)
};
static struct dmsdx_platform_data dmsdx_data1 = {
    .block_offset_start = (CONFIG_SDNAND_HFS_LEN + CONFIG_SDNAND_HFAT_LEN
    .block_offset_end = (CONFIG_SDNAND_HFS_LEN + CONFIG_SDNAND_HFAT_LEN +
};
#endif
                                                    打开设备pwm
REGISTER_DEVICES(device_table) = {
#ifdef CONFIG_UI_ENABLE
    { "pap",    &pap_dev_ops,  (void *)&pap_data },
    { "emi",    &emi_dev_ops,  (void *)&emi_data },
    { "imd",    &imd_dev_ops,  NULL },
#endif
#ifdef TCFG_LED_STATUES_PORT
    { "pwm0",    &pwm_dev_ops,   (void *)&pwm_data0},
#endif
```

e. 采用定时器实现呼吸灯效果

注意：为避免因重复调用呼吸灯初始化操作而创建多个定时器，应在生成新定时器前先检查是否

已有相关定时器。若存在，需先将其关闭。

```
static u16 led_huxi_id;

// 全局变量保存亮度和方向状态
static int pwm = 0;
static int step = 1;

// 呼吸灯核心函数：更新亮度
void led_huxi(void) {
    pwm += step;                    // 按步长更新亮度
    if (pwm >= 100) step = -5;      // 达到最大亮度时反转方向
    if (pwm <= 0) step = 5;         // 达到最小亮度时反转方向
    led_pwm_write(pwm);             // 输出PWM信号
}
int led_huxi_uninit(void)
{
    led_pwm_close();
    if(led_huxi_id){
        sys_timer_del(led_huxi_id);
    }

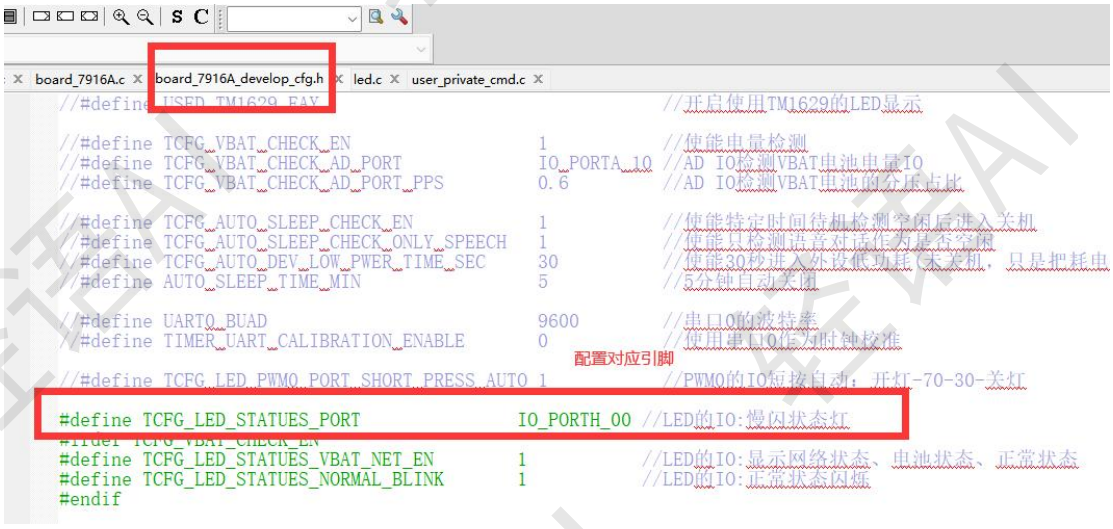
}

int led_huxi_init(void)
{
    led_pwm_open();
    if(!pwm_dev_handler){
        return 0;
    }
    if(led_huxi_id){
        led_huxi_id = 0;
    }
    led_huxi_id = sys_timer_add_to_task("sys_timer", NULL, led_huxi, 50);//
}
void led_init(void)
{
    led_huxi_init();
}
```

采用定时器实现呼吸灯效果

4. 语音控制开关灯——语音控制

a. 配置对应 led 引脚



b. 配置 pwm 相关信息



c. 选择正确的 PWM 通道：通道支持多选，但需注意每组中仅能选择一个通道使用

2.11.1. PWM硬件IO说明

1.MCPWM是有多通道的PWM硬件模块，总共8组PWM通道，每组PWM包含2通道（即H和L通道），MCPWM包含死区时间设置、反向电平设置等，专用与驱动电机。

MCPWM为： - PWMCH0_H、PWMCH0_L：硬件IO为PA3、PA4

- PWMCH1_H、PWMCH1_L：硬件IO为PA7、PA8
- PWMCH2_H、PWMCH2_L：硬件IO为PC7、PC8
- PWMCH3_H、PWMCH3_L：硬件IO为PH0、PH1
- PWMCH4_H、PWMCH4_L：硬件IO为PC0、PC2
- PWMCH5_H、PWMCH5_L：硬件IO为PH3、PH7
- PWMCH6_H、PWMCH6_L：硬件IO为PB2、PB3
- PWMCH7_H、PWMCH7_L：硬件IO为PB6、PB7

d. 打开设备 pwm 通道



e. 定义在线指令数组

```
static const char *open_light[] = {
    "开灯",
    "打开灯光",
    NULL,
};
static const char *close_light[] = {
    "关灯",
    "关闭灯光",
    NULL,
};
static const char *pwm_light[] = {
    "暗一点","灯光暗一点",
    "亮一点","灯光亮一点",
    NULL,
};
```

f. 定义语音回调函数

```
}
int light_pwm_instruction_word_callback(char *word)
{
    char status = 0;
#if (!defined TCFG_LED_STATUES_VBAT_NET_EN || TCFG_LED_STATUES_VBAT_NET_EN == 0)
    for(int i = 0; open_light[i] != NULL; i++){//开灯
        if(!strcmp(word, open_light[i])){
            status = 1;
            puts("-> open light \n");
            goto exit;
            break;
        }
    }
    for(int i = 0; close_light[i] != NULL; i++){//关灯
        if(!strcmp(word, close_light[i])){
            status = 2;
            puts("-> close light \n");
            goto exit;
            break;
        }
    }
    for(int i = 0; pwm_light[i] != NULL; i++){//暗一点、亮一点
        if(!strcmp(word, pwm_light[i])){
            status = 3;
            light_pwm_decinc(i >= 2 ? 1 : 0, 1);
            printf("-> pwm = %d\n",light_pwm);
            goto exit;
            break;
        }
    }
exit:
    switch(status){
        case 1://开灯操作
            light_pwm_init(1);
            break;
        case 2://关灯操作
            light_pwm_uninit(1);
            break;
        case 3://亮度调节操作
            printf("亮度调节操作\n");
            break;
        default:
            break;
    }
#endif
    return status ? 1 : 0;
}
```

g. 在网络语音识别文本的回调函数里，调用语音回调函数

```
rd_7916A.c × board_7916A_develop_cfg.h × led.c × user_private_cmd.c ×
    emoji_tts_callback(buf);
#endif
#endif
    return 0;
}
int net_ai_asr_txt_recv_callback(char *buf, int len)//网络语音识别文本回调函数
{
#ifdef AI_UART_CMD_CTROL_ENABLE
    ai_uart_cmd_data_push(AI_UART_CMD_STT, buf, len);
#endif
    int ret = user_sys_close_check(buf);
    if(ret){
        return ret;
    }
    ret = sys_timeout_stop_off_callback(buf, len);//定时关闭和定时停止播放：1小时后关闭，
    if(ret){
        return ret;
    }
#ifdef TCFG_LED_STATUES_PORT
    ret = light_pwm_instruction_word_callback(buf);
    if(ret){
        return ret;
    }
#endif

    printf("asr_txt : %s \n", buf);
    if(!strcmp("模式切换", buf)){//"模式切换"用户自定义私有命令检测
        return 1;//返回1立刻结束对话，返回0则走大模型对话
    }
    return 0;
}
```

h. 注意点

```
int led_pwm_open(void)
{
    int ret;      而这样的修改方案，能够解决闪烁问题。
    u32 duty;
    u32 channel;

    if(pwm_dev_handler){
        return 0;
    }

    memset(&led_pwm, 0, sizeof(led_pwm));

    gpio_direction_output(TCFG_LED_STATUES_PORT, 0);
    gpio_latch_en(TCFG_LED_STATUES_PORT, 1);//锁定IO

    /*1. open £¬openÕ*º óµÃPWMÍ ¨ µÀ×¡¶àÕ§³ÖÍ¬Ö»¸öÍ¨ µÀµÄHº ÍLÉèÖÃ£¬»ÄÜ¼ûDD¶àÍ ¨ µÀµÄÉèÖÃ*/
    pwm_dev_handler = dev_open("pwm0", &led_pwm);// Ö¸¶¨PWM0Éè±¸£¬£¬ «²Í¿ÉÔÖ»ñÈ¡board.cÀïÖÃµÃ²ÎÊý
    if (!pwm_dev_handler) {
        printf("open pwm err !!!\n\n");
        return -1;
    }
    gpio_set_hd(TCFG_LED_STATUES_PORT, 1);//¿ª£ôÇ¿Cý
    sys_timeout_add_to_task("sys_timer", NULL, led_pwm_release, 100);//释放IO
    return 0;
}
```

```
memset(&led_pwm, 0, sizeof(led_pwm));
if(pwm_dev_handler){
    dev_close(pwm_dev_handler);
}
```

当前操作存在一个问题：灯开着的时候，发出 “开灯” 的语音指令后，灯具会先闪一下再正常亮起。

i. 一个 pwm 多个通道

```
C++
#ifdef TCFG_LED_STATUES_ENABLE
PWM_PLATFORM_DATA_BEGIN(pwm_data0)
    .port     = -1,
//     .pwm_ch = PWM_TIMER3_OPCH3 | TCFG_LED_PWMCH_PORT,//初始化可选多通道,如:PWMCH0_H | PWMCH0_L | PWMCH1_H ... | PWMCH7_H | PWMCH7_L | PWM_TIMER2_OPCH2 | PWM_TIMER3_OPCH3 ,
    .pwm_ch = PWMCH1_L | PWMCH2_L | PWMCH5_L,
    .freq    = 100000,//频率
```

```
    .duty   = 25,//占空比
    .point_bit = 0,//根据 point_bit 值调节占空比小数点精度位:
0<freq<=4K,point_bit=2;4K<freq<=40K,point_bit=1;
freq>40K,point_bit=0;
PWM_PLATFORM_DATA_END()
#endif
```

j. 分别写入 PWM

```
Java
int led1_pwm_write(int org_duty) //上灯
{
    int ret = 0;
    if(pwm_dev_handler){
        int duty = org_duty;  // 直接使用传入的 PWM 值
        duty = duty > LIGHT1_PWM_MAX ? LIGHT1_PWM_MAX : duty;
        duty = duty < 0 ? 0 : duty;  // 确保不小于 0
        led_pwm.pwm_ch = PWMCH1_L;
        led_pwm.duty = duty;
        light1_pwm = org_duty;
        if(light1_pwm_last != light1_pwm){
            light1_pwm_last = light1_pwm;
        }
        dev_write(pwm_dev_handler, (void *)&led_pwm, 0);
//        ret = dev_read(pwm_dev_handler, (void *)&led_pwm,
0);//dev_read 函数返回值为占空比（不带小数点）0-100
//        printf("pwm0 read duty : %d \n", ret);
    }
      printf("pwm1 = %f!!!\n\n",led_pwm.duty);
    if(light1_pwm_last == 0 && light2_pwm_last == 0 &&
light3_pwm_last == 0){
        light_pwm_uninit(0);
    }
    if(org_duty){
        LED_UP_STATUS = true;
    }

    return pwm_dev_handler ? 0 : -1;
}
int led2_pwm_write(int org_duty) //下暖灯
{
    int ret = 0;
    if(pwm_dev_handler){
```

```
        int duty = org_duty;  // 直接使用传入的 PWM 值
        duty = duty > LIGHT_PWM_MAX ? LIGHT_PWM_MAX : duty;
        duty = duty < 0 ? 0 : duty;  // 确保不小于 0
        led_pwm.pwm_ch = PWMCH2_L;
        led_pwm.duty = duty;
        light2_pwm = org_duty;
        if(light2_pwm_last != light2_pwm){
            light2_pwm_last = light2_pwm;
        }
        dev_write(pwm_dev_handler, (void *)&led_pwm, 0);

//        ret = dev_read(pwm_dev_handler, (void *)&led_pwm,
0);//dev_read 函数返回值为占空比（不带小数点）0-100
//        printf("pwm0 read duty : %d \n", ret);
    }

    printf("pwm2 = %f!!!\n\n",led_pwm.duty);
    if(light1_pwm_last == 0 && light2_pwm_last == 0 &&
light3_pwm_last == 0){
       light_pwm_uninit(0);
    }
    if(org_duty){
       LED_DOWN_STATUS = true;
    }
    return pwm_dev_handler ? 0 : -1;
}
int led3_pwm_write(int org_duty) //下白灯
{
    int ret = 0;
    if(pwm_dev_handler){
        int duty = org_duty;  // 直接使用传入的 PWM 值
        duty = duty > LIGHT_PWM_MAX ? LIGHT_PWM_MAX : duty;
        duty = duty < 0 ? 0 : duty;  // 确保不小于 0
        led_pwm.pwm_ch = PWMCH5_L;
        led_pwm.duty = duty;
        light3_pwm = org_duty;
        if(light3_pwm_last != light3_pwm){
            light3_pwm_last = light3_pwm;
        }
        dev_write(pwm_dev_handler, (void *)&led_pwm, 0);
//        ret = dev_read(pwm_dev_handler, (void *)&led_pwm,
0);//dev_read 函数返回值为占空比（不带小数点）0-100
//        printf("pwm0 read duty : %d \n", ret);
    }
```

```
    printf("pwm3 = %f!!!\n\n",led_pwm.duty);
    if(light1_pwm_last == 0 && light2_pwm_last == 0 &&
light3_pwm_last == 0){
        light_pwm_uninit(0);
    }
    if(org_duty){
        LED_DOWN_STATUS = true;
    }
    return pwm_dev_handler ? 0 : -1;
}
```

5. RGB 灯

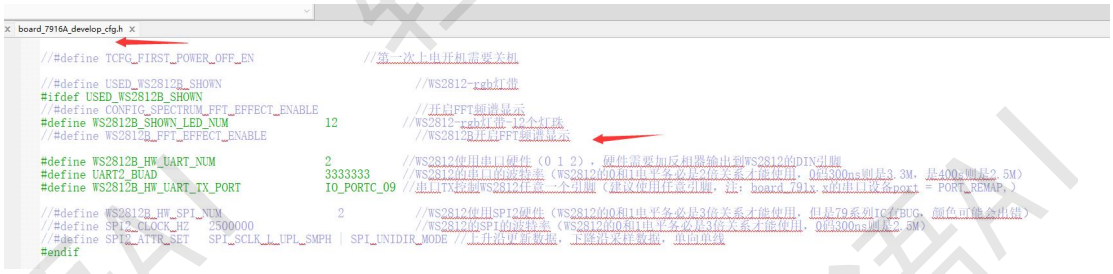
a. **WS2812B RGB LED 灯带** 的驱动控制程序，支持通过 UART 或 SPI 接口驱动灯带，并实现了多种动态显示效果（如彩虹渐变、流水灯、音乐频谱 FFT 显示、呼吸灯等）。

b. 开启相关宏定义

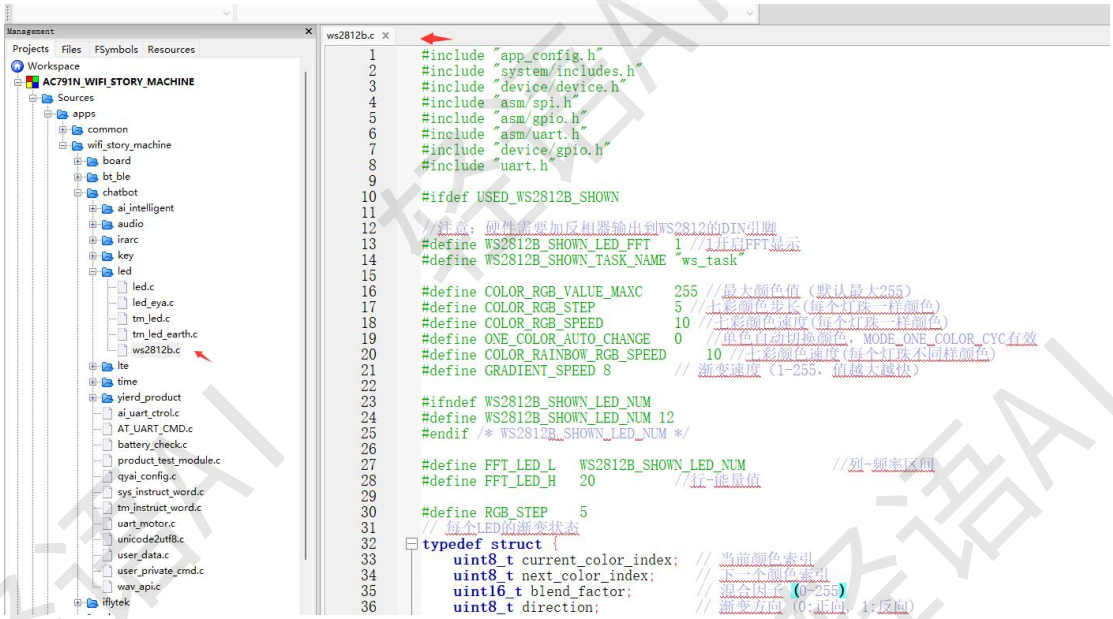
i. **硬件接口选择**：通过宏 WS2812B_HW_UART_NUM 选择 UART/SPI 作为驱动接口（UART 需加反相器，SPI 直接驱动）。

ii. **LED 参数配置**:

iii. WS2812B_SHOWN_LED_NUM：灯带 LED 数量（默认 12 个）。



c. 具体函数实现文件位置如下



d. 显示效果实现（核心功能）

- **静态彩虹渐变**
  - ws2812_rainbow_static　　灯带从左到右固定显示红→黄→绿→青→蓝→紫的渐变，无动态偏移。
- **流水彩虹**
  - ws2812_rainbow_flow　　彩虹渐变整体向右偏移，形成“流水”效果，偏移量 offset 循环递增。
- **全灯渐变（逐色过渡）**
  - ws2812_update_all_gradient　　每个 LED 独立从当前颜色渐变到下一个颜色，整体保持彩虹分布。
- **单色全局渐变**
  - ws2812_shown_one_color_gradient　　所有 LED 同步从红→黄→绿→青→蓝→紫循环渐变（如：红→黄时，绿色逐步增加）。
- **单色自动循环**
  - ws2812_shown_one_color_auto　　所有 LED 固定显示一种颜色，可自动切换颜色（红→黄→绿→…→红），支持亮度调节。
- **音乐 FFT 频谱显示**
  - ws2812_shown_fft　　读取音频 FFT 数据，将不同频率区间的能量值映射为 LED 亮度 / 颜色，支持流水频谱。

- **额外效果：**

- ws2812_shown_face_breath_led:　　呼吸灯效果（亮度从高到

低渐变，颜色循环切换）。

- ws2812_shown_clear: 清空所有 LED。

e. 核心任务 ws_shown_task

- 初始化：创建互斥锁、硬件初始化、清空灯带、默认效果配置。
- 主循环逻辑：
  - 非 FFT 模式：循环调用当前选中的效果函数（如彩虹流水）。
  - FFT 模式：读取音频 FFT 数据（网络 / 蓝牙 / USB/SD/AUX 音乐），映射为频谱显示；无音频时触发呼吸灯 / 自动模式。
  - 任务队列处理：响应外部指令（如笑脸显示、自动模式开关）。

f. 对外 API（应用层调用）

- ws2812_shown_mode(mode, percent) 设置显示模式（如流水彩虹），percent 仅对单色循环模式生效（亮度 0-100）。
- ws2812_shown_one_color_mode() 手动设置单色模式的颜色（如固定显示红色）。
- ws2812_shown_one_color_auto_mode() 手动切换单色循环的颜色（如从红切到黄）。
- ws2812_shown_face(on) 触发笑脸显示效果（随机亮度闪烁）。
- ws2812_shown_auto_led(on) 开启 / 关闭自动模式（呼吸灯 / 彩虹效果）。

6. 按键功能

a. 按键操作流程：

```
static int ai_speaker_event_handler(struct application *app, struct sys_event *event)
{
    switch (event->type) {
    case SYS_KEY_EVENT:
        return ai_speaker_key_event_handler((struct key_event *)event->payload);
    case SYS_NET_EVENT:
        return ai_net_event_handler((struct net_event *)event->payload);
    case SYS_DEVICE_EVENT:
        return ai_speaker_device_event_handler(event);
    case SYS_BT_EVENT:;
        return app_music_bt_event_handler(event);
    default:
        return false;
    }
}
```

```
/*
 * 默认的系统事件处理函数
 * 当所有活动的app的事件处理函数都返回false时此函数会被调用
 */
void app_default_event_handler(struct sys_event *event)
{
    switch (event->type) {
    case SYS_KEY_EVENT:
        struct key_event *key = (struct key_event *)event->payload;
        printf("=================================================");
        printf("action:%d  value:%d\r\n", key->action, key->value);
        printf("=================================================");
//        process_key_event(key);
        break;
```

```
static int bt_music_event_handler(struct application *app, struct sys_event *event)
{
    switch (event->type) {
    case SYS_KEY_EVENT:
#ifdef CONFIG_MP3_DEC_ENABLE
//        post_msg_play_flash_mp3("Volume.mp3", MP3_VOLUME);
//        main_key_event_handler((struct key_event *)event->payload);
#endif
        return bt_music_key_event_handler((struct key_event *)event->payload);
    case SYS_DEVICE_EVENT:
        return main_dev_event_handler((struct device_event *)event->payload);
    case SYS_NET_EVENT:
        extern int ai_net_event_handler(struct net_event *event);
        return ai_net_event_handler((struct net_event *)event->payload);
    case SYS_BT_EVENT:
        return app_music_bt_event_handler(event);
    default:
        return false;
    }
}
```

b. 在.c 文件中上述位置开启按键触发事件，

```
static int bt_music_key_event_handler(struct key_event *key)
{
    u8 volume = 0;
    auto_sleep_check_clear();
    switch (key->action) {
    case KEY_EVENT_UP:
        switch (key->value) {
        case KEY_MODE:
            if(((__this->last_key >> 8) & 0xFF) == KEY_EVENT_LONG){
                printf("-->KEY_EVENT_LONG hand_up\n");
            }
            break;

        }

    case KEY_EVENT_CLICK:
        switch (key->value) {
        case KEY_OK:
            bt_music_play_pause();
            break;
        case KEY_VOLUME_DEC:
            bt_music_set_volume_step(-VOLUME_STEP);
```

c. 在具体函数中开启按键判断逻辑，处理具体按键事件。

- key->value：键值；key->action：按键状态（长按/短按/双击等）。
- 在 key_event.h 文件中配置 key 的触发事件 case 值。

https://note.youdao.com/yws/res/139/WEBRESOURCE490b81396c9a1db8ac996a7169d8d715

- 在 board_7916A.c 文件中配置 ADKEY 键值。

https://note.youdao.com/yws/res/136/WEBRESOURCE63ca479dd7befda3a36658891c4d9674

- 在 board_7916A_develop_cfg.h 文件中使能 AD/IO 按键。

https://note.youdao.com/yws/res/103/WEBRESOURCEfcc1190b45a3b2c7c369a14bf6dd5c89

d. AD 按键:

https://note.youdao.com/yws/res/120/WEBRESOURCE66704fe25d5191c74f66f8459485c897

- 一个 IO 口控制多个按键，通过不同电阻 ADC 数据采集实现不同按键的选择。



```
#if TCFG_ADKEY_ENABLE
/*
#define ADKEY_UPLOAD_R      22  //上拉电阻22K，单位K
#define ADKEY0_DOWNLOAD_R   0
#define ADKEY1_DOWNLOAD_R   10
#define ADKEY2_DOWNLOAD_R   22
#define ADKEY3_DOWNLOAD_R   51
#define ADKEY4_DOWNLOAD_R   75
#define ADKEY5_DOWNLOAD_R   100

#define ADKEY_NONE_V    (0x3FF)//1023
#define ADKEY0_V        (0x3FF * ADKEY0_DOWNLOAD_R / (ADKEY_UPLOAD_R + ADKEY0_DOWNLOAD_R))
#define ADKEY1_V        (0x3FF * ADKEY1_DOWNLOAD_R / (ADKEY_UPLOAD_R + ADKEY1_DOWNLOAD_R))
#define ADKEY2_V        (0x3FF * ADKEY2_DOWNLOAD_R / (ADKEY_UPLOAD_R + ADKEY2_DOWNLOAD_R))
#define ADKEY3_V        (0x3FF * ADKEY3_DOWNLOAD_R / (ADKEY_UPLOAD_R + ADKEY3_DOWNLOAD_R))
#define ADKEY4_V        (0x3FF * ADKEY4_DOWNLOAD_R / (ADKEY_UPLOAD_R + ADKEY4_DOWNLOAD_R))
#define ADKEY5_V        (0x3FF * ADKEY5_DOWNLOAD_R / (ADKEY_UPLOAD_R + ADKEY5_DOWNLOAD_R))

#define ADKEY0_VAL      (ADKEY0_V + ADKEY1_V) / 2
#define ADKEY1_VAL      (ADKEY1_V + ADKEY2_V) / 2
#define ADKEY2_VAL      (ADKEY2_V + ADKEY3_V) / 2
//#define ADKEY2_END      (ADKEY2_V + ADKEY_NONE_V) / 2
#define ADKEY3_VAL      (ADKEY3_V + ADKEY4_V) / 2
//#define ADKEY3_VAL      (ADKEY3_V + ADKEY_NONE_V) / 2
#define ADKEY4_VAL      (ADKEY4_V + ADKEY5_V) / 2
#define ADKEY5_VAL      (ADKEY5_V + ADKEY_NONE_V) / 2
```

- 判断逻辑。注意电阻值要按照从小到大的顺序来设置，否则会读不到正确的按键值。

- **开关机按键电阻一定为 0R**

在 board_7916A.c 文件中配置键值。

- 单个 IOKEY

```
#if TCFG_IOKEY_ENABLE
static const struct iokey_port iokey_list[] = {
    {
        .connect_way = ONE_PORT_TO_LOW,          //IO按键的连接方式
        .key_type.one_io.port = TCFG_IOKEY_PORT,     //IO按键对应的引脚
        .key_value = KEY_OK,                     //按键值
    },
};
const struct iokey_platform_data iokey_data = {
    .enable = 1,                                 //是否使能IO按键
    .num = ARRAY_SIZE(iokey_list),               //IO按键的个数
    .port = iokey_list,                          //IO按键参数表
};
#endif
```

- 多个 IOKEY 配置

```
#if TCFG_IOKEY_ENABLE
static const struct iokey_port iokey_list[] = {
    {
        .connect_way = ONE_PORT_TO_LOW,          //IO按键的连接方式
        .key_type.one_io.port = TCFG_IOKEY_PORT,     //IO按键对应的引脚
        .key_value = KEY_OK,                     //按键值
    },
    {
        .connect_way = ONE_PORT_TO_LOW,          //IO按键的连接方式
        .key_type.one_io.port = TCFG_IOKEY_PORT1,    //IO按键对应的引脚
        .key_value = KEY_UP,                     //按键值
    },
    {
        .connect_way = ONE_PORT_TO_LOW,          //IO按键的连接方式
        .key_type.one_io.port = TCFG_IOKEY_PORT2,    //IO按键对应的引脚
        .key_value = KEY_DOWN,                   //按键值
    },
};
const struct iokey_platform_data iokey_data = {
    .enable = 1,                                 //是否使能IO按键
    .num = ARRAY_SIZE(iokey_list),               //IO按键的个数
    .port = iokey_list,                          //IO按键参数表
};
#endif
```

- 旋转编码器配置

```
#if TCFG_RDEC_KEY_ENABLE
const struct rdec_device rdeckey_list[] = {
    {
        .index = RDEC0 ,
        .sin_port0 = IO_PORTB_06,
        .sin_port1 = IO_PORTB_07,
        .key_value0 = KEY_VOLUME_DEC | BIT(7),
        .key_value1 = KEY_VOLUME_INC | BIT(7),
    },
};
const struct rdec_platform_data rdec_key_data = {
    .enable = 1,
    .num = ARRAY_SIZE(rdeckey_list),             //RDEC按键的个数
    .rdec = rdeckey_list,                        //RDEC按键参数表
};
#endif
```

e. 长按按键实现开关机功能

. 在 board_7916A_develop_cfg.h 文件中打开长按开关使能宏



- 注意：默认下降沿开机，如果硬件电路是上升沿开机，board_7916a.c 此处改成上升沿

```
/************************** PWR config ***************************/
//#define PORT_VCC33_CTRL_IO        IO_PORTA_03                 //VCC33 DCDC控制引脚,该引脚控制DCDC器件输出的3.3V连接芯片HPVDD、VDDIO、VDD33
#ifdef TCFG_ADKEY_PORT
#define PORT_WAKEUP_IO          TCFG_ADKEY_PORT
#elif (defined TCFG_IOKEY_PORT)
#define PORT_WAKEUP_IO          TCFG_IOKEY_PORT
#else
#define PORT_WAKEUP_IO          IO_PORTB_01                 //软关机和休眠唤醒引脚
#endif
#define PORT_WAKEUP_NUM         (PORT_WAKEUP_IO/IO_GROUP_NUM)//默认:0-7:GPIOA-GPIOH, 可以指定0-7组

static const struct port_wakeup port0 = {
    .edge       = FALLING_EDGE,                             //唤醒方式选择,可选: 上升沿/下降沿
    .attribute  = BLUETOOTH_RESUME,                         //保留参数
    .iomap      = PORT_WAKEUP_IO,                           //唤醒口选择
    .low_power  = POWER_SLEEP_WAKEUP|POWER_OFF_WAKEUP,      //低功耗IO唤醒,不需要写0
};
#ifdef CONFIG_PRESS_LONG_KEY_POWERON
#define LONG_RESET_EN       TRUE
#else
#define LONG_RESET_EN       FALSE
#endif
static const struct long_press lpres_port = {
    .enable     = LONG_RESET_EN,
    .use_sec4   = FALSE,                                    //enable = TRUE , use_sec4: TRUE --> 4 sec , FALSE --> 8 sec
    .edge       = FALLING_EDGE,                             //长按方式,可选: FALLING_EDGE / RISING_EDGE --> 低电平/高电平
    .iomap      = PORT_WAKEUP_IO,                           //长按复位IO和IO唤醒共用一个IO
};
```

- 注意：硬件没有设计低功耗电路时，弱 VOOIO 电压档位电压不能支持开机，此时需要把弱 VDDIO 电压档位改成强 VDDIO 电压档位

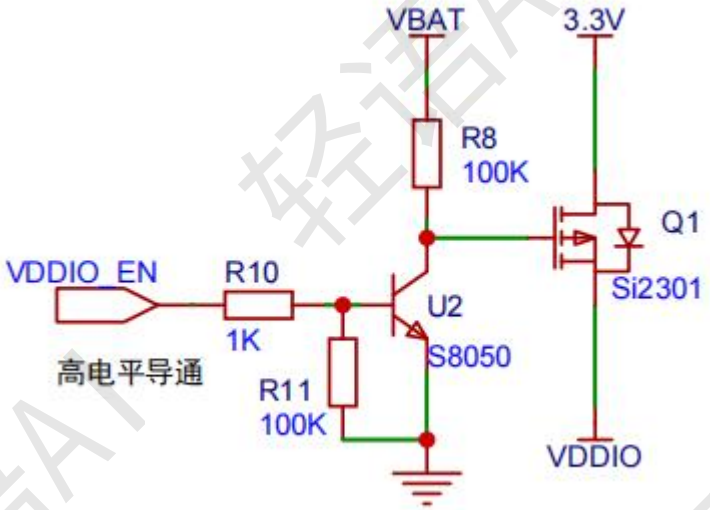
```
//*********************************************************************//
#define CONFIG_FM_DEV_ENABLE                0       //打开外挂FM模块功能
#define CONFIG_FM_LINEIN_ADC_GAIN           100
#define CONFIG_FM_LINEIN_ADC_CHANNEL        3       //FM音频通回采AD通道
#define TCFG_FM_QN8035_ENABLE               1
#define TCFG_FM_BK1080_ENABLE               0
#define TCFG_FM_RDA5807_ENABLE              0


//*********************************************************************//
//                              电源配置                               //
//*********************************************************************//
// #define CONFIG_LOW_POWER_ENABLE                      //低功耗总开关
#define TCFG_LOWPOWER_BTOSC_DISABLE         0
#ifdef CONFIG_LOW_POWER_ENABLE
#define TCFG_LOWPOWER_LOWPOWER_SEL          (RF_SLEEP_EN)// | SYS_SLEEP_EN | RF_FORCE_SYS_SLEEP_EN | SYS_SLEEP_BY_IDLE)
#else
#define TCFG_LOWPOWER_LOWPOWER_SEL          0
#endif
#ifdef CONFIG_INTERNAL_VDDIO_POWER_SUPPLY_ENABLE
#ifdef CONFIG_NO_SDRAM_ENABLE
#define TCFG_LOWPOWER_VDDIOM_LEVEL          VDDIOM_VOL_33V      //强VDDIO电压档位
#else
#define TCFG_LOWPOWER_VDDIOM_LEVEL          VDDIOM_VOL_35V      //强VDDIO电压档位,采用内部VDDIO供电,带sdram的封装实际工作电压至少要满足3.3V
#endif
#else
#define TCFG_LOWPOWER_VDDIOM_LEVEL          VDDIOM_VOL_32V      //强VDDIO电压档位,不要点于外部DCDC的电压
#endif
#define TCFG_LOWPOWER_VDDIOW_LEVEL          VDDIOW_VOL_21V      //弱VDDIO电压档位
#define VDC14_VOL_SEL_LEVEL                 VDC14_VOL_SEL_140V  //内核电压档位
#define SYSVDD_VOL_SEL_LEVEL                SYSVDD_VOL_SEL_126V //内核电压档位

//*********************************************************************//
```

- 低功耗电路

轻语AI



7. 串口通信——串口数据 & AT 指令

a. 打开串口使能宏



b. 配置串口参数（**注意波特率一致**）

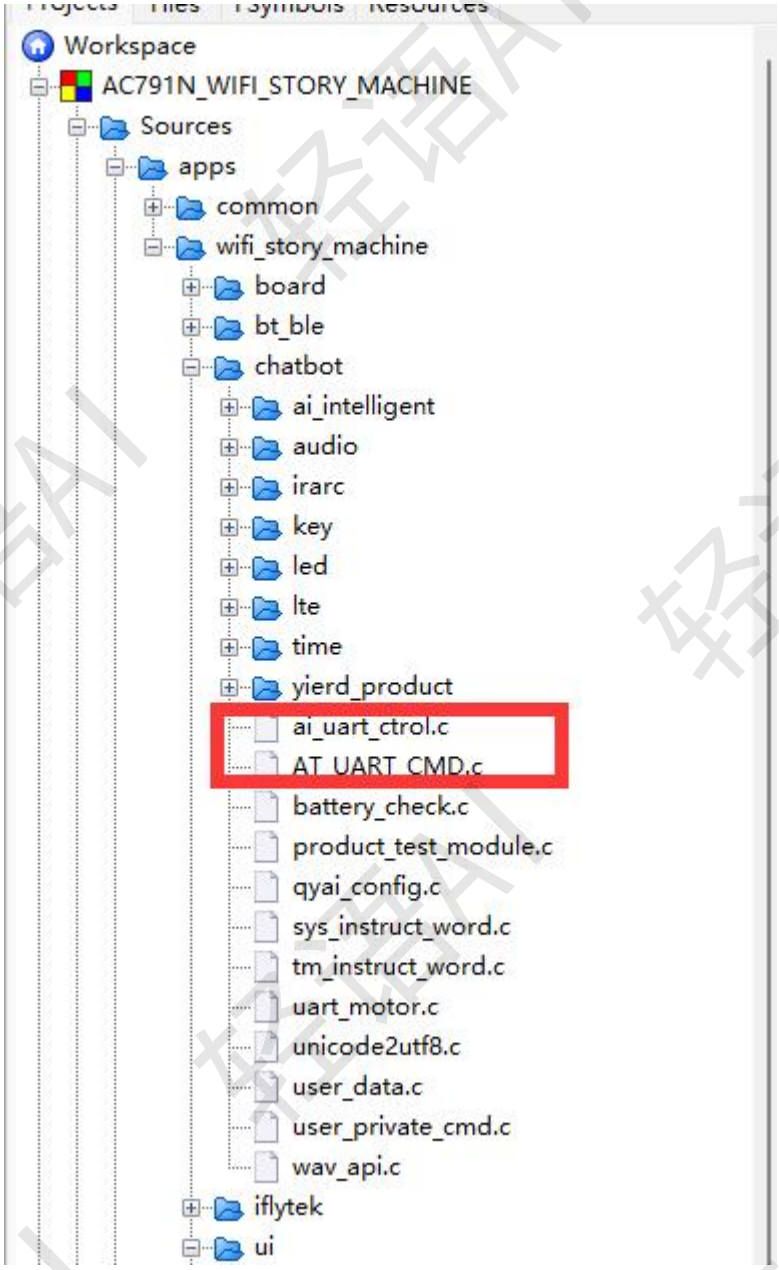
ai_uart_ctrol.c | board_7916A.c | board_7916A_develop_cfg.h | user_private_cmd.c

```
#define TCFG_DEBUG_PORT -1
#endif
#ifndef UART0_BUAD
#define UART0_BUAD  19200
#endif
#ifndef UART2_BUAD
#define UART2_BUAD  9600
#endif
#ifndef UART2_PORT
#define UART2_PORT 0
#endif
```

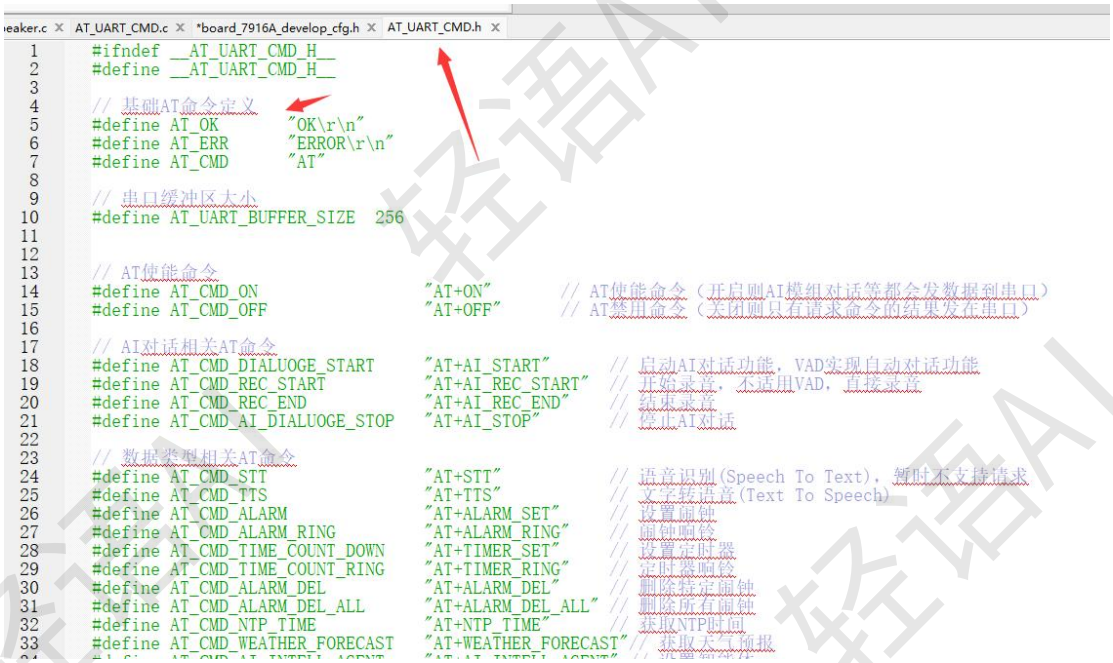
配置串口参数

```
UART0_PLATFORM_DATA_BEGIN(uart0_data)
    .baudrate = UART0_BUAD,
#ifdef TIMER_UART_TX_PORT
    .port = PORT_REMAP,
    .tx_pin = TIMER_UART_TX_PORT, //IO_PORT_USB_DPA,
    .rx_pin = -1, //IO_PORT_USB_DMA,
    .output_channel = OUTPUT_CHANNEL0,
#elif defined AI_UART_CMD_TX_PORT
    .port = PORT_REMAP,
    .tx_pin = AI_UART_CMD_TX_PORT, //IO_PORT_USB_DPA,
    .rx_pin = -1, //IO_PORT_USB_DMA,
    .output_channel = OUTPUT_CHANNEL0,
#else
    .port = PORTA_5_6,
    .tx_pin = IO_PORTA_05, //IO_PORT_USB_DPA,
    .rx_pin = IO_PORTA_06, //IO_PORT_USB_DMA,
#endif
    .max_continue_recv_cnt = 1024,
    .idle_sys_clk_cnt = 500000,
    .clk_src = PLL_48M,
    .flags = UART_DEBUG,
UART0_PLATFORM_DATA_END();
```

c. 具体的串口相关函数在 ai_uart_ctrol.c/AT_UART_CMD.c 文件中



d. 具体的串口相关定义在 ai_uart_ctrol.h/AT_UART_CMD.h 文件中

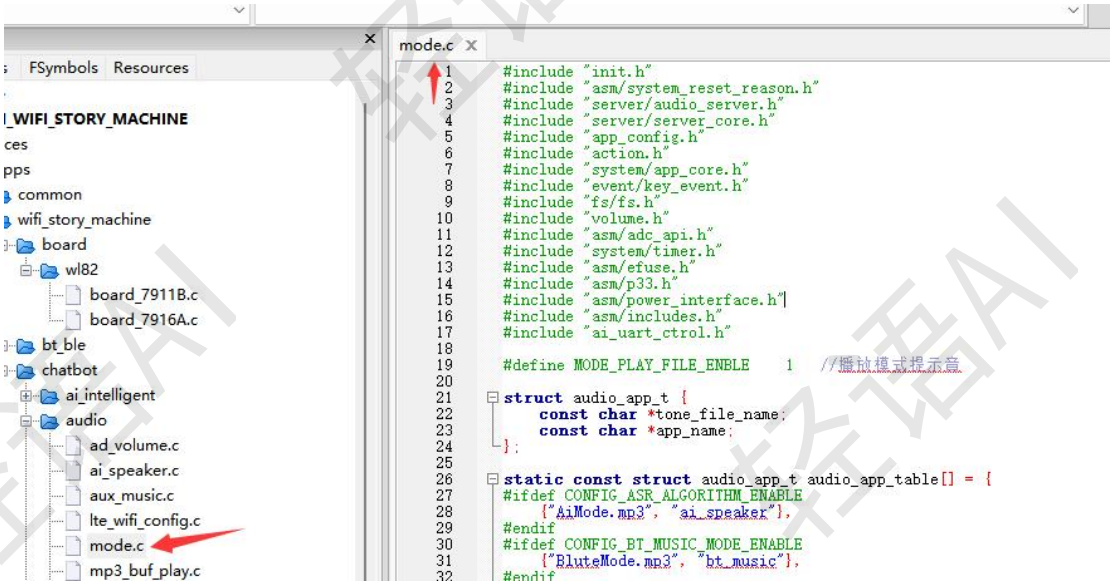


[AI 大模型模组 UI 串口协议 v1.4.txt]

[AT 指令表格说明.xlsx]

8. 模式切换

a. 模式切换逻辑在 mode.c 函数中



b. 音频模式与提示音映射表（编译期通过宏控制是否启用对应模式）

```
static const struct audio_app_t audio_app_table[] = {
#ifdef CONFIG_ASR_ALGORITHM_ENABLE
    {"AiMode.mp3", "ai_speaker"},
#endif
#ifdef CONFIG_BT_MUSIC_MODE_ENABLE
    {"BluteMode.mp3", "bt_music"},
#endif
#ifdef CONFIG_USB_DISK_MUSIC_MODE_ENABLE
    {"UsbDiskMode.mp3", "usbdisk_music"},
#endif
#ifdef CONFIG_SD_MUSIC_MODE_ENABLE
    {"SdMode.mp3", "sd_music"},
#endif
#ifdef CONFIG_AUX_MUSIC_MODE_ENABLE
    {"AuxMode.mp3", "aux_music"  },
#endif
};
```

c. 模式状态检测函数，返回当前运行的音频模式索引（0=AI，1 = 蓝牙，2=SD，3=U 盘，4=AUX），方便上层逻辑判断。

```
int audio_app_mode_check(void)
{
    char play = 0;
    struct intent it;
    struct application *app = get_current_app();
    if (app) {
        if (!strcmp(app->name, "ai_speaker")) {
            return 0;
        } else if (!strcmp(app->name, "bt_music")) {
            return 1;
        } else if (!strcmp(app->name, "sd_music")) {
            return 2;
        } else if (!strcmp(app->name, "usbdisk_music")) {
            return 3;
        } else if (!strcmp(app->name, "aux_music")) {
            return 4;
        }
    }
    return -1;
}
```

d. 模式切换核心函数（重点）

- **audio_app_mode_switch()：按名称 / 自动轮询切换模式**
- 支持两种切换方式：
  - 传入模式名称：精准切换到指定模式；
  - 不传名称/传入 NULL：自动轮询（mode_index 自增，超出数组则重置为 0）；
- 特殊逻辑：如果系统不是“闹钟唤醒重启”，则播放模式提示音。

```
248 }
249 int audio_app_mode_switch(char *name)
250 {
251     char play = 0;
252     struct intent it;
253     if (mode_index >= ARRAY_SIZE(audio_app_table)) {
254         mode_index = 0;
255     }
256     if (name) {
257         for (char i = 0; i < ARRAY_SIZE(audio_app_table); i++) {
258             if (!strcmp(name, audio_app_table[i].app_name)) {
259                 mode_index = i;
260                 goto __find;
261             }
262         }
263         return -1;
264     }
265 __find:
266     if (get_current_app()) {
267         init_intent(&it);
268         it.name = audio_app_table[mode_index].app_name;
269         it.action = ACTION_STOP;     //退出当前模式
```

- **audio_app_mode_num_switch()：index：按索引切换模式，note：是否播报提示音**

```
152 int audio_app_mode_num_switch(char index, char note)
153 {
154     char play = 0;
155     struct intent it;
156     struct application *app = get_current_app();
157
```

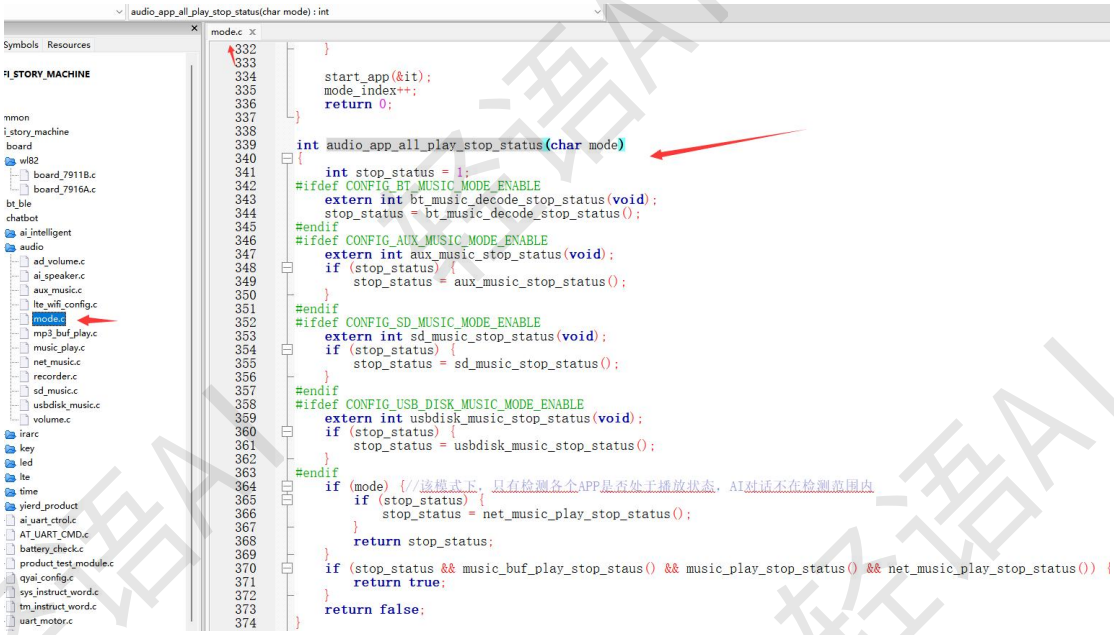
- **audio_app_mode_switch_set()：name：按名称匹配索引后切换，note：是否播报提示音**

```
237 int audio_app_mode_switch_set(char *name, char note)
238 {
239     if (name) {
240         for (char i = 0; i < ARRAY_SIZE(audio_app_table); i++) {
241             if (!strcmp(name, audio_app_table[i].app_name)) {
242                 return audio_app_mode_num_switch(i, note);
243             }
244         }
245
246     }
247     return 0;
248 }
```

9. 音乐播放

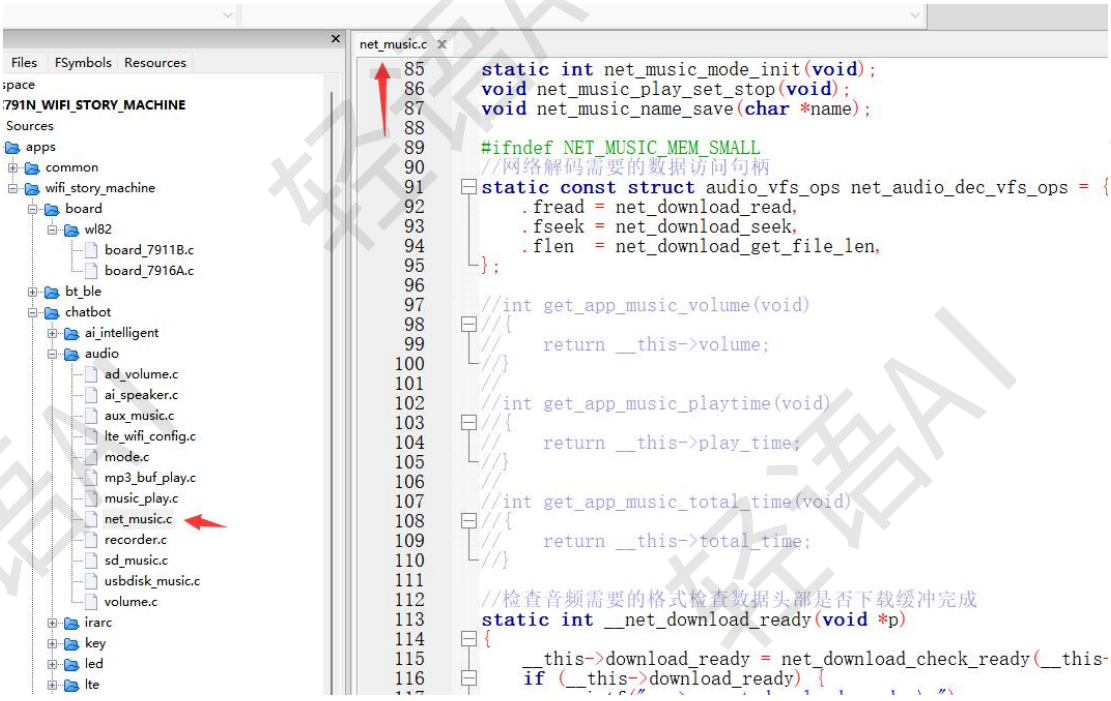
a. **全局播放状态检测（audio_app_all_play_stop_status()）**

作用：检测所有音频通道（本地 / 网络）是否均处于停止状态，用于判断设备是否空闲。



b. **网络音乐播放**

- net_music.c 文件时网络音乐播放的核心实现，完整支持网络音乐（HTTP/websocket）的下载、解码、播放、控制（暂停 / 继续 / 上一首 / 下一首 / 音量 / 循环）、状态管理、UI 联动、异常处理等全流程。



- 播放状态管理（基础控制）
  - net_music_play_stop_status()功能：查询网络音乐的播放停止状态。
    - 返回值：布尔型，true 表示网络音乐当前处于**停止播放**状态；false 表示网络音乐未停止（可能为播放中 / 暂停状态）。

- net_music_play_pause_status()功能：查询网络音乐的播放暂停状态。
  - 返回值：布尔型，true 表示网络音乐当前处于**暂停播放**状态；false 表示网络音乐未暂停（可能为播放中 / 停止状态）。
- net_music_play_start_status()功能：查询网络音乐的播放中状态。
  - 返回值：布尔型，true 表示网络音乐当前**正在播放**；false 表示网络音乐未播放（可能为停止 / 暂停状态）。

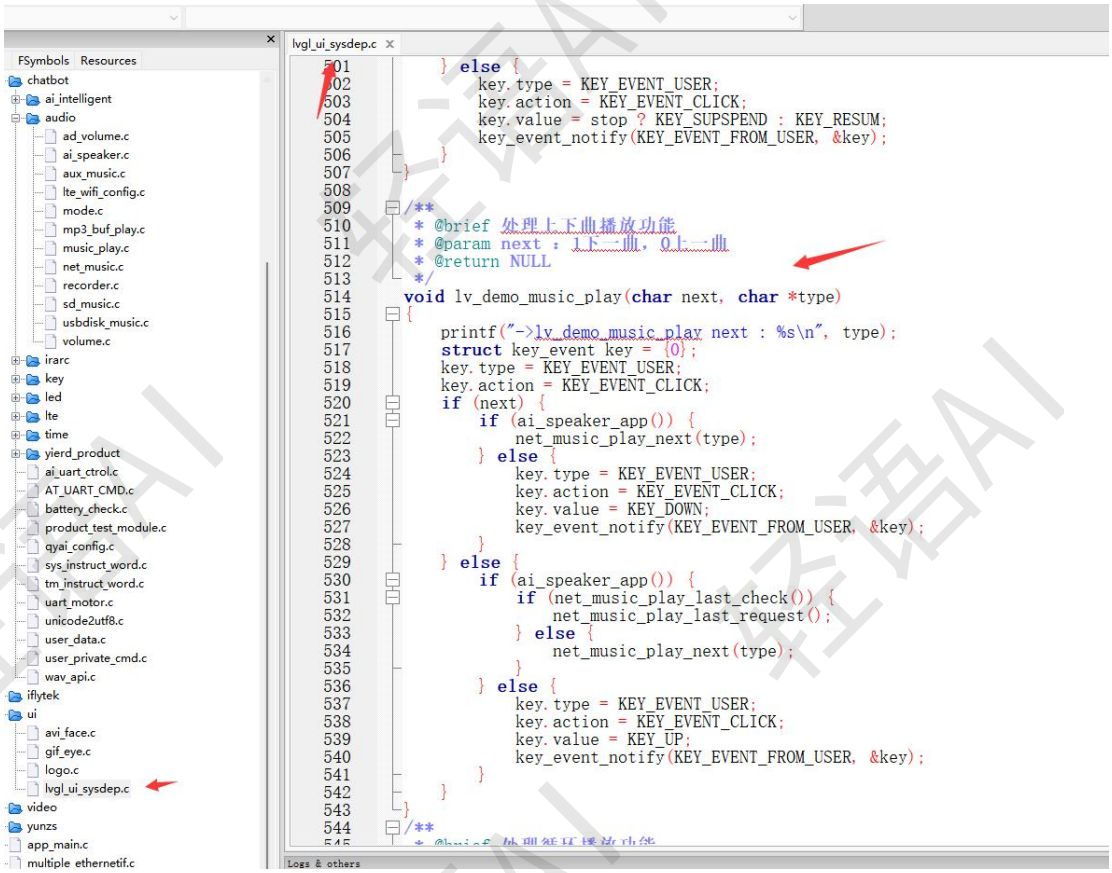
播放控制函数（暂停 / 继续 / 停止 / 循环）

- net_music_play_pause(u8 stop)：暂停 / 继续播放。stop 控制指令：0-恢复播放，非 0-暂停播放
- net_music_play_loop()：循环播放
- net_music_play_loop_clear()：停止循环播放-->随机播放

切歌逻辑（上一首 / 下一首）

- net_music_play_next (char *priv)：切换播放下一首音乐。
  - 通过 websocket 从云端获取下一首音乐 URL 并播放。
  - 支持两种切换方式：
    - 传入内容名称：切换到指定内容播放；例如：传入“podcast”播放博客内容、传入 {"story_play_next":"content:children_story"}json 字符串，播放儿童故事。
    - 不传名称/传入 NULL：播放网络音乐。
- net_music_play_last (void)：切换播放上一首音乐。

c. **非 AI 模式下音乐播放**




```
    } else {
        key.type = KEY_EVENT_USER;
        key.action = KEY_EVENT_CLICK;
        key.value = stop ? KEY_SUPSPEND : KEY_RESUM;
        key_event_notify(KEY_EVENT_FROM_USER, &key);
    }
}

/**
 * @brief 处理上下曲播放功能
 * @param next : 1下一曲，0上一曲
 * @return NULL
 */
void lv_demo_music_play(char next, char *type)
{
    printf("->lv_demo_music_play next : %s\n", type);
    struct key_event key = {0};
    key.type = KEY_EVENT_USER;
    key.action = KEY_EVENT_CLICK;
    if (next) {
        if (ai_speaker_app()) {
            net_music_play_next(type);
        } else {
            key.type = KEY_EVENT_USER;
            key.action = KEY_EVENT_CLICK;
            key.value = KEY_DOWN;
            key_event_notify(KEY_EVENT_FROM_USER, &key);
        }
    } else {
        if (ai_speaker_app()) {
            if (net_music_play_last_check()) {
                net_music_play_last_request();
            } else {
                net_music_play_next(type);
            }
        } else {
            key.type = KEY_EVENT_USER;
            key.action = KEY_EVENT_CLICK;
            key.value = KEY_UP;
            key_event_notify(KEY_EVENT_FROM_USER, &key);
        }
    }
}
/**
```

- lv_demo_music_play_pause (char stop, char *type)：处理音乐暂停 / 播放。
  - stop 控制指令：1 - 暂停播放，0 - 恢复播放；AI 模式下直接调用网络音乐接口，非 AI 模式下发按键事件通知，实现音乐播放暂停逻辑。
  - lv_demo_music_play (char next, char *type)：处理音乐上下曲切换。
    - next 控制指令：1 - 切换下一曲，0 - 切换上一曲；，AI 模式下切换上一曲时先检查缓存链表，无则切下一曲，非 AI 模式下发按键事件通知，实现上下曲切换。
- lv_demo_music_play_loop (char enable)：处理音乐循环播放开关。
  - enable 控制指令：1 - 开启循环播放，0 - 关闭循环播放；AI 模式下直接调用网络音乐循环接口，非 AI 模式下发按键事件通知，实现音乐循环逻辑。

10. 音量控制

- ai_speaker_volume_set 是核心函数，负责同步设置音箱所有音频播放模块的音量；
- sys_all_volume_auto_set 是封装后的简化接口，调用核心函数实现 “仅设置

音量” 的快捷操作



11. 语音唤醒打断、麦克风启停和静音切换 API

a. 语音唤醒打断

```
if (keyworld_start) {
    // AI对话状态下，退出对话
    aisp_clear(1);
    ai_led_off();
    printf("AI对话状态下，退出对话\n");
} else {
    // 非AI对话状态下，唤醒AI功能
    aisp_wake(0);
    ai_led_on();
    printf("非AI对话状态下，唤醒AI功能\n");
}
```

b. 麦克风启停

```
    // 切换麦克风按钮的状态
    mic_state = !mic_state;
    if (mic_state) {
#ifdef APPLAYER_ENABLE
        // 开启麦克风
        aisp_resume();
#endif
    } else {
#ifdef APPLAYER_ENABLE
        // 禁用麦克风
        aisp_suspend();
#endif
    }
    // 打印状态信息
    printf("-> 麦克风按钮状态: %s\n", mic_state ? "关闭" : "开启");
```

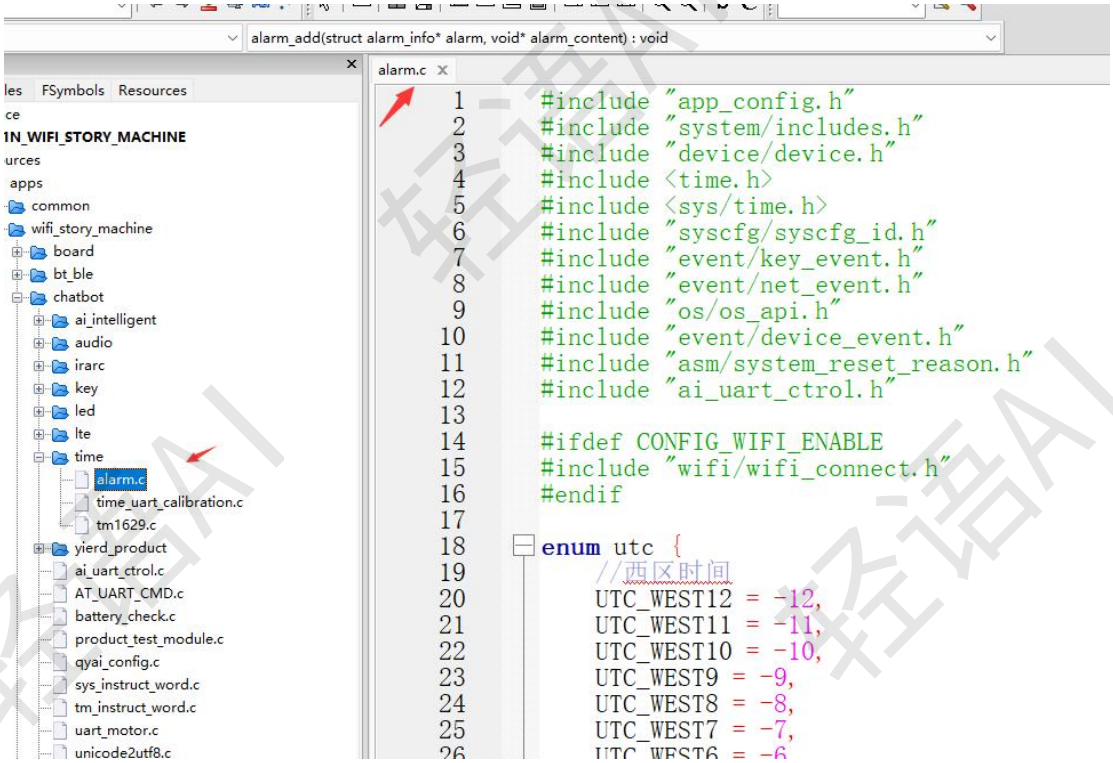
c. 静音切换 API

```
        // 切换扬声器按钮的状态
        speaker_state = !speaker_state;
        if (!speaker_state) {
#ifdef APPLAYER_ENABLE
            // 开启扬声器
            dac_mute_control(0, 1); // 0表示取消静音
#endif
        } else {
#ifdef APPLAYER_ENABLE
            // 静音扬声器
            dac_mute_control(1, 1); // 1表示静音
#endif
        }
        // 打印状态信息
        printf("-> 扬声器按钮状态: %s\n", speaker_state ? "静音" : "开启");
```

12. 闹钟事件

a. **alarm.c 文件是闹钟事件的核心文件**

- 核心功能包括:
  - 支持最多 10 个闹钟，可设置单次 / 循环（每天 / 指定星期）闹钟
  - 基于 RTC（实时时钟）硬件实现闹钟定时，NTP 网络校时同步时间
  - 支持闹钟的增删改查、掉电存储（syscfg）、响铃通知、自定义音乐 URL 存储
  - 提供外部调用接口，支持通过时间 / 索引删除闹钟、通过时分添加闹钟等
  - 内置时间校准机制（30 分钟自动校时），支持 UTC 东八区时间转换



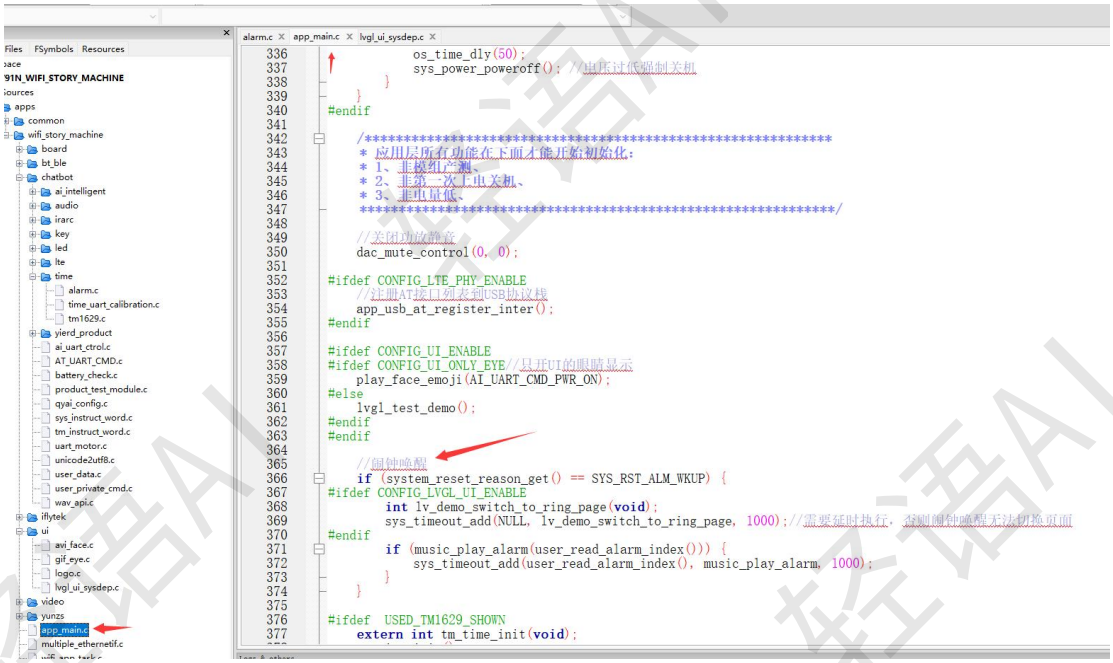
b. **闹钟管理模块**

- alarm_add()添加闹钟到数组，满了则替换第一个，保存到 syscfg（掉电存储）
- alarm_del()按时间删除闹钟（允许 ±2 分钟误差），同步通知串口 / AI 模块
- alarm_first_set()开机初始化：清理重复闹钟，选择最近的闹钟设置到 RTC 硬件
- alarm_chack_time()检测闹钟时间是否过期：过期则删除（单次）或更新下一次触发时间（循环）
- alarm_rings()闹钟响铃中断回调：发送响铃通知，更新循环闹钟的下一次触发时间

c. **外部接口模块（供其他模块调用）**

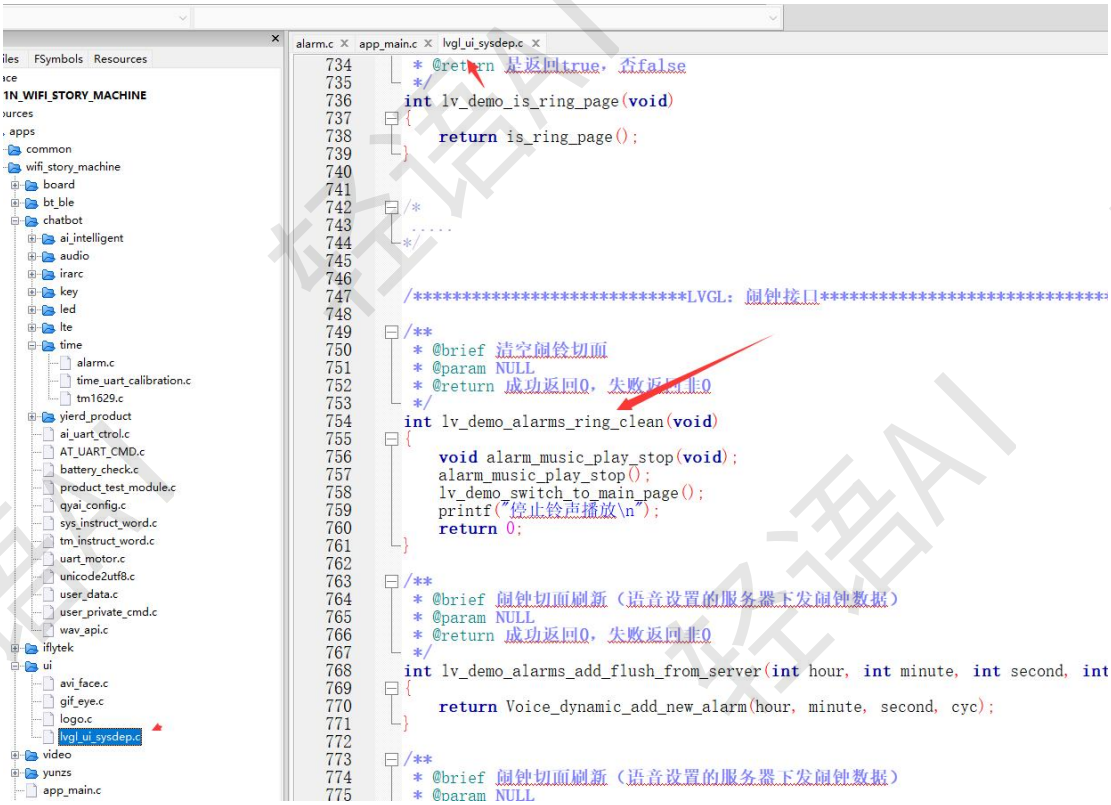
- alarm_delete_by_index()通过索引（0-9）删除指定闹钟
- alarm_delete_by_time()通过时间删除闹钟
- alarm_add_by_hour_min()通过时 / 分添加闹钟（自动处理当天 / 次日）
- alarm_time_read/write()读取 / 写入所有闹钟配置
- get_sys_time()获取当前系统时间（从 RTC 读取）

d. 设备关机后，此前设置的闹钟到点将自动开机并触发响铃，相关函数如下：



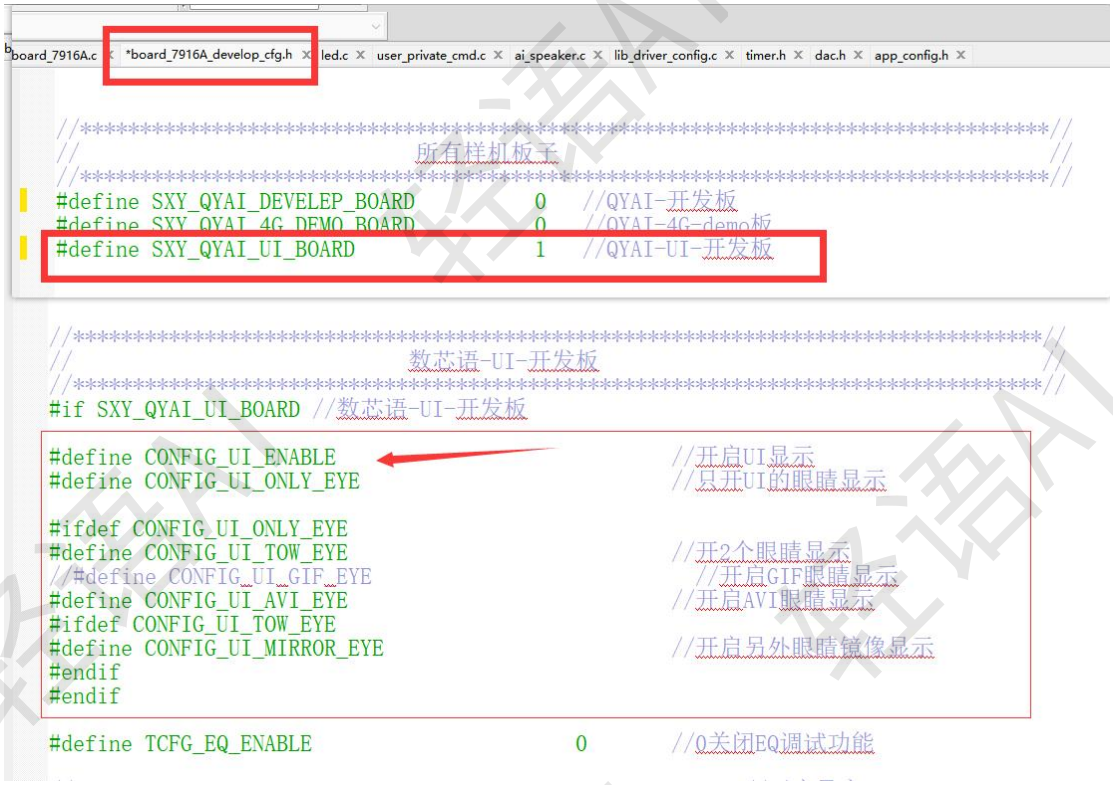
e. 关闭闹铃函数：

. lv_demo_alarms_ring_clean() 关闭闹铃

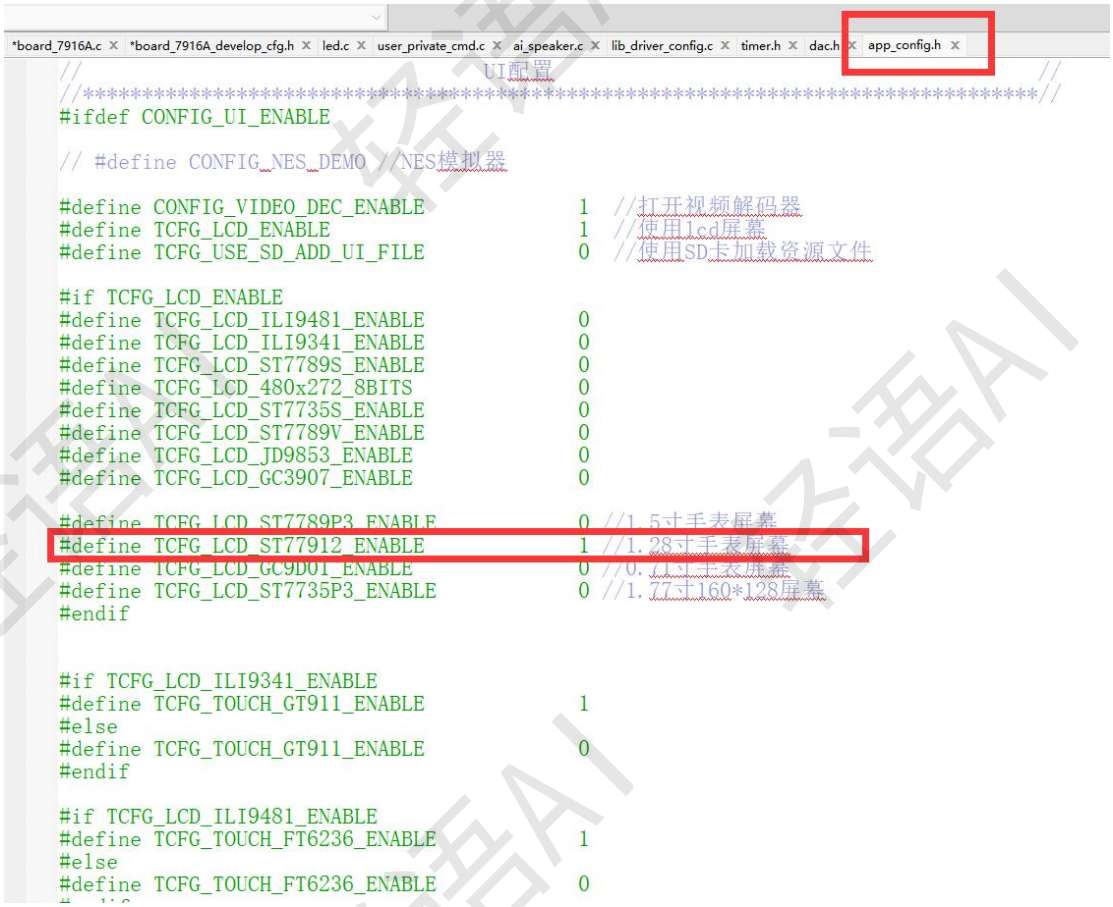


13. 屏幕初始化测试

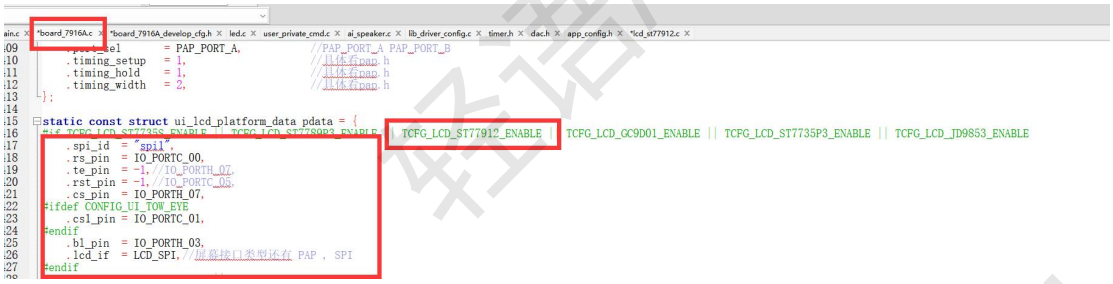
a. 开启 UI 开发板的使能配置



b. 在 app_config.h 文件中开启对应屏幕的使能配置，配置正确的分辨率（16 倍数）



c. 在 board_7916A.c 文件中配置屏幕正确的引脚

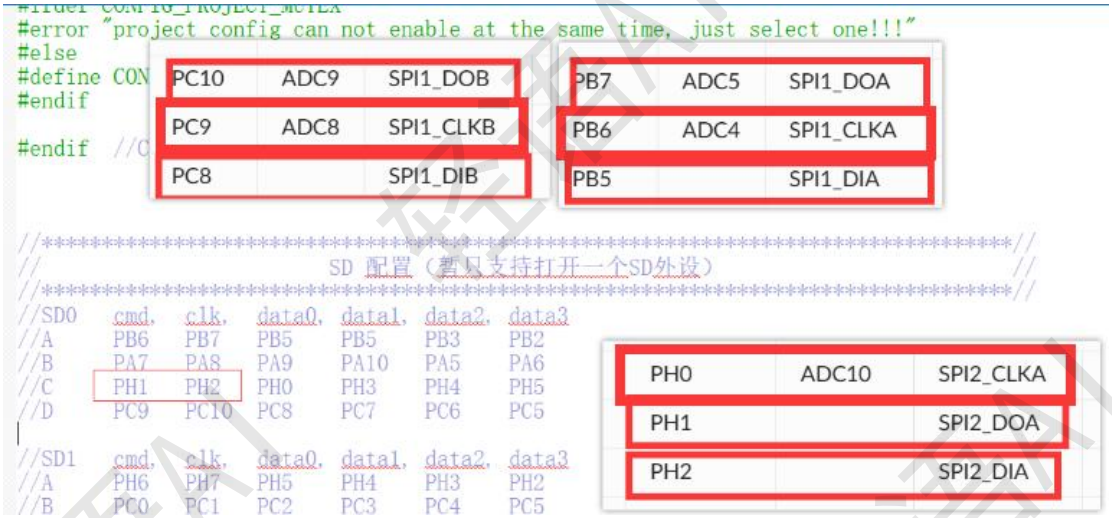



```
/*
 * spi0挂falsh
 */
SPI0_PLATFORM_DATA_BEGIN(spi0_data)
    .clk     = 40000000,
    .mode    = SPI_DUAL_MODE,
    .port    = 'A',
    .attr    = SPI_SCLK_L_UPL_SMPH | SPI_UPDATE_SAMPLE_SAME,
SPI0_PLATFORM_DATA_END()

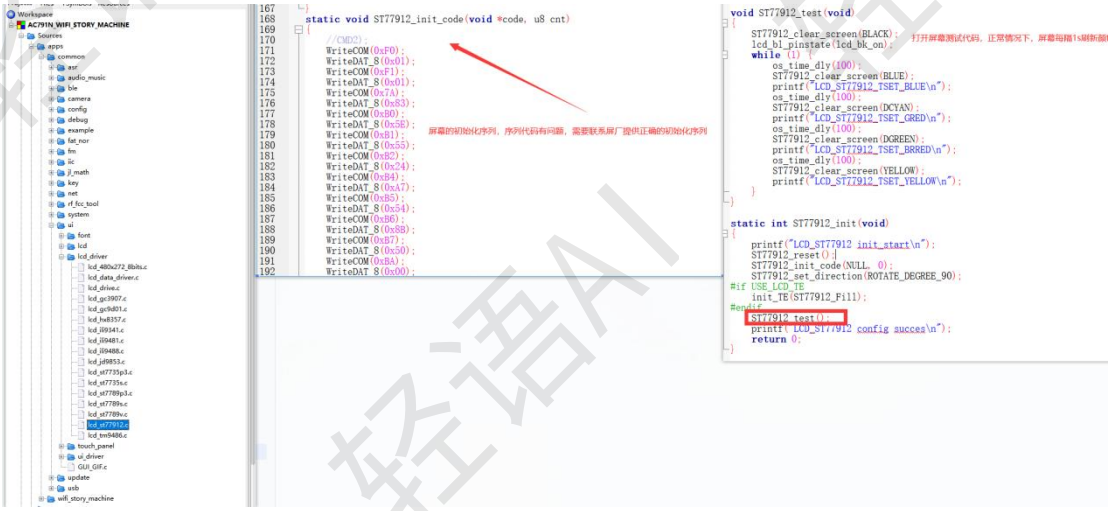
SPI1_PLATFORM_DATA_BEGIN(spi1_data)
    .clk     = 40000000,
    .mode    = SPI_STD_MODE,
    .port    = 'B',
//      .attr = SPI_SCLK_L_UPH_SMPH | SPI_BIDIR_MODE,
//      .attr = SPI_SCLK_L_UPH_SMPL | SPI_BIDIR_MODE,
    .attr    = SPI_SCLK_L_UPL_SMPH | SPI_BIDIR_MODE,
//      .attr = SPI_SCLK_L_UPL_SMPL | SPI_BIDIR_MODE,
//      .attr = SPI_SCLK_L_UPH_SMPH | SPI_BIDIR_MODE,//WS2818 RGB灯
SPI1_PLATFORM_DATA_END()

#ifndef SPI2_CLOCK_HZ
#define SPI2_CLOCK_HZ    500000
#endif
#ifndef SPI2_ATTR_SET
#define SPI2_ATTR_SET    SPI_SCLK_H_UPL_SMPH | SPI_UNIDIR_MODE //单向单线
#endif
SPI2_PLATFORM_DATA_BEGIN(spi2_data)
    .clk     = SPI2_CLOCK_HZ,
    .mode    = SPI_STD_MODE,
    .port    = 'A',
    .attr    = SPI2_ATTR_SET,
SPI2_PLATFORM_DATA_END()
```

- spi 的 LCD_CLK 和 LCD_SDA 是固定的，（使用 spi2 时要关掉 sd 卡——> 有引脚冲突）



d. 在屏幕驱动文件中，打开屏幕测试代码



注意：要留意屏幕接口与开发板上插座的接口顺序完全匹配，经常会有因接口不匹配而造成短路的情况。

14. AVI & GIF 动画显示

a. 完成屏幕测试后，请关闭驱动文件中的屏幕测试代码。

b. 请选择相应功能进行启用，并确认以下设置：是否需要双目模式，是开启 AVI 动画还是 GIF 动画。

```
//                          数芯语-UI-开发板                                    //
//*****************************************************************************//
#if SXY_QYAI_UI_BOARD //数芯语-UI-开发板

#define CONFIG_UI_ENABLE                        //开启UI显示
#define CONFIG_UI_ONLY_EYE                      //只开UI的眼睛显示

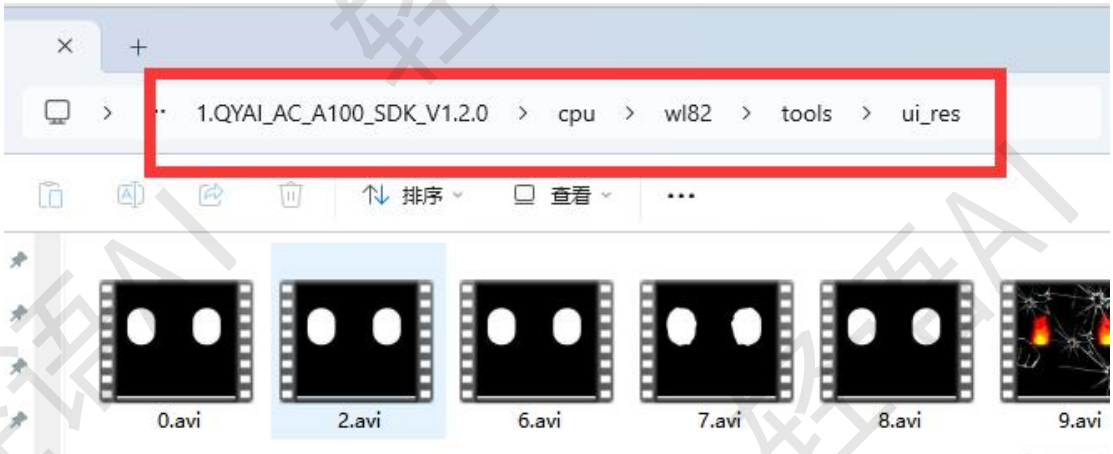
#ifdef CONFIG_UI_ONLY_EYE
#define CONFIG_UI_TOW_EYE                       //开2个眼睛显示
//#define CONFIG_UI_GIF_EYE                       //开启GIF眼睛显示
#define CONFIG_UI_AVI_EYE                       //开启AVI眼睛显示
#ifdef CONFIG_UI_TOW_EYE
#define CONFIG_UI_MIRROR_EYE                    //开启另外眼睛镜像显示
#endif
#endif

#define TCFG_EQ_ENABLE                  0       //0关闭EQ调试功能
```

c. 表情动画调用，调用 play_face_emoji()函数，即可播放动画

```
d) : char
app_main.c | *board_7916A.c | *board_7916A_develop_cfg.h | led.c | user_private_cmd.c | ai_speaker.c | lib_driv
301  #endif
302
303      dac_mute_control(0);//关闭功放静音
304
305  #ifdef CONFIG_UI_ENABLE
306  #ifdef CONFIG_UI_ONLY_EYE//只开UI的眼睛显示
307      extern int play_face_emoji(char index, int gif_index);
308      play_face_emoji(0, 0);
309  #else
310      extern void lvgl_test_demo(void);
311      lvgl_test_demo();
312  #endif
313  #endif
314
```

d. 动画文件地址

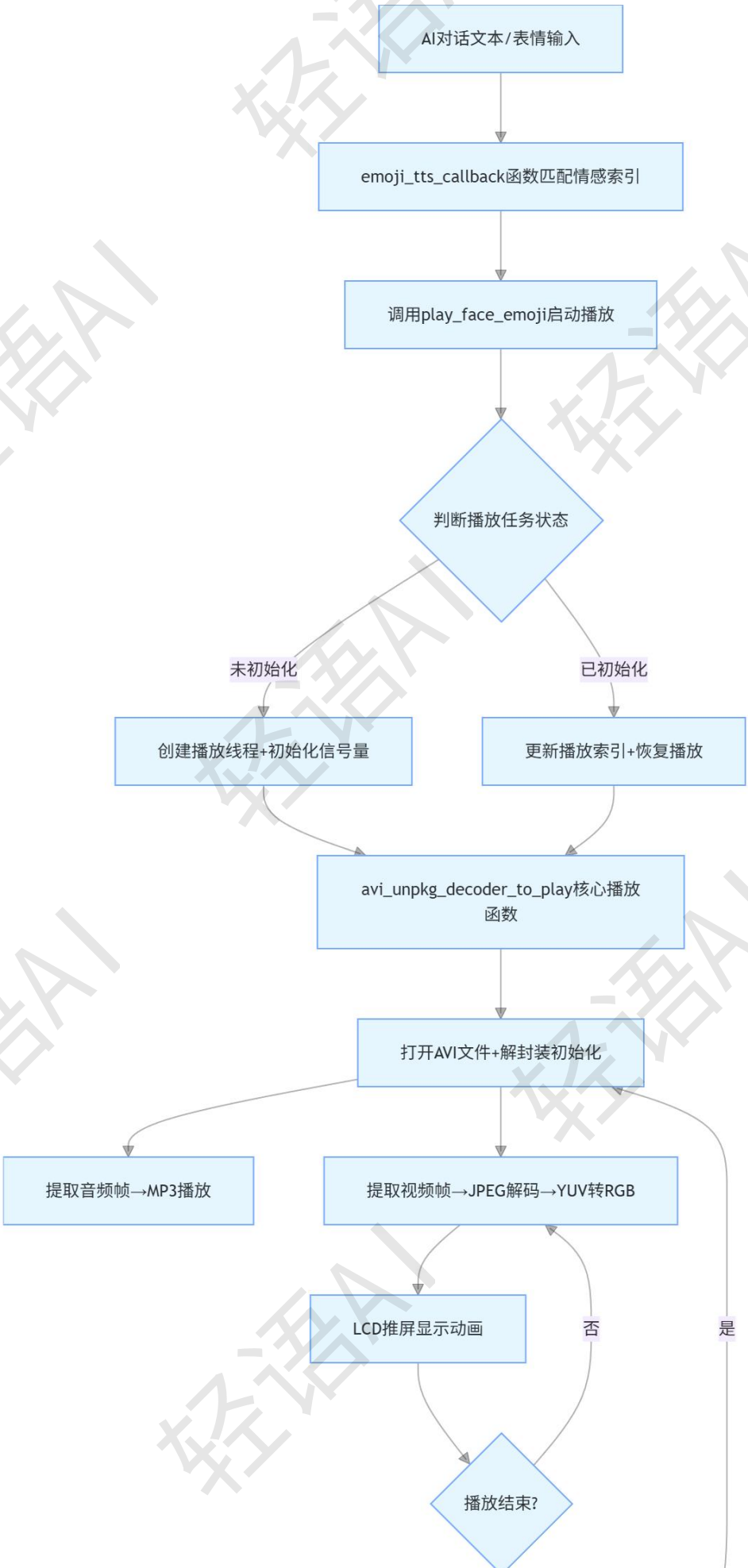


注意：

- 要留意动画文件的命名需和代码里设置的文件名称及路径严格一致；
- 动画文件的分辨率与屏幕一致（仅支持 16 倍数分辨率的动画文件解码）；
- 动画文件仅 1 帧会导致解码失败；

- 串口打印输出 kill 动画文件相关信息时，无法开启下一个动画文件。

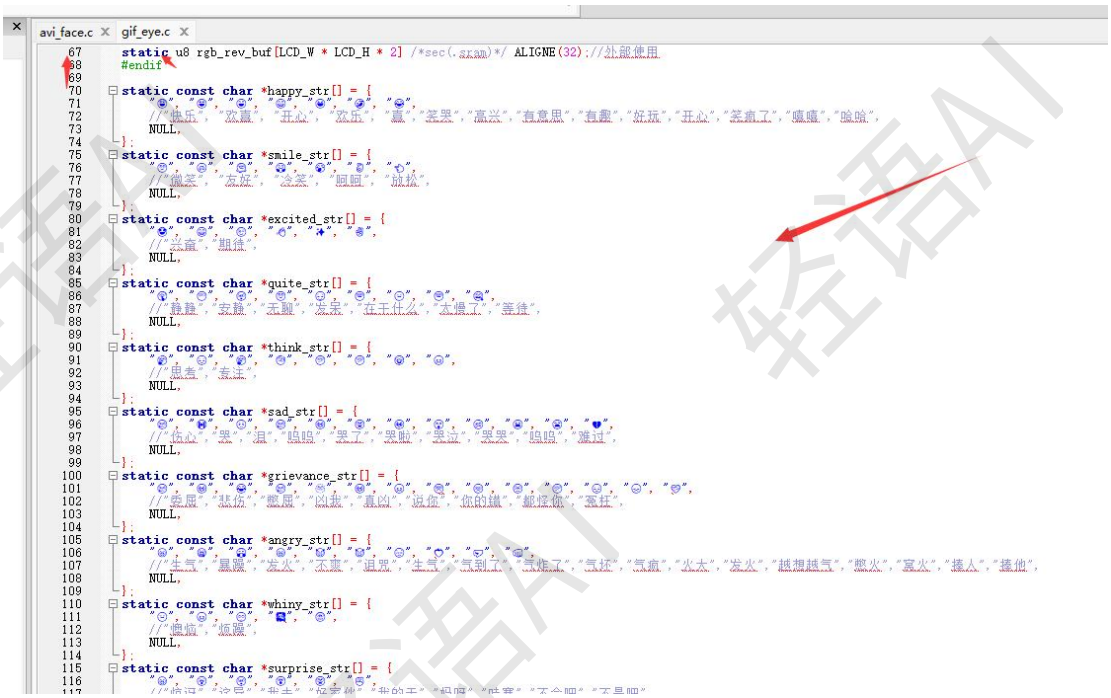
e. **avi_face.c 文件执行逻辑**


AI对话文本/表情输入
emoji_tts_callback函数匹配情感索引
调用play_face_emoji启动播放
判断播放任务状态
未初始化
已初始化
创建播放线程+初始化信号量
更新播放索引+恢复播放
avi_unpkg_decoder_to_play核心播放函数
打开AVI文件+解封装初始化
提取音频帧→MP3播放
提取视频帧→JPEG解码→YUV转RGB
LCD推屏显示动画
否
是
播放结束?

- 压缩视频参考
- ui 资源地址不够请参考

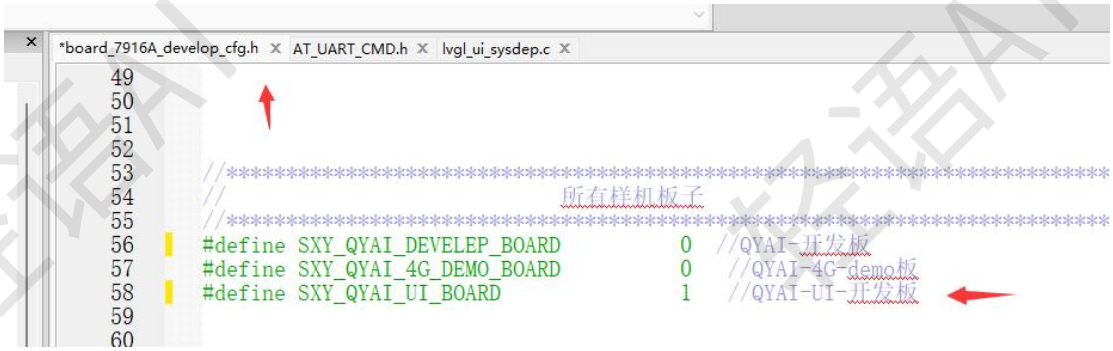
f. **表情触发词修改**

. 在表情映射数组中修改触发关键词与关联表情包，对话里一旦出现匹配的词汇或表情包，就能自动切换为对应的表情样式。



15. LVGL 相关接口说明

15.1 配置 ui 开发板



15.2 打开 lvgl 使能

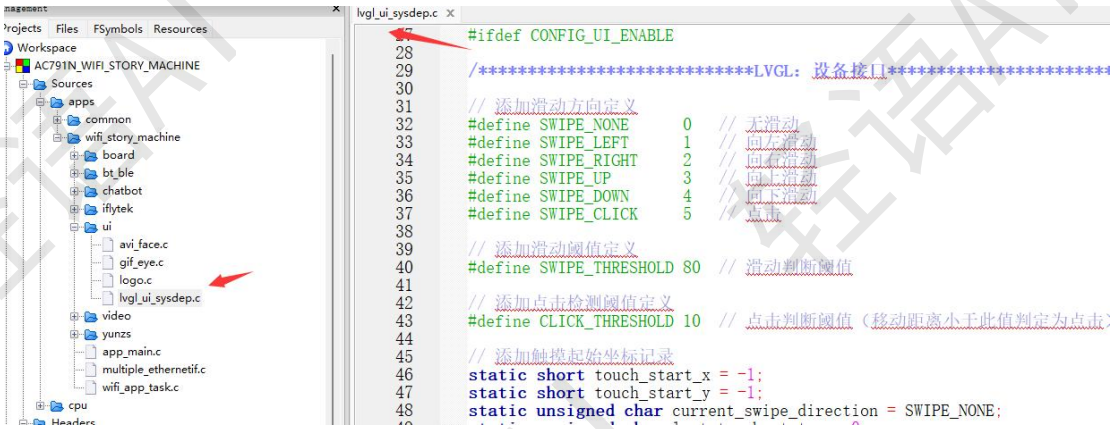
```
*board_7916A_develop_cfg.h × AT_UART_CMD.h × lvgl_ui_sysdep.c ×
227  //*****************************************************************************//
228  //                                数芯语-UI-开发板                              //
229  //*****************************************************************************//
230  #if SXY_QYAI_UI_BOARD //数芯语-UI-开发板
231
232  #define CONFIG_UI_ENABLE                          //开启UI显示
233  #define CONFIG_UI_FONT_RES_SAVE_IN_FLASH          //UI字库存放在flash
234
235  //#define USE_LCD_TE_ROTATE_ENABLE          1 //开启TE使用时播放视频翻转视频数据
236
237  /* CONFIG_LVGL_UI_ENABLE和CONFIG_UI_ONLY_EYE只能开一个*/
238  #define CONFIG_LVGL_UI_ENABLE
239  //#define CONFIG_UI_ONLY_EYE                        //只开UI的眼睛显示
240
241  #ifdef CONFIG_UI_ONLY_EYE
242  #define CONFIG_UI_TOW_EYE                         //开2个眼睛显示
243  //#define CONFIG_UI_GIF_EYE                       //开启GIF眼睛显示
244  #define CONFIG_UI_AVI_EYE                         //开启AVI眼睛显示
245  #ifdef CONFIG_UI_TOW_EYE
246  #define CONFIG_UI_MIRROR_EYE                      //开启另外眼睛镜像显示
247  #endif
248  #endif
```

15.3 lvgl 启动函数

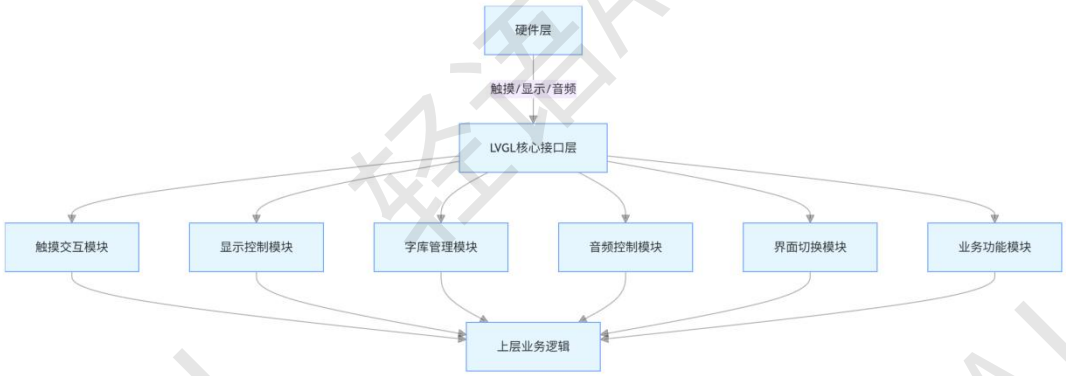
```
app_main.c ×
354      app_usb_at_register_inter();
355  #endif
356
357  #ifdef CONFIG_UI_ENABLE
358  #ifdef CONFIG_UI_ONLY_EYE//只开UI的眼睛显示
359      play_face_emoji(AI_UART_CMD_PWR_ON);
360  #else
361      lvgl_test_demo();
362  #endif
363  #endif
364
```

15.4 lvgl_ui_sysdep.c 文件

i. 基于 LVGL 的嵌入式 UI 控制系统实现，主要面向带屏智能设备，提供了**触摸交互、显示控制、音频管理、界面切换、系统状态展示**等核心功能，衔接硬件驱动与上层业务逻辑。



15.5 核心模块架构



15.6 屏幕相关函数说明

- lvgl_touch_x_y_status_get 触摸数据采集与解析
  - 读取触摸坐标（适配不同触摸芯片）
  - 坐标校准（XY 交换 / Y 轴翻转）
  - 滑动 / 点击状态判断
  - 触发滑动处理函数
- lvgl_handle_page_swipe 滑动 / 点击事件处理
  - 响铃页面滑动时停止闹钟 / 计时器
  - 左滑 / 右滑：预留页面切换接口
  - 上滑：返回主页面
  - 下滑：打开下拉框页面
  - 点击：主页面触发图标点击事件
- lvgl_disp_init 显示屏初始化系统启动时调用，初始化 LCD 硬件
- lvgl_disp_get_width_hight 获取分辨率给 LVGL 提供屏幕尺寸（LCD_W/LCD_H）
- lvgl_disp_flush 显示数据写入将 LVGL 渲染数据刷到 LCD 硬件
- lvgl_flush_enable 异步强制刷新主动触发屏幕刷新（如界面切换后）
- lvgl_flaush_disable 禁用刷新特殊场景（如动画播放）暂停刷新

15.7 页面切换函数说明

- lv_demo_switch_to_main_page 切换到设备主页面，是最基础的页面切换入口

- lv_demo_switch_to_music_page　切换到音乐页面，支持区分网络 / 蓝牙 / SD 卡 / AUX/U 盘音乐类型，特殊处理播客（podcast）类型并跳转至播客页面
- lv_demo_switch_to_timer_page　切换到计时器页面，用于展示和操作计时器功能
- lv_demo_switch_to_alarm_page　切换到闹钟页面，用于展示和管理已设置的闹钟
- lv_demo_switch_to_ring_page　切换到响铃页面（闹钟 / 计时器触发时的专属页面）
- lv_demo_is_main_page　判断当前是否处于主页面，供上层逻辑做条件判断
- lv_demo_is_music_page　判断当前是否处于音乐 / 播客页面，用于音频相关逻辑适配
- lv_demo_is_timer_page　判断当前是否处于计时器页面，用于计时器功能逻辑控制
- lv_demo_is_alarms_page　判断当前是否处于闹钟设置页面，用于闹钟管理逻辑适配
- lv_demo_is_ring_page　判断当前是否处于闹钟 / 计时器响铃页面，是响铃联动逻辑的核心判断依据

15.8 闹钟 / 计时器模块

- lv_demo_alarms_ring_clean　停止闹钟 / 计时器的响铃播放，并自动切换回主页面
- lv_demo_alarms_add_flush_from_server　根据服务器下发的时间参数新增闹钟，并刷新闹钟页面展示
- lv_demo_alarms_del_flush_from_server　根据指定时间删除对应闹钟，并刷新闹钟页面展示
- lv_demo_timer_add_sec　添加指定秒数的计时器（支持番茄钟 / 普通计时器两种模式）

15.9 AI 对话模块

- lv_demo_ai_dialogue_stt　将用户语音识别（STT）后的文本显示在 AI 对话页面
- lv_demo_ai_dialogue_tts　将 AI 生成的回复（TTS）文本显

示在 AI 对话页面

- lv_demo_ai_dialogue_start　　启动 AI 对话流程，显示唤醒提示文本（如 “在呢”/“正在聆听...”），展示聆听状态图标，并切换到对话页面
- lv_demo_ai_dialogue_speak_listen　　控制 AI 聆听状态图标的显示 / 隐藏（1 显示、0 隐藏）
- lv_demo_ai_dialogue_start_mode　　预留接口，用于标记 AI 唤醒模式（如语音唤醒 / 按键唤醒），当前仅返回 0 无实际逻辑

16. 天气、时间和星期访问接口

16.1 获取天气数据

```
int fetch_weather_data(void)
{
    int ret = -1;
    char *buf = (char *)malloc(2048);
    if (buf == NULL) {
        printf("mem alloc err\n");
        return ret;
    }
    memset(buf, 0, 2048);

    // 1. 检查网络连接状态
    if (!sys_connect_net_success()) {
        free(buf);
        return ret;
    }

    // 2. 获取天气数据
    int result = qyai_weather_forecast_get(NULL, 0, buf, 2048);

    // 3. 处理结果
    if (result <= 0) {
        if (result == -1) {
            printf("[天气错误] 未绑定账号\n");
        } else {
            printf("[天气错误] 获取天气数据失败。\n");
        }
    } else {
        printf("\n%s\n", buf);
        // 4. 解析JSON
        ret = update_weather_ui_from_json(buf);
        if (ret) {
            printf("[天气错误] 获取天气数据异常\n");
        }
    }
    //printf("========== 天气数据获取结束 ==========\n\n");
    free(buf);
    return ret;
}
```

16.2 获取时间&星期接口

```
// 外部函数声明 - 来自alarm.c
extern uint8_t sys_time_to_weekday(struct sys_time *t);
// 外部函数声明
extern int get_sys_time(struct sys_time *t);  // 系统时间获取函数
```

第四阶段：支持外设

1. 外设硬件基础与适配

1.1 对话流水灯 —— 任务创建

1.1.1 流水灯函数定义

```
C++
// 流水灯状态变量
static int liushui_step = 0;
extern bool liushui_running = false;
// 流水灯开启函数
// 流水灯开启函数
static void led_liushui_shown(void *param)
{
    int direction = 1;  // 方向控制：1 表示正向(1→2→3)，-1 表示反向(3→2→1)

    while(1){
        // 更新 LED 状态
        gpio_direction_output(TCFG_LED_LIUSHUI_PORT1, liushui_step == 0 ? 0 : 1);
        gpio_direction_output(TCFG_LED_LIUSHUI_PORT2, liushui_step == 1 ? 0 : 1);
        gpio_direction_output(TCFG_LED_LIUSHUI_PORT3, liushui_step == 2 ? 0 : 1);


        // 根据方向更新步进
        liushui_step += direction;

        // 到达边界时改变方向
        if(liushui_step >= 2) {
            direction = -1;  // 到达 3 号灯，开始反向
```

```
        } else if(liushui_step <= 0) {
            direction = 1;   // 到达 1 号灯，开始正向
        }

        printf("liushui_step = %d\n", liushui_step);

printf("=========================================\n");
        os_time_dly(20);
    }
}
// 流水灯打开函数
void led_liushui_open()
{
    os_task_create(led_liushui_shown, NULL, 10, 1600, 64,
LED_LIUSHUI_SHOWN);
}

// 流水灯关闭函数
void led_liushui_close(void *param)
{
    os_task_del(LED_LIUSHUI_SHOWN);
    leds_liushui_close();
}
```

1.1.2 流水灯函数调用

```
}
extern bool liushui_running;
int net_ai_dialogue_txt_recv_callback(char *buf, int len)//网络对话文本回调函数
{
    printf("resp_txt : %s \n",buf);
    my_bt_music_play_set(1);
    //put_buf(buf, len);
#ifdef AI_UART_CMD_CTROL_ENABLE
    ai_uart_cmd_data_push(AI_UART_CMD_TTS, buf, len);
    int emoji_tts_uart_cmd_callback(char *emoji);
    emoji_tts_uart_cmd_callback(buf);
#endif

    user_emoji_check(buf, len);//UI显示的在这行下面处理（已去掉表情图标）
#ifdef CONFIG_UI_ENABLE
#if (defined CONFIG_UI_ONLY_EYE)
    extern int emoji_tts_callback(char *emoji);
    emoji_tts_callback(buf);
#endif
#endif
#if TCFG_LED_STATUES_EBANLE
    if(!liushui_running){
        liushui_running = true;
        extern void led_liushui_open(void);
        led_liushui_open();
    }
#endif

    return 0;
}
```

说明有对话时打开呼吸灯，多次对话时在关闭呼吸灯任务前仅打开一次

```
int ai_audio_play_end_callback(void)
{
    if(user_close_status){
        extern int websockets_dialogue_timeout_init(int time_out, char use_voice_note);
        websockets_dialogue_timeout_init(0, 0);//立马退出对话
    }
#ifdef AI_UART_CMD_CTROL_ENABLE
    ai_uart_cmd_data_push(52, NULL, 0);
#endif
#if TCFG_LED_STATUES_EBANLE
    if(liushui_running){
        liushui_running = false;
        extern void led_liushui_close(void);
        led_liushui_close();
    }
#endif
    return 0;
}
```

对话结束时关闭呼吸灯

1.2 拍打传感器（任务循环，中断触发）

1.2.1 拍打传感器，拍打时短路，其他时候断路。

1.2.2 任务创建：

```
static void shake_demo_task(void *p)
{
    uint8_t current_state;
    int shake_state = gpio_read(SHAKE_PORT);
    gpio_set_pull_down(SHAKE_PORT,0);
    gpio_set_pull_up(SHAKE_PORT,1);
    // 初始化随机数种子
    srand(time(NULL));
    extern int play_face_emoji(char index, int avi_index);
    while (1) {
        // 读取传感器状态

        current_state  = gpio_read(SHAKE_PORT);
                // 检测到由非零变化（高->低 or 低->高）
        if (current_state == 0) {
            printf("检测到拍打动作 \r\n");
            play_face_emoji(0, 50);
            os_time_dly(300);
            // 生成51到54之间的随机整数

            int emoji_value = rand() % 4 + 51;

            play_face_emoji(0, emoji_value);
            os_time_dly(150);
            play_face_emoji(0, 0);
            printf("=============================================================");
            printf("==================emoji_value = %d==========================\r\n",emoji_value);
            printf("=============================================================");
//            shakex_state = current_state;  // 更新状态
        }
//        // 根据状态输出信息
//        if (current_state == 0) {  // 假设低电平表示短路状态
//            printf("震动检测到 - 短路状态 \r\n");
//        } else {  // 高电平表示开路状态
//            printf("静止 - 开路状态 \r\n");
//        }

    }
}
    printf("\r\n\r\n\r\n\r\n\r\n\r\n ------------shake demo run %s------------\r\n\r\n\r\n\r\n\r\n\r\n\r\n",__func__);
    os_task_create(shake_demo_task, NULL, 10, 1000, 0, "shake_demo_task");
```

1.2.3 中断：

```
static void PA1IntHandler(void)
{
  printf("enter irq");

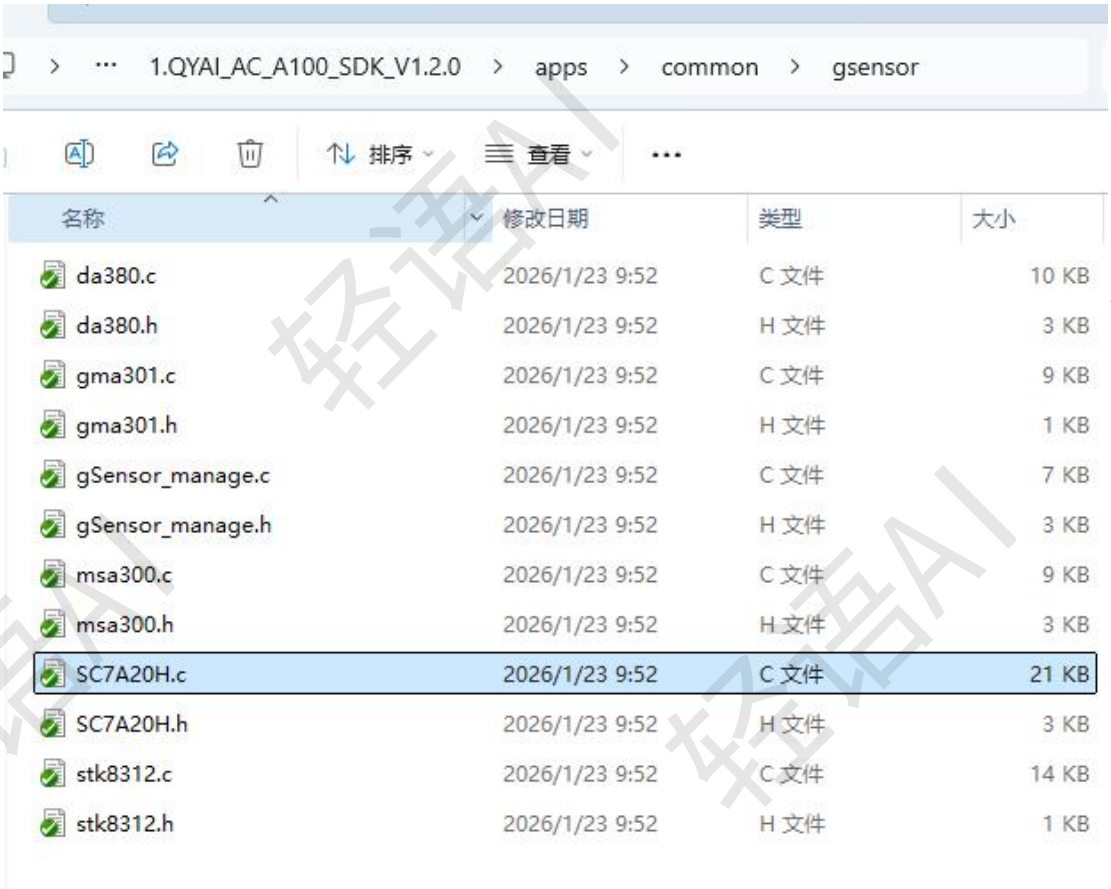
}

// 初始化函数，用于配置GPIO和注册中断
void init_interrupt(void)
{

   gpio_direction_input(IO_PORTA_01);
   gpio_set_pull_down(IO_PORTA_01, 1);
   // gpio中断模式使用（i/o口作外部中断，外部中断引脚，上升沿触发，中断响应函数）
   port_wakeup_reg(EVENT_IO_0, IO_PORTA_01, EDGE_POSITIVE, PA1IntHandler);
}
```

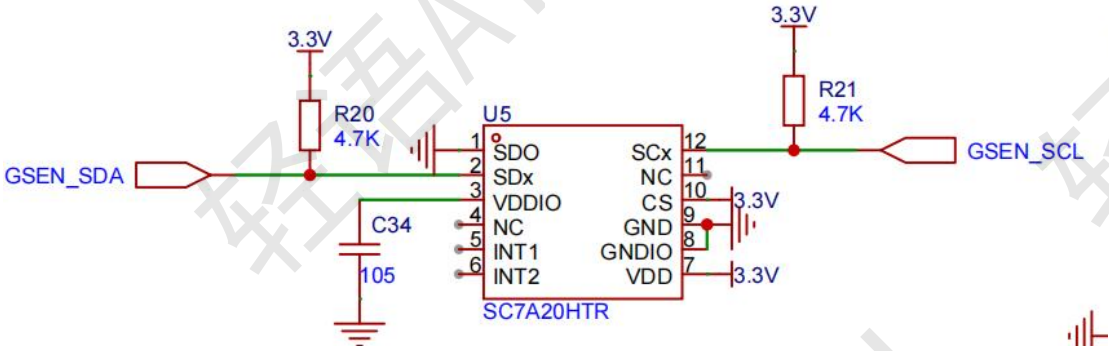
1.3 IMU 陀螺仪

1.3.1 开发板支持的陀螺仪如下

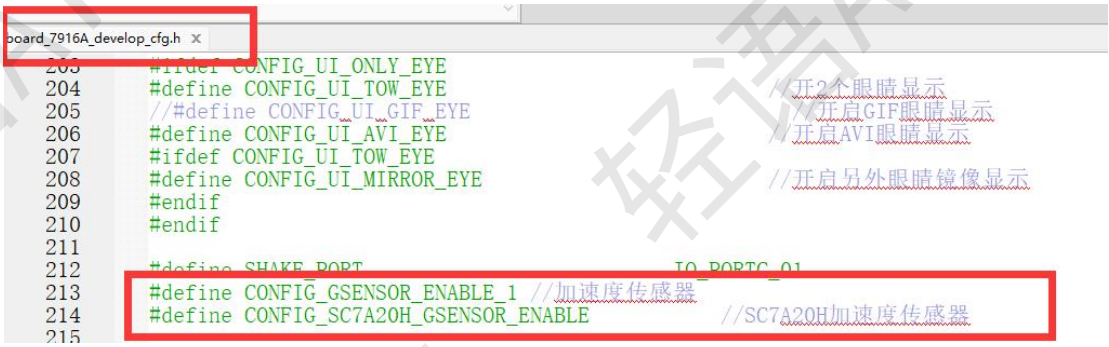


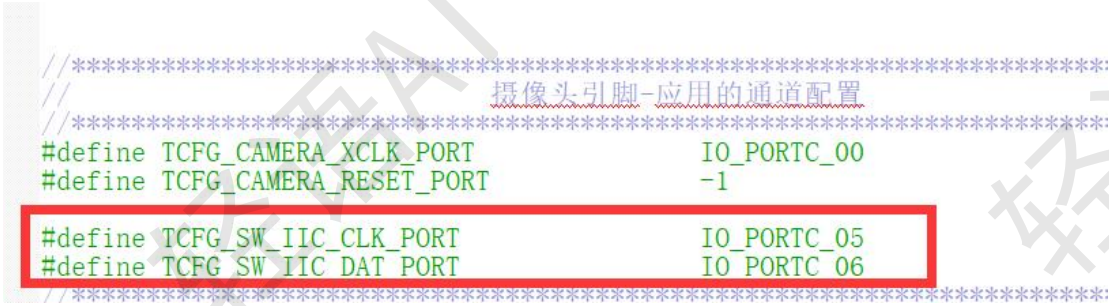
1.3.2 SC7A20

- 原理图：

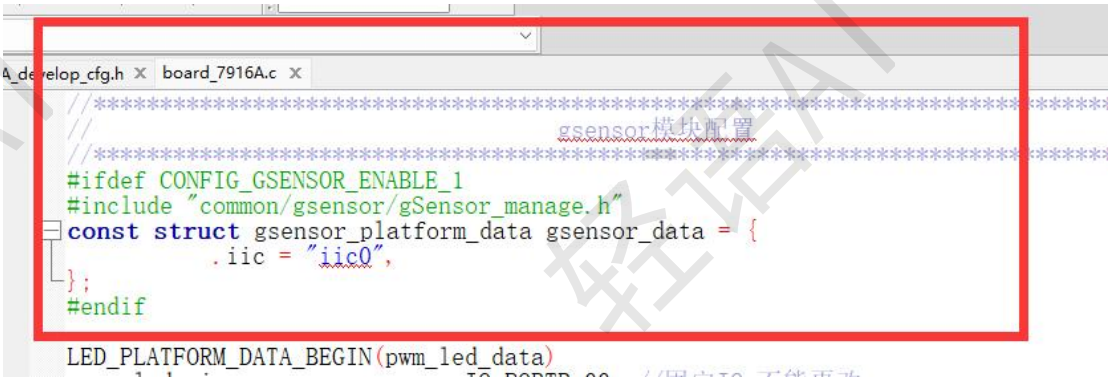


- 打开加速度传感器的使能宏，配置 IIC 通道引脚





- 在 board...c 文件中进行陀螺仪的相关配置



velop_cfg.h × board_7916A.c ×

```
#endif
/*****************SD模块**********************/

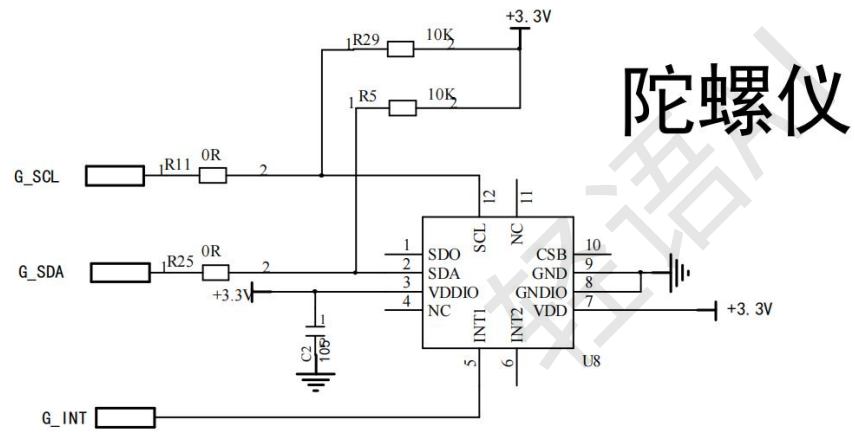

HW_IIC0_PLATFORM_DATA_BEGIN(hw_iic1_data)
    .clk_pin = IO_PORTH_00,
    .dat_pin = IO_PORTH_01,
    .baudrate = 0x3f,
HW_IIC0_PLATFORM_DATA_END()

SW_IIC_PLATFORM_DATA_BEGIN(sw_iic0_data)
    .clk_pin = TCFG_SW_IIC_CLK_PORT,
    .dat_pin = TCFG_SW_IIC_DAT_PORT,
    .sw_iic_delay = 50,
SW_IIC_PLATFORM_DATA_END()
```

board_7916A_develop_cfg.h × board_7916A.c ×

```
764         .block_offset_start = (CONFIG_SDNAND_HFS_LEN + CONFIG_SDNAND_HFA
765         .block_offset_end = (CONFIG_SDNAND_HFS_LEN + CONFIG_SDNAND_HFAT_
766     };
767     #endif
768
769     REGISTER_DEVICES(device_table) = {
770     #ifdef CONFIG_UI_ENABLE
771         { "pap",    &pap_dev_ops,  (void *)&pap_data },
772         { "emi",    &emi_dev_ops,  (void *)&emi_data },
773         { "imd",    &imd_dev_ops,  NULL },
774         { "iic0",   &iic_dev_ops,  (void *)&sw_iic0_data },
775         { "iic1",   &iic_dev_ops,  (void *)&hw_iic1_data },
776         {"gsensor", &gsensor_dev_ops, (void *)&gsensor_data},
777     #endif
```

[gsensor.zip]



- SDO：低电平

```
#define _SC7A20H_H

#define WRITE_COMMAND_FOR_SC7A20H     0x30 // I2C写入命令 0x32
#define READ_COMMAND_FOR_SC7A20H      0x31 // I2C读取命令
```

- SDO：高电平/悬空

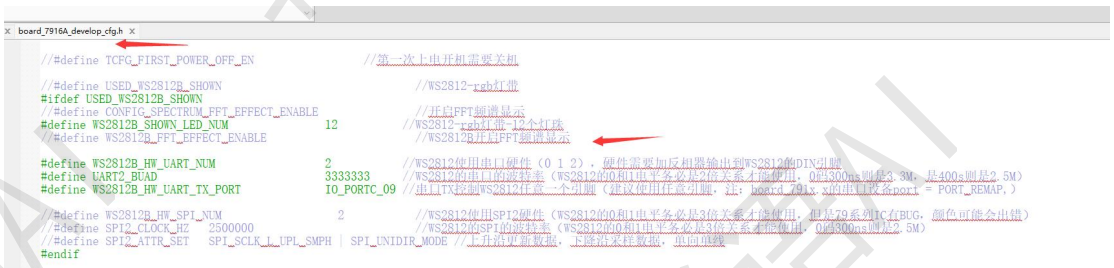
```
1   #ifndef _SC7A20H_H
2   #define _SC7A20H_H
3
4   #define WRITE_COMMAND_FOR_SC7A20H       0x32  // I2C写入命令  0x32
5   #define READ_COMMAND_FOR_SC7A20H        0x33  // I2C读取命令
6
7   // 低通滤波频率定义
8   #define LPF_CUTOFF_800HZ  0x00  // CTRL_REG2: 00xx xxxx
```

1.4 RGB & LED

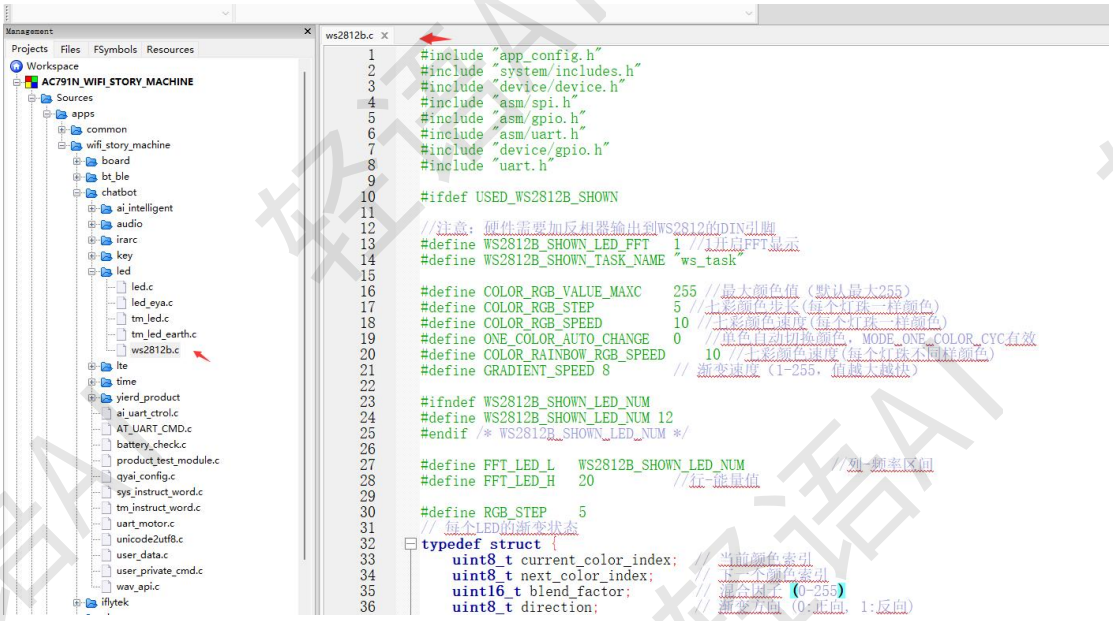
1.4.1 WS2812B RGB LED 灯带 的驱动控制程序，支持通过 UART 或 SPI 接口驱动灯带，并实现了多种动态显示效果（如彩虹渐变、流水灯、音乐频谱 FFT 显示、呼吸灯等）。

1.4.2 开启相关宏定义

- **硬件接口选择**：通过宏 WS2812B_HW_UART_NUM 选择 UART/SPI 作为驱动接口（UART 需加反相器，SPI 直接驱动）。
- **LED 参数配置**：
- WS2812B_SHOWN_LED_NUM：灯带 LED 数量（默认 12 个）。



1.4.3 具体函数实现文件位置如下



1.4.4 显示效果实现（核心功能）

- **静态彩虹渐变**
  - ws2812_rainbow_static 灯带从左到右固定显示红→黄→绿→青→蓝→紫的渐变，无动态偏移。
- **流水彩虹**
  - ws2812_rainbow_flow 彩虹渐变整体向右偏移，形成 “流水” 效果，偏移量 offset 循环递增。
- **全灯渐变（逐色过渡）**
  - ws2812_update_all_gradient 每个 LED 独立从当前颜色渐变到下一个颜色，整体保持彩虹分布。
- **单色全局渐变**
  - ws2812_shown_one_color_gradient 所有 LED 同步从红→黄→绿→青→蓝→紫循环渐变（如：红→黄时，绿色逐步增加）。
- **单色自动循环**
  - ws2812_shown_one_color_auto 所有 LED 固定显示一种颜色，可自动切换颜色（红→黄→绿→…→红），支持亮度调节。
- **音乐 FFT 频谱显示**
  - ws2812_shown_fft 读取音频 FFT 数据，将不同频率区间的能量值映射为 LED 亮度 / 颜色，支持流水频谱。

- **额外效果：**
  - ws2812_shown_face_breath_led：呼吸灯效果（亮度从高到低渐变，颜色循环切换）。
  - ws2812_shown_clear：清空所有 LED。

1.4.5 核心任务 ws_shown_task

- 初始化：创建互斥锁、硬件初始化、清空灯带、默认效果配置。
- 主循环逻辑：
  - 非 FFT 模式：循环调用当前选中的效果函数（如彩虹流水）。
  - FFT 模式：读取音频 FFT 数据（网络 / 蓝牙 / USB/SD/AUX 音乐），映射为频谱显示；无音频时触发呼吸灯 / 自动模式。
  - 任务队列处理：响应外部指令（如笑脸显示、自动模式开关）。

1.4.6 对外 API（应用层调用）

- ws2812_shown_mode(mode, percent)　设置显示模式（如流水彩虹），percent 仅对单色循环模式生效（亮度 0-100）。
- ws2812_shown_one_color_mode()　手动设置单色模式的颜色（如固定显示红色）。
- ws2812_shown_one_color_auto_mode()　手动切换单色循环的颜色（如从红切到黄）。
- ws2812_shown_face(on)　触发笑脸显示效果（随机亮度闪烁）。
- ws2812_shown_auto_led(on)　开启 / 关闭自动模式（呼吸灯 / 彩虹效果）。

1.5 NFC（更新中……）

1.6 屏幕

· 开发板支持的屏幕类型和触摸芯片如下

app_config.h

```
812
813     #if TCFG_LCD_ENABLE
814     #define TCFG_LCD_ILI9481_ENABLE             0
815     #define TCFG_LCD_ILI9341_ENABLE             0
816     #define TCFG_LCD_ST7789S_ENABLE             0
817     #define TCFG_LCD_480x272_8BITS              0
818     #define TCFG_LCD_ST7735S_ENABLE             0
819     #define TCFG_LCD_ST7789V_ENABLE             1 //2.1寸320*240屏幕
820     #define TCFG_LCD_JD9853_ENABLE              0
821     #define TCFG_LCD_GC3907_ENABLE              0
822
823     #define TCFG_LCD_ST7789P3_ENABLE            0 //1.5寸手表屏幕
824     #define TCFG_LCD_ST77912_ENABLE             0 //1.28寸手表屏幕
825     #define TCFG_LCD_GC9D01_ENABLE              0 //0.71寸手表屏幕
826     #define TCFG_LCD_ST7735P3_ENABLE            0 //1.77寸160*128屏幕
827     #define TCFG_LCD_NV3031A_ENABLE             0 //2.8寸320*240屏幕
828     #endif
829
830     #if TCFG_LCD_NV3031A_ENABLE && (!defined TCFG_TOUCH_GST820_ENABLE)
831     #define TCFG_TOUCH_GST820_ENABLE            1 //触摸
832     #elif TCFG_LCD_ST7789V_ENABLE && (!defined TCFG_TOUCH_GST328_ENABLE)
833     #define TCFG_TOUCH_GST328_ENABLE            1 //触摸
834     #endif
835
836
837     #if TCFG_LCD_ILI9341_ENABLE
838     #define TCFG_TOUCH_GT911_ENABLE             1
839     #else
840     #define TCFG_TOUCH_GT911_ENABLE             0
841     #endif
842
843     #if TCFG_LCD_ILI9481_ENABLE
844     #define TCFG_TOUCH_FT6236_ENABLE            1
845     #else
846     #define TCFG_TOUCH_FT6236_ENABLE            0
847     #endif
```

2. 时钟与显示功能开发与配置

2.1 时钟、12-24 小时制

[pengzhihao_htc1740.c]

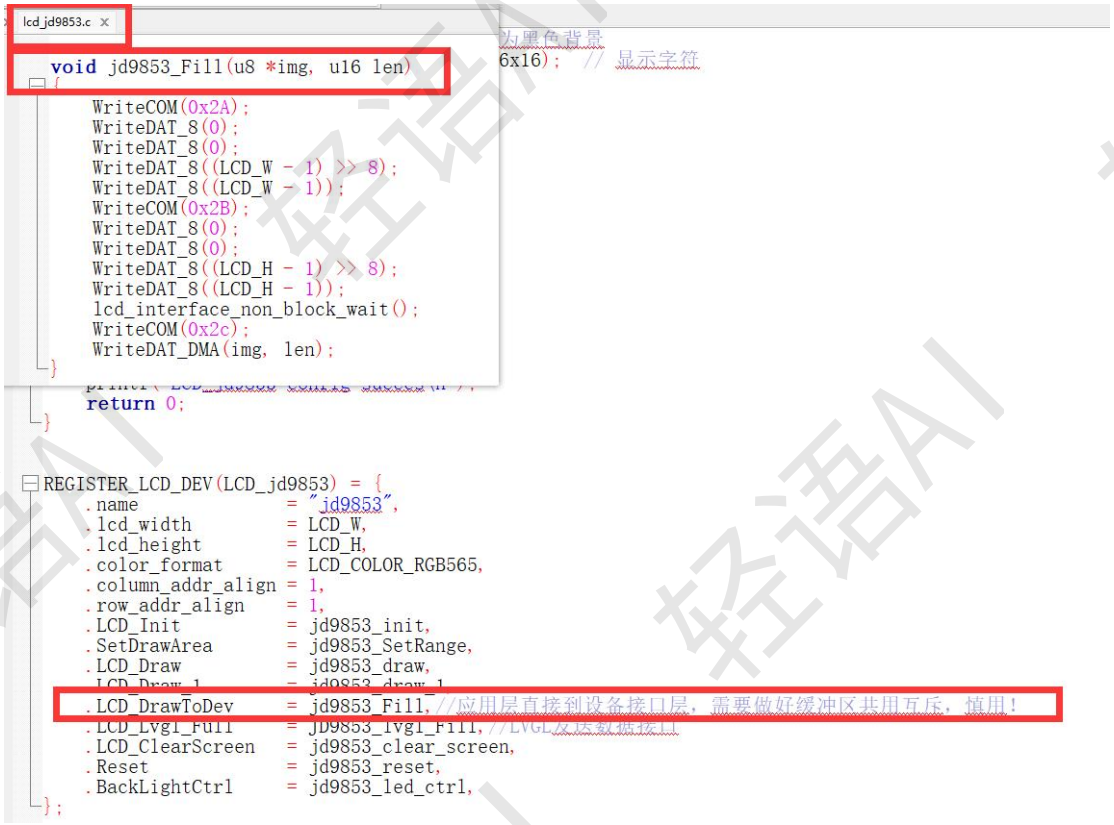
```
//    for(i = 0; time_format_24h[i] != NULL; i++){//24小时制
//        if(strcmp(time_format, time_format_24h[i]) == 0){
//            time_format_24h_flag = true;
//            goto exit;   字符串匹配严格，多标点符号或者空格会导致匹配失败
//        }
//    }
//
//    for(i = 0; time_format_12h[i] != NULL; i++){//12小时制
//        if(strcmp(time_format, time_format_12h[i]) == 0){
//            time_format_24h_flag = false;
//            goto exit;
//        }
//    }
    for(i = 0; time_format_24h[i] != NULL; i++){//24小时制
        if(strstr(time_format, time_format_24h[i])){
            time_format_24h_flag = true;
            goto exit;
        }
    }

    for(i = 0; time_format_12h[i] != NULL; i++){//12小时制
        if(strstr(time_format, time_format_12h[i])){
            time_format_24h_flag = false;
            goto exit;
        }
    }
```

2.2 avi_lvgl 分时显示

需要注意的是，LVGL 采用的是局部刷新机制 —— 仅当屏幕内容发生变化时，才会对变化的部分进行刷新，而非全屏刷新。在从 AVI 播放状态切换到 LVGL 页面时，需要执行两个关键操作以确保切换正常：

- 首先必须结束 AVI 播放任务，避免两个显示任务产生冲突；
- 结束 AVI 任务后，需强制触发一次屏幕全量刷新（而非依赖 LVGL 默认的局部刷新），确保 LVGL 页面能完整、正确地显示在屏幕上，避免出现画面残留或显示不全的问题。
- 在 avi_face.c 文件中实现 avi 和 lvgl 任务的切换，实现 avi 动画的循环与不循环播放。
- lvgl 屏幕全部重刷：



- 具体实现参考代码 lvgl_main.c 和 avi_face.c

3. 音频与多媒体功能调试优化

3.1 蓝牙 & 音乐律动

- 关闭蓝牙电话功能，以解决蓝牙播放音乐时若有通话接入，两种音频（音乐与通话音）同时叠加播放的问题；
- 在此基础上，实现 LED 灯与 WS2812 灯带仅随音乐节奏单独进行律动闪烁的效果。
- **Ws2812 效果：**效果分成 Ai 对话，蓝牙播放，待机三种状态。即：Ai 对话时可以随说话的声音起伏闪动，蓝牙播放时随音乐律动，待机时多色自由呼吸状态。
- **尚未解决的问题：**由于开发板硬件限制，无法输出准确的 SPI 信号来驱动 WS2812，导致 WS2812 的渐变效果无法实现。
- 蓝牙已断开：

```
lop_cfg.h × board_7916A.c × SC7A20H.c × *ifly_wake_demo.c × ai_speaker.c × yunzs_wake_demo.c × bt_music.c ×
        start_app(&it);
        bt_connecting_flag = 1;

        handle_connect_successl_accumulation_and_enable();


        break;
    case BT_STATUS_FIRST_DISCONNECT:
    case BT_STATUS_SECOND_DISCONNECT:
        log_info("BT_STATUS_DISCONNECT");

        bt_status = false;
          #ifdef CONFIG_UI_ENABLE
          #ifdef CONFIG_UI_ONLY_EYE//只开UI的眼睛显示
              extern int play_face_emoji(char index, int avi_index);
              play_face_emoji(0, 85);
          #endif
          #endif
        struct application *app = get_current_app();
        if(app && (!strcmp(app->name, "bt_music"))){//蓝牙播放模式下延时
            music_play_res_file("BtDisc.mp3");
            #ifdef CONFIG_UI_ENABLE
            #ifdef CONFIG_UI_ONLY_EYE//只开UI的眼睛显示
                extern int play_face_emoji(char index, int avi_index);
                play_face_emoji(0, 85);
            #endif
            #endif
        }
```

- 蓝牙连接成功：

```
evelop_cfg.h × board_7916A.c × SC7A20H.c × *ifly_wake_demo.c × ai_speaker.c × yunzs_wake_demo.c × bt_music.c ×
}

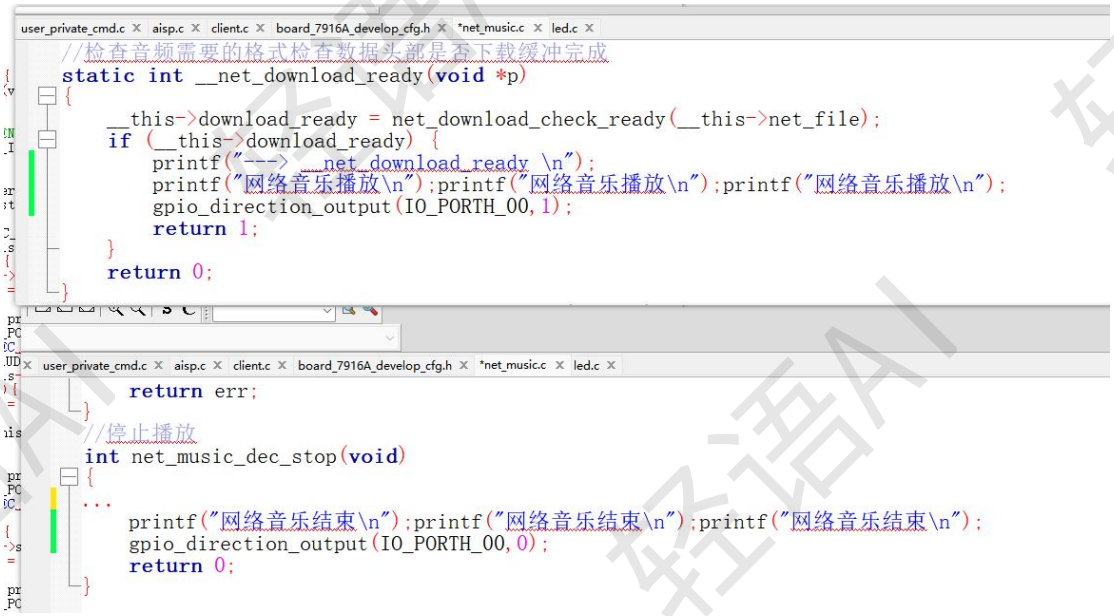
static int bt_music_state_machine(struct application *app, enum app_state state, struct it
{
    switch (state) {
    case APP_STA_CREATE:
        break;
    case APP_STA_START:
        switch (it->action) {
            case ACTION_HOME_MAIN:;
                bt_music_volume_init();
                bt_connection_enable();
                break;
            case ACTION_MUSIC_PLAY_VOICE_PROMPT:
                if (!strcmp((const char *)it->data, "BtSucceed.mp3")) {
                    music_play_res_file("BtSucceed.mp3");
                    __this->media_pause = 0;
                    bt_status = true;
                #ifdef CONFIG_UI_ENABLE
                #ifdef CONFIG_UI_ONLY_EYE//只开UI的眼睛显示
                    extern int play_face_emoji(char index, int avi_index);
                    play_face_emoji(0, 84);
                    os_time_dly(250);
                    extern int play_face_emoji(char index, int avi_index);
                    play_face_emoji(0, 82);
                #endif
                #endif
```

- 蓝牙连接失败：



3.2 有音频时开灯，无音频时关灯

3.2.1 网络音乐开始与停止播放



3.2.2 网络音乐播放暂停

```
play.c | user_private_cmd.c | aisp.c | client.c | board_7916A_develop_cfg.h | *net_music.c | led.c
10      }
11      int net_music_set_dec_volume(int volume);
12      static int net_music_dec_play_pause(u8 notify)
13      {
14          union audio_req r = {0};
15          union ai_req req  = {0};
16          int err = 0;
17
18          if (!__this->net_file || !__this->dec_server) {
19              __this->play_status = 0;
20              return -1;
21          }
22          int volume = 0;
23          sys_volume_read(&volume);
24          if(__this->volume != volume){
25              net_music_set_dec_volume(volume);
26          }
27
28      #ifdef CONFIG_DEC_ANALOG_VOLUME_ENABLE
29          r.dec.attr = AUDIO_ATTR_FADE_INOUT;
30      #endif
31          r.dec.cmd = AUDIO_DEC_PP;
32          server_request(__this->dec_server, AUDIO_REQ_DEC, &r);
33          __this->play_status = r.dec.status;
34
35          if (r.dec.status == AUDIO_DEC_START) {//播放状态
36              net_download_set_pp(__this->net_file, 0);//播放状态
37              if(__this->suspend_time){
38                  sys_timer_del(__this->suspend_time);
39                  __this->suspend_time = 0;
40              }
41              printf("网络音乐继续\n");printf("网络音乐继续\n");printf("网络音乐继续\n");
42              gpio_direction_output(IO_PORTH_00,1);
43              printf("======== AUDIO_DEC_STAR\n");
44          } else if (r.dec.status == AUDIO_DEC_PAUSE) {//暂停状态
45              net_download_set_pp(__this->net_file, 1);//暂停播放
46              if(!__this->suspend_time){
47                  __this->suspend_time = sys_timeout_add(NULL, net_music_dec_play_pause_tolong, 20*60*1000);//20分钟暂停则关闭
48              }else{
49                  sys_timer_modify(__this->suspend_time, 20*60*1000);//20分钟暂停则关闭
50              }
51              err = 1;
52              printf("网络音乐暂停\n");printf("网络音乐暂停\n");printf("网络音乐暂停\n");
53              gpio_direction_output(IO_PORTH_00,0);
54              printf("======== AUDIO_DEC_PAUSE\n");
55          } else {
56              if(__this->suspend_time){
57                  sys_timer_del(__this->suspend_time);
58                  __this->suspend_time = 0;
59              }
60              printf("网络音乐错误\n");printf("网络音乐错误\n");printf("网络音乐错误\n");
61              gpio_direction_output(IO_PORTH_00,0);
62              printf("======== AUDIO_DEC err = %d\n",r.dec.status);
63          }
64          if (notify && __this->ai_server && r.dec.status != AUDIO_DEC_STOP) {
65              //notify : 是否需要通知云端暂停
66              req.evt.event    = AI_EVENT_PLAY_PAUSE;
67              req.evt.ai_name = __this->ai_name;
```

3.2.3 蓝牙音乐播放与暂停：

```
.c | user_private_cmd.c | aisp.c | client.c | board_7916A_develop_cfg.h | net_music.c | led.c | bt_music.c | bt.c | ai_speaker.c
        case SYS_BT_EVENT:
            return app_music_bt_event_handler(event);
        default:
            return false;
        }
    }

    static int bt_music_state_machine(struct application *app, enum app_state state, struct intent *it)
    {
        switch (state) {
        case APP_STA_CREATE:
            break;
        case APP_STA_START:
            switch (it->action) {
                case ACTION_HOME_MAIN:;
                    bt_music_volume_init();
                    bt_connection_enable();
                    break;
                case ACTION_MUSIC_PLAY_VOICE_PROMPT:
                    if (!strcmp((const char *)it->data, "BtSucceed.mp3")) {
                        __this->bt_conncted = true;
                        music_play_res_file("BtSucceed.mp3");
                        os_time_dly(300);
                        printf("-======================-\n");
                        printf("-======BtSucceed=========-\n");
                        gpio_direction_output(IO_PORTH_00,1);
                    } else if (!strcmp((const char *)it->data, "BtDisc.mp3")) {
                        __this->bt_conncted = false;
    #ifdef BT_DISCON_MUSIC_NOTICE_ENABLE
                        music_play_res_file("BtDisc.mp3");
```

由于蓝牙音乐播放的打印信息出现在提示音播放结束之前，为确保自动播放蓝牙音乐时能顺利开灯，需在蓝牙连接成功且音频开始播放后，延迟 3 秒再执行开灯操作。

蓝牙连接成功后，自动播放音乐时开灯

```
er_private_cmd.c | aisp.c | client.c | board_7916A_develop_cfg.h | net_music.c | led.c | *bt_music.c | bt.c | ai_speaker.c

    return 0;
}

//暂停/继续播放
int bt_music_play_get_pause(void)
{
    return __this->media_pause;
}

//暂停/继续播放
static int bt_music_play_pause(void)
{
    if (__this->phone_ring_flag) {
        user_send_cmd_prepare(USER_CTRL_HFP_CALL_ANSWER, 0, NULL);
    } else if (__this->call_flag) {
        user_send_cmd_prepare(USER_CTRL_HFP_CALL_HANGUP, 0, NULL);
    } else {
        if(!__this->media_pause){
            user_send_cmd_prepare(USER_CTRL_AVCTP_OPID_PAUSE, 0, NULL);
            __this->media_pause = true;
            printf("蓝牙音乐暂停\n");printf("蓝牙音乐暂停\n");printf("蓝牙音乐暂停\n");
            gpio_direction_output(IO_PORTH_00,0);
        }else{
            user_send_cmd_prepare(USER_CTRL_AVCTP_OPID_PLAY, 0, NULL);
            __this->media_pause = 0;
            printf("蓝牙音乐继续\n");printf("蓝牙音乐继续\n");printf("蓝牙音乐继续\n");
            gpio_direction_output(IO_PORTH_00,1);
        }
    }
    return 0;
}
```

3.2.4 提示音开始与结束：

```
music_play.c | user_private_cmd.c

196  #ifdef CONFIG_DEC_ANALOG_VOLUME_ENABLE
197      r.dec.attr = AUDIO_ATTR_FADE_INOUT;
198  #endif
199      r.dec.cmd = AUDIO_DEC_PP;
200      return server_request(__this->dec_server, AUDIO_REQ_DEC, &r);
201  }
202  int music_play_stop_status(void)
203  //解码文件
204  int music_play_file(const char *path)
205  {
206
207  ...
208      printf("提示音开始\n");printf("提示音开始\n");printf("提示音开始\n");
209      gpio_direction_output(IO_PORTH_00,1);
210      return 0;
211  }
212
213  int music_play_stop(void *priv)
214  {
215
216      int err = 0;
217      union audio_req req = {0};
218
219      if (!__this->file || !__this->dec_server) {
220          return 0;
221      }
222      log_i("music_play_stop\n");
223      req.dec.cmd = AUDIO_DEC_STOP;
224      server_request(__this->dec_server, AUDIO_REQ_DEC, &req);
225
226      int argv[2];
227      argv[0] = AUDIO_SERVER_EVENT_END;
228      argv[1] = (int)__this->file;
229      server_event_handler_del(__this->dec_server, 2, argv);
230
231      if(__this->file){
232          fclose(__this->file);
233          __this->file = NULL;
234      }
235      os_sem_post(&__this->wait_sem);
236      printf("提示音结束\n");printf("提示音结束\n");printf("提示音结束\n");
237      gpio_direction_output(IO_PORTH_00,0);
238      return 0;
239  }
```

3.2.5 AI 对话与用户对话：

```c
int net_ai_dialogue_txt_recv_callback(char *buf, int len)//网络对话文本回调函数
{
    printf("resp_txt : %s \n",buf);
#ifdef AI_UART_CMD_CTROL_ENABLE
    ai_uart_cmd_data_push(AI_UART_CMD_TTS, buf, len);
    int emoji_tts_uart_cmd_callback(char *emoji);
    emoji_tts_uart_cmd_callback(buf);
#endif
    user_emoji_check(buf, len);//UI显示的在这行下面处理（已去掉老板图标）
#ifdef CONFIG_UI_ENABLE
#if (defined CONFIG_UI_ONLY_EYE)
    extern int emoji_tts_callback(char *emoji);
    emoji_tts_callback(buf);
#endif
#endif
    return 0;
}
int net_ai_asr_txt_recv_callback(char *buf, int len)//网络语音识别文本回调函数
{
#ifdef AI_UART_CMD_CTROL_ENABLE
    ai_uart_cmd_data_push(AI_UART_CMD_STT, buf, len);
#endif
    int ret = user_sys_close_check(buf);
    if(ret){
        return ret;
    }
    ret = sys_timeout_stop_off_callback(buf, len);//定时关闭和定时停止播放：1小时后关闭，1小时后关闭音乐
    if(ret){
        return ret;
    }
#ifdef CFG_LED_STATUES_PORT
    ret = light_pwm_instruction_word_callback(buf);
    if(ret){
        return ret;
    }
#endif
    printf("asr_txt : %s \n",buf);
    if(!strcmp("模式切换", buf)){// 模式切换 用户自定义有需要检测
        return 1;//返回1立刻结束对话，返回0则走大模型云端
    }
    return 0;
}
```

ai回答的话

回调返回值为0时，会播报“我还没有学会这个功能呢”

用户说的话

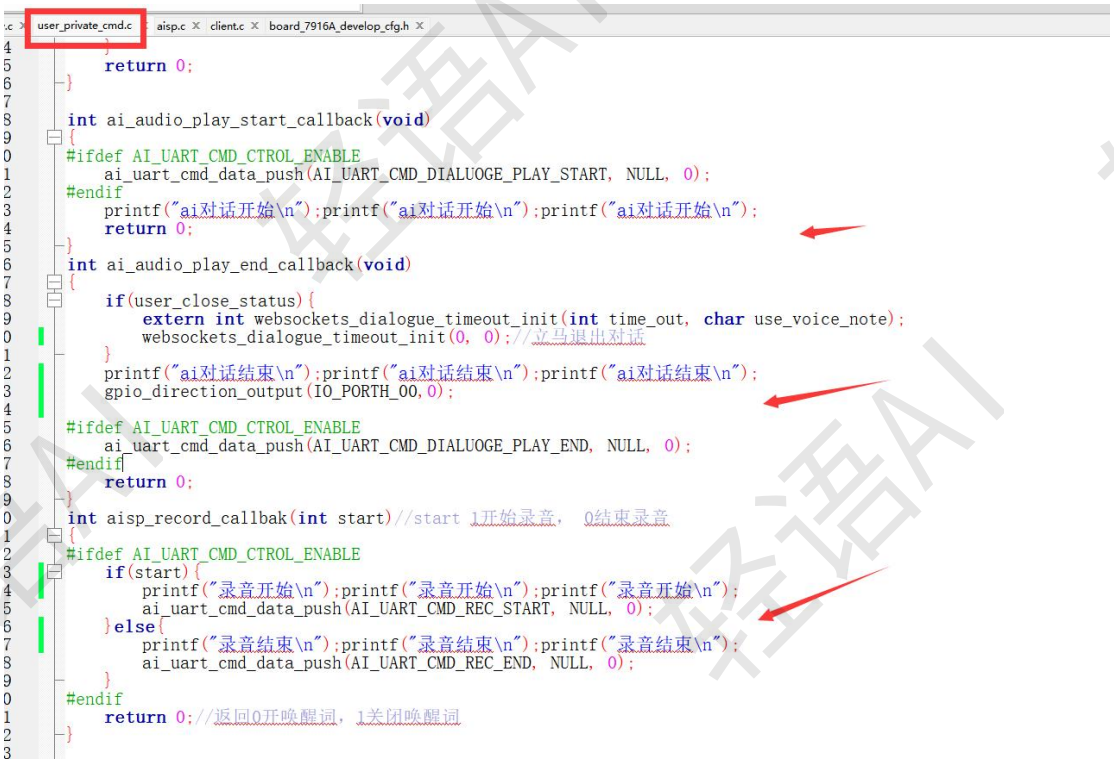
3.2.6 AI 对话开始与结束：

```c
    }
    return 0;
}

int ai_audio_play_start_callback(void)
{
#ifdef AI_UART_CMD_CTROL_ENABLE
    ai_uart_cmd_data_push(AI_UART_CMD_DIALUOGE_PLAY_START, NULL, 0);
#endif
    printf("ai对话开始\n");printf("ai对话开始\n");printf("ai对话开始\n");
    return 0;
}
int ai_audio_play_end_callback(void)
{
    if(user_close_status){
        extern int websockets_dialogue_timeout_init(int time_out, char use_voice_note);
        websockets_dialogue_timeout_init(0, 0);//立马退出对话
    }
    printf("ai对话结束\n");printf("ai对话结束\n");printf("ai对话结束\n");
    gpio_direction_output(IO_PORTH_00,0);

#ifdef AI_UART_CMD_CTROL_ENABLE
    ai_uart_cmd_data_push(AI_UART_CMD_DIALUOGE_PLAY_END, NULL, 0);
#endif
    return 0;
}
int aisp_record_callbak(int start)//start 1开始录音， 0结束录音
{
#ifdef AI_UART_CMD_CTROL_ENABLE
    if(start){
        printf("录音开始\n");printf("录音开始\n");printf("录音开始\n");
        ai_uart_cmd_data_push(AI_UART_CMD_REC_START, NULL, 0);
    }else{
        printf("录音结束\n");printf("录音结束\n");printf("录音结束\n");
        ai_uart_cmd_data_push(AI_UART_CMD_REC_END, NULL, 0);
    }
#endif
    return 0;//返回0开唤醒词，1关闭唤醒词
}
```

3.2.7 开启对话眨眼使能宏后，当设备在 AI 对话过程中有声音输出时，LED 灯会同步亮起。



4. 网络通信与服务配置

4.1 wifi 连接

4.1.1 配网模式（两种情况进入配网模式）：



4.1.2 第一次进入配网模式：



4.1.3 正在连接 wifi：

```
board_7916A.c  SC7A20H.c  *ifly_wake_demo.c  ai_speaker.c  yunzs_wake_demo.c
        case NET_NTP_GET_TIME_SUCC: //NTP获取成功事件返回
            break;
        case NET_NTP_GET_TIME_SUCC + 32://初始化连接网络
            __this->net_dhcp_time_out = timer_get_sec();
            if(!keyworld_wifi_enter_congfig && system_reset_reason_get() != SYS_RST_ALM_WKUP  && !is_production_test_enter(0)){
                if(net_ch == 0){
                    music_play_res_file("WifiConting.mp3");
                    #ifdef CONFIG_UI_ENABLE
                    #ifdef CONFIG_UI_ONLY_EYE//只开UI的眼睛显示
                        extern int play_face_emoji(char index, int avi_index);
                        play_face_emoji(0, 98);
                        os_time_dly(150);
                    #endif
                    #endif
                    printf("=============================================\n");
                    printf("============WifiConting.mp3================\n");
                    printf("=============================================\n");
                }else {
                    music_play_res_file("NetConting.mp3");
                }
```

4.1.4 wifi 连接成功：

```
lop_cfg.h  board_7916A.c  SC7A20H.c  *ifly_wake_demo.c  ai_speaker.c
                    __this->net_dhcp_time_out = 0;
                    music_play_res_file("NetCfgRecv.mp3");
                }
#ifdef CONFIG_ASR_ALGORITHM
                aisp_resume();
#endif
            }
            break;
        case NET_EVENT_CONNECTED:
            __this->net_dhcp_time_out = 0;
            __this->net_dhcp_time_out_cnt = 0;
            __this->associat_timeout_cnt = 0;
            __this->associat_connting = 0;
            __this->wifi_enter_congfig_time_out = 0;
            printf("NET_EVENT_CONNECTED : %d \n",keyworld_wifi_enter_congfig);
            if(keyworld_wifi_enter_congfig >= 2 && system_reset_reason_get() != SYS_RST_ALM_WKUP && !is_production_test_enter(0)){
                if(net_ch == 0){
                    music_play_res_file("WifiConSucc.mp3");
                    printf("=============================================\n");
                    printf("============WifiConSucc.mp3================\n");
                    printf("=============================================\n");
                    #ifdef CONFIG_UI_ENABLE
                    #ifdef CONFIG_UI_ONLY_EYE//只开UI的眼睛显示
                        extern int play_face_emoji(char index, int avi_index);
                        play_face_emoji(0, 99);
                        os_time_dly(150);
                        play_face_emoji(0, 0);
                    #endif
                    #endif

                }else{
                    music_play_res_file("NetConSucc.mp3");
                }
            }
            keyworld_wifi_enter_congfig = 0;

#if BT_NET_CFG_EN
```

第五阶段：网络部分（更新中……）

1. 设备自联网
2. 热点释放
3. 蓝牙设备控制
4. Cat1 卡号获取 & 查询信号强度 & 查看 Cat1 协议支持
5. MQTT 协议 & 配网交换密钥

6. 配对码

第六阶段：问题点

1. 芯片硬件基础与上板适配

1.1 AC791 芯片特性

- AC791 芯片支持 wifi，不支持以太网（需要外挂）。
- 上传原始 PCN 音频格式到服务器，32k /s 的速率上传至服务器端。
- 支持 mp3、opus 等格式的音频解码。
- 由于芯片当前采样率仅 16k，低于线路输入模式所需的 44.1k 采样率，导致音质很差。为保证系统性能，线路输入模式下唤醒词不再响应，如需启用需手动按键切换。
- 芯片的供电流程如下：电池的 VBAT 端为芯片供电，芯片随后输出 V3.3_EN 信号使能 LDO（低压差线性稳压器），LDO 被使能后输出 3.3V 电压。其中，VDDIO 的默认电压为 2.1V，在没有设计低功耗电路的情况下，可以设置强 VDDIO 档位-->3.2V。

1.2 7911 系列（芯片上板）

- 7911BA：使用 2M 工程
- 7911BB：使用普通工程
- 7911B0：使用 SFC 工程
- 4g 设备 ： 使用 LTE 工程

1.3 mic 安装与密封要求

- **接触与固定**：开孔 MIC 和外壳接触处用垫片垫平，不留空隙；按 MIC 本体 + 皮套厚度算尺寸，做专属卡槽让 MIC 卡紧。
- **密封处理**：安装后打低温热熔胶密封，避免高温胶破坏 MIC 一致性。
- **型号要求**：严格使用指定型号与厂家的 MIC，不可替换。

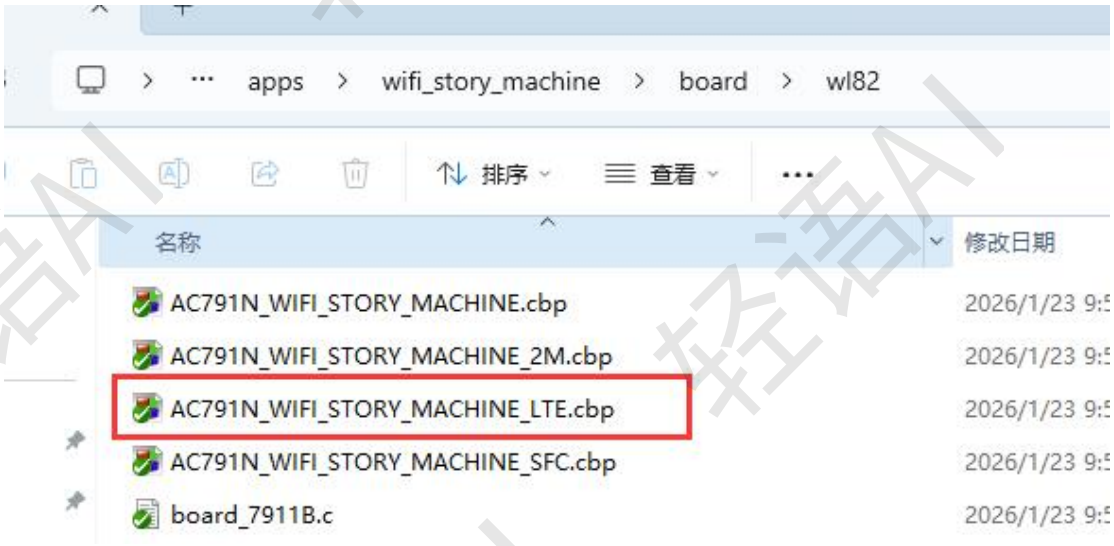
[sd-4015m26d02-hf-asl60 绞线端子高增益电容麦克风规格承认书.pdf]

1.4 焊接 USB 线的样机更新程序步骤

[焊接 usb 线的样机更新程序步骤.pdf]

1.5 4g 工程

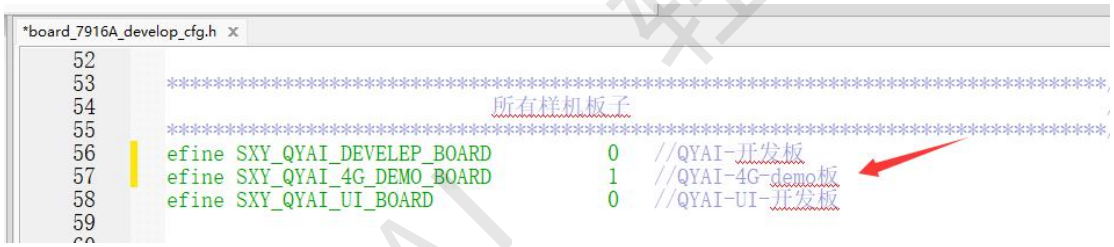
1.5.1 打开 LTE 工程，全编辑即可打开 4g 工程



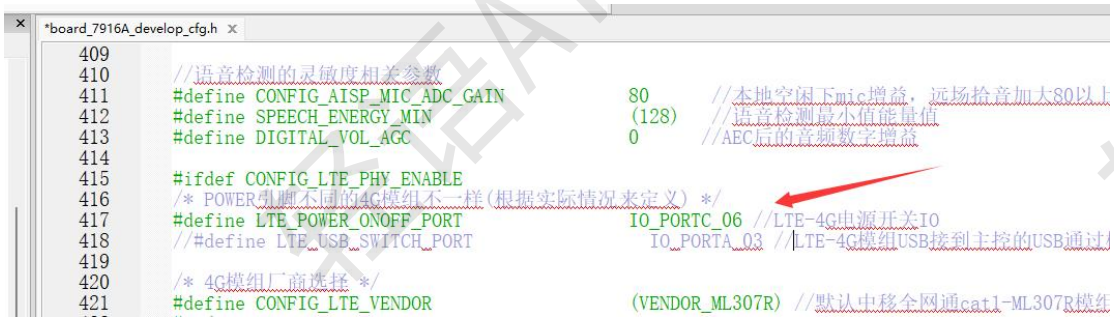
1.5.2 sdk 支持的 4g 模组芯片



1.5.3 配置 4g 开发板



1.5.4 正确配置 4g 模块电源开关 IO



2. 开发调试与串口相关问题

2.1 开发板测试与串口通信调试核心要点总结

2.1.1 开发板测试前置关键步骤：

- 启动测试前，需先明确所用开发板具体型号。因不同型号开发板在硬件设计、接口定义及功能支持上存在差异，对应的系统配置、驱动参数及功能使能项也不相同，只有确定型号，才能精准完成适配性配置，避免因配置不匹配导致测试异常，为后续测试奠定基础。

2.1.2 串口通信项目调试核心前提：

- 调试前必须优先确保两项关键条件：
  - 一是确认串口连接有效，通过检查串口线接口接触、引脚对应关系等，保障硬件链路通畅；
  - 二是确保通信参数一致，其中**波特率一致是核心前提**，同时需核对数据位、停止位、校验位等参数。若波特率不匹配，会直接导致数据传输混乱、信号无法正确解析，后续调试无法获取有效数据，造成调试工作无效，影响项目进度。

2.2 怎么看板子的串口打印？

- 板子上的 TX 引脚和 GND 引脚用导线引出来，再在电脑上打开串口调试助手软件，按照硬件需求设置对应波特率

2.3 为什么部分产品没有串口打印

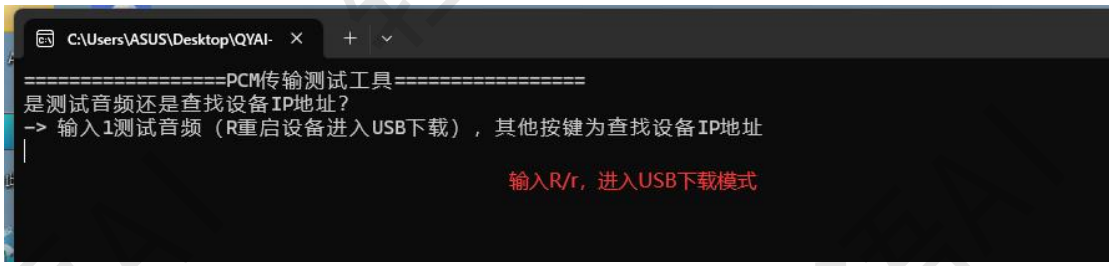
- 产品进入产测模式后，串口没有打印信息输出。

2.4 不拆机进入下载模式

2.4.1 电脑和设备连接的 wifi 在同一个路由器下

2.4.2 双击 QYAI-WIFI-下载模式.exe 工具

[QYAI-WIFI-下载模式.exe]







2.4.3 注意：

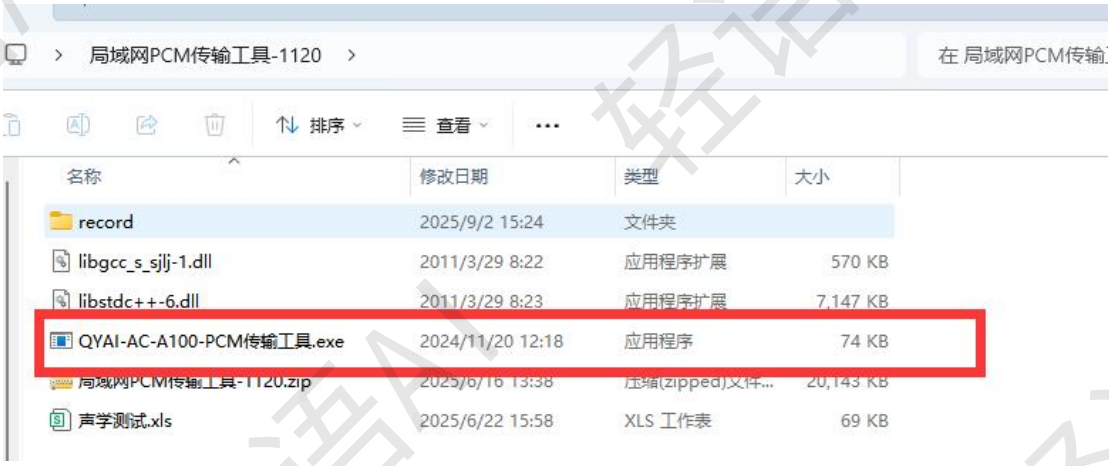
- 设备进入下载模式后，需及时完成代码下载操作；若未下载代码，设备会持续停留在下载模式。
- 由于样机采用电池供电，无法实现完全断电，处于下载模式时，设备无法正常开机并进行对话功能。

3. 麦克风与音频调试优化

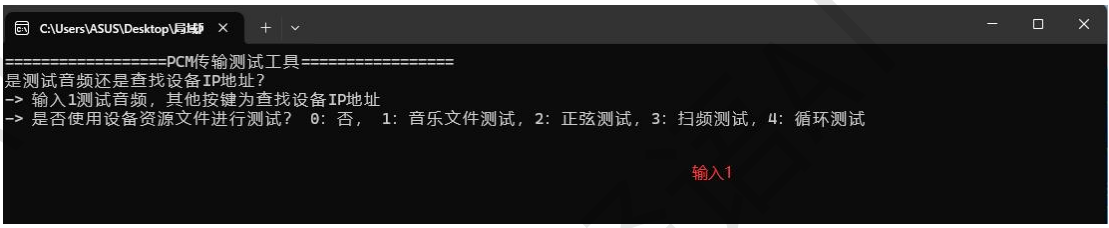
3.1 怎么调试麦克风？

设备已完成安装，麦克风也已封装妥当，此时调试麦克风才能达到预期效果。

3.1.1 请确保设备与电脑处于同一个局域网内，然后双击运行 QYAI-AC-A100-PCM 传输工具.exe 即可。









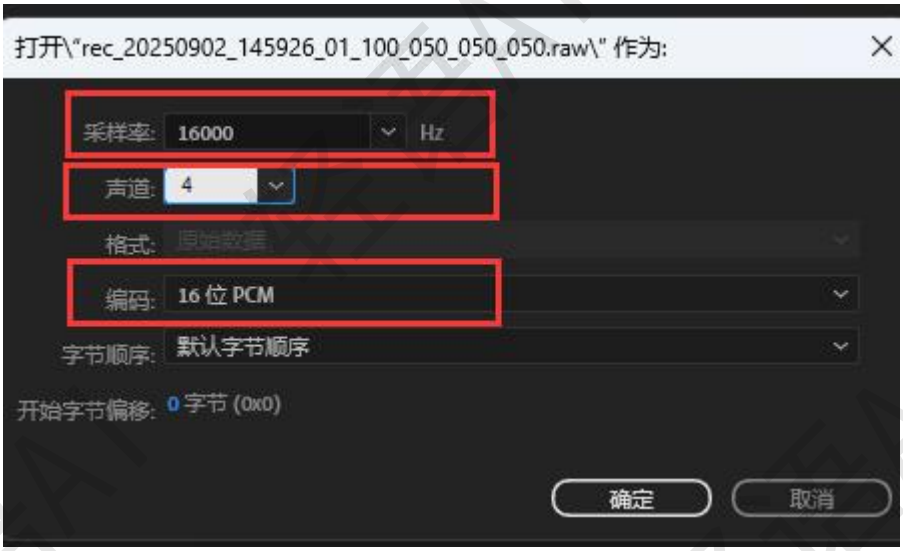




3.1.2 音频文件存放在下图所在文件夹中，用 AU 软件打开这个音频文件

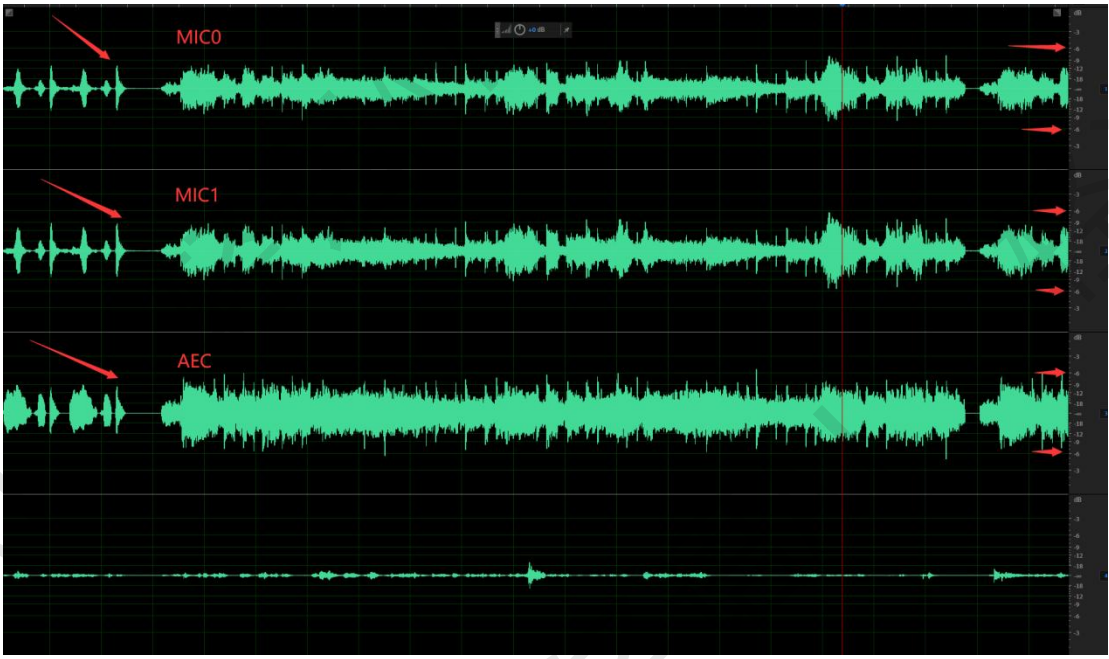


3.1.3 配置采样率 16000Hz；4 声道；16PCM 编码打开音频文件（单麦选择 3 声道）



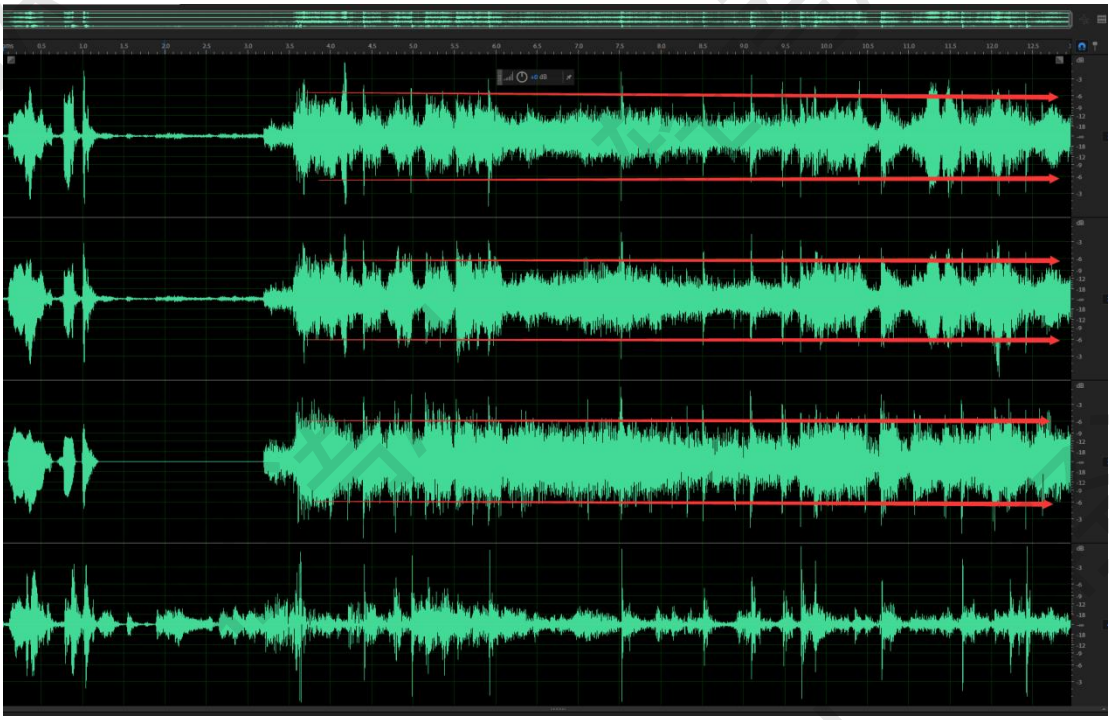
3.1.4 打开音频文件如下图

- **调整增益值，让 mic0、mic1 和 aec 最高点的平均值在-6 附近，**小于-6，增加增益值；大于-6，减小增益值



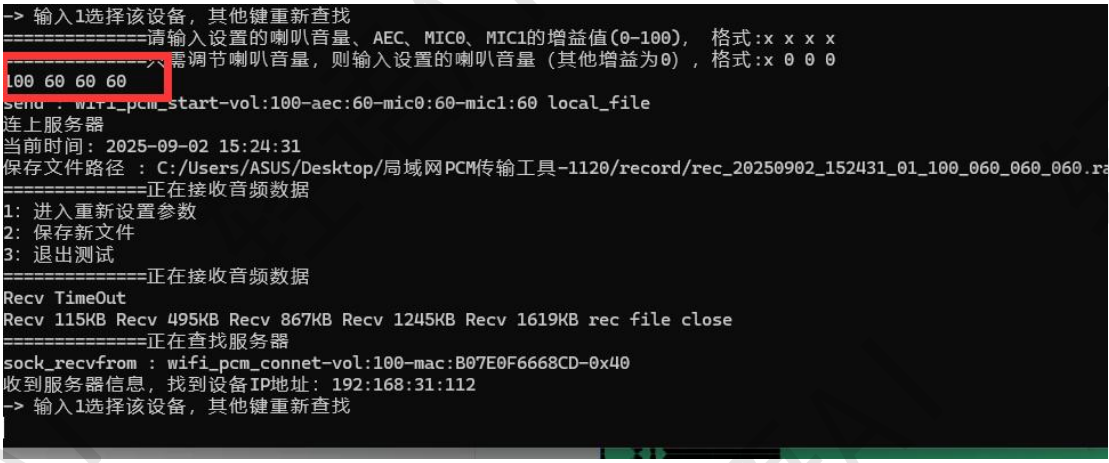
- 如下图可见，当音量值为 100 时，增益值设置为 60 60 60，音频效果最佳

```
-> 输入1选择该设备，其他键重新查找
==============请输入设置的喇叭音量、AEC、MIC0、MIC1的增益值(0-100)，  格式:x x x x
==============如需调节喇叭音量，则输入设置的喇叭音量（其他增益为0），格式:x 0 0 0
100 60 60 60
Send : wifi_pcm_start-vol:100-aec:60-mic0:60-mic1:60 local_file
连上服务器
当前时间：2025-09-02 15:24:31
保存文件路径 ： C:/Users/ASUS/Desktop/局域网PCM传输工具-1120/record/rec_20250902_152431_01_100_060_060_060.ra
==============正在接收音频数据
1：进入重新设置参数
2：保存新文件
3：退出测试
==============正在接收音频数据
Recv TimeOut
Recv 115KB Recv 495KB Recv 867KB Recv 1245KB Recv 1619KB rec file close
==============正在查找服务器
sock_recvfrom : wifi_pcm_connet-vol:100-mac:B07E0F6668CD-0x40
收到服务器信息，找到设备IP地址：192:168:31:112
-> 输入1选择该设备，其他键重新查找
```



3.1.5 打开声学测试表格，记录测试数据

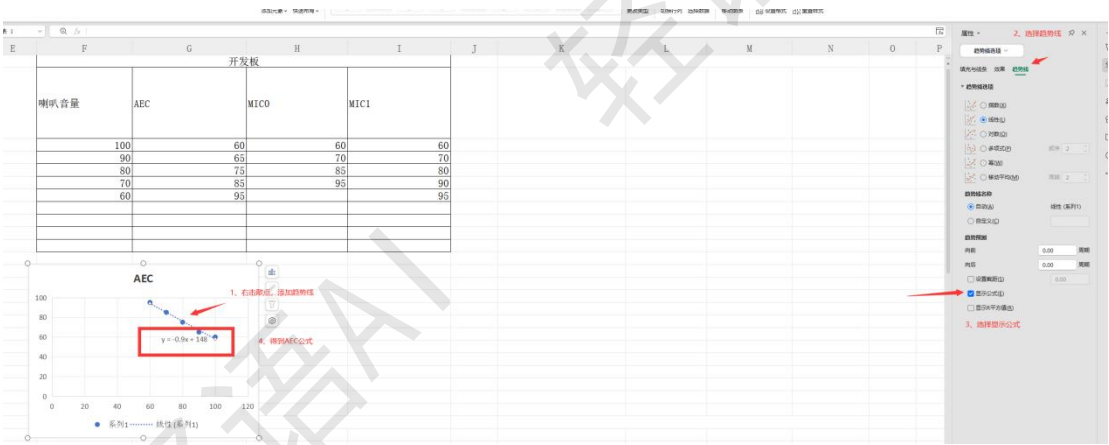
开发板			
喇叭音量	AEC	MIC0	MIC1
100	60	60	60



3.1.6 把音量调到 90，继续测试；直到三个通道增益值达到 95 停止，每次测试结果记录在 excel 表格中（只保留达到要求的测试值，AU 打开的音频文件显示三个通道的平均值达到-6）

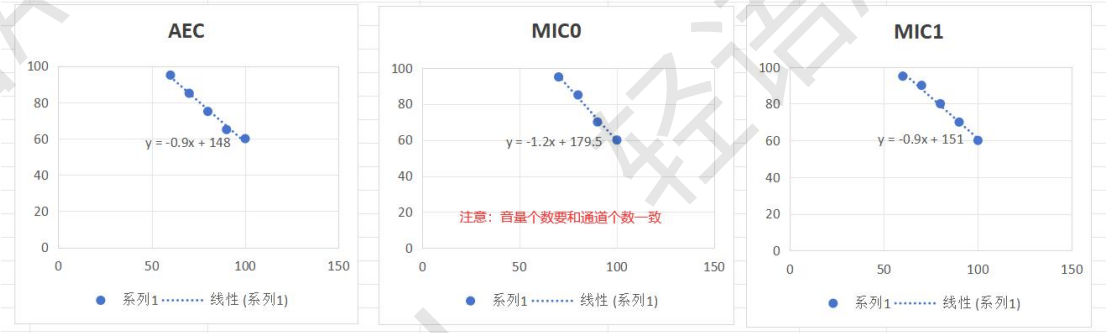
	开发板		
喇叭音量	AEC	MIC0	MIC1
100	60	60	60
90	65	70	70
80	75	85	80
70	85	95	90
60	95		95

3.1.7 记录完测试结果后，生成散点图函数



3.1.8 最后生成三个散点图和三个公式

开发板			
喇叭音量	AEC	MIC0	MIC1
100	60	60	60
90	65	70	70
80	75	85	80
70	85	95	90
60	95		95



3.1.9 打开工程 ai_speaker.c 文件，修改增益值



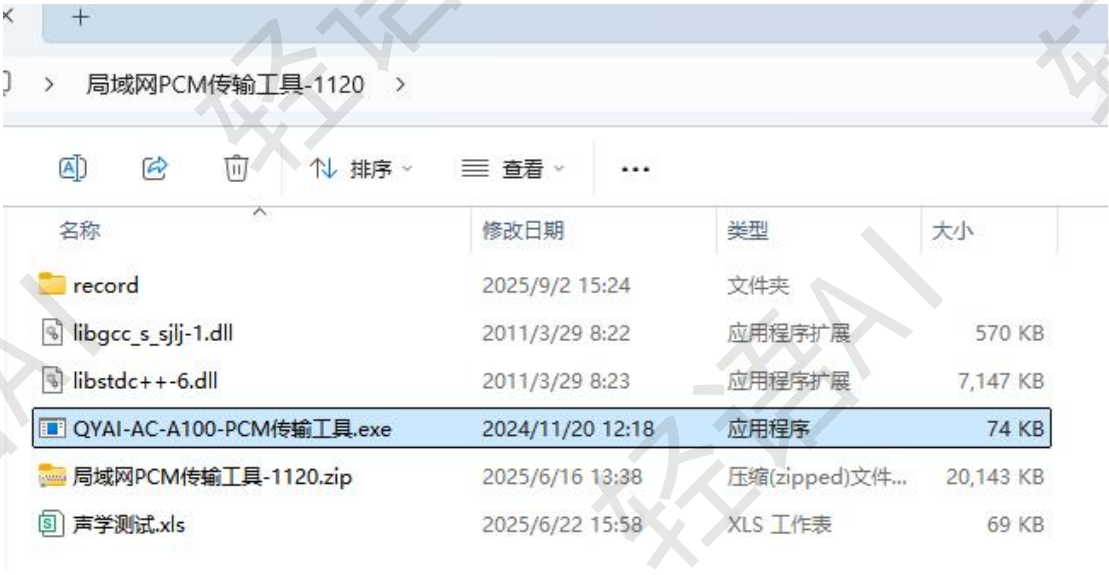
3.2 样机拾音效果不好，怎么排查原因？

3.2.1 硬件问题：

- 麦克风孔小于 1.5mm 拾音效果不好
- 麦克风孔没有做好密封，拾音效果不好
- 喇叭与麦克风同侧，拾音效果不好

3.2.2 使用 QYAI-AC-A100-PCM 传输工具录音排查

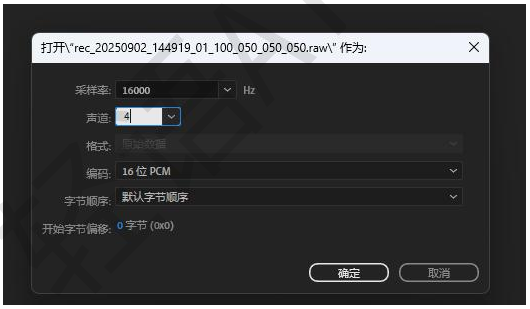
- 请确保设备与电脑处于同一个局域网内，然后双击运行 QYAI-AC-A100-PCM 传输工具.exe 即可。



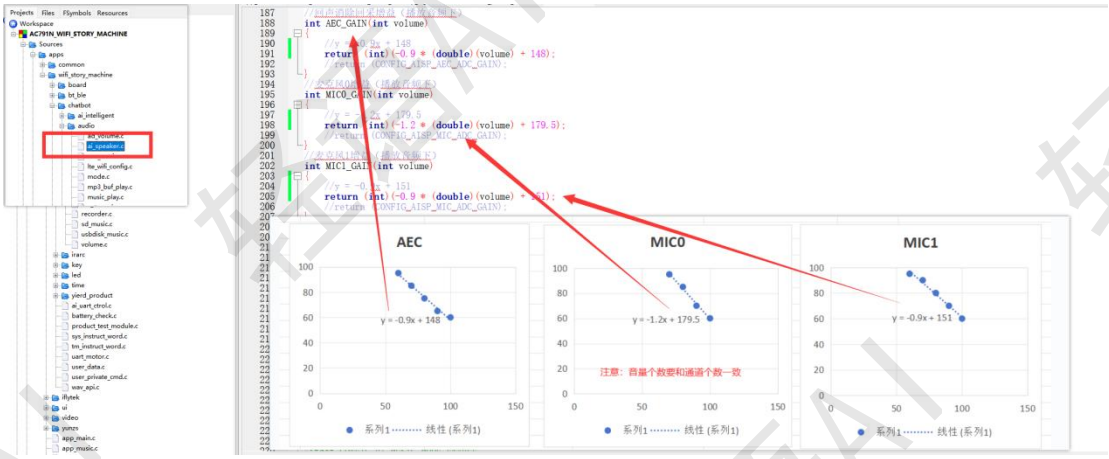


- 音频文件存放在下图所在文件夹中，用 AU 软件打开这个音频文件（单麦选择 3 声道）



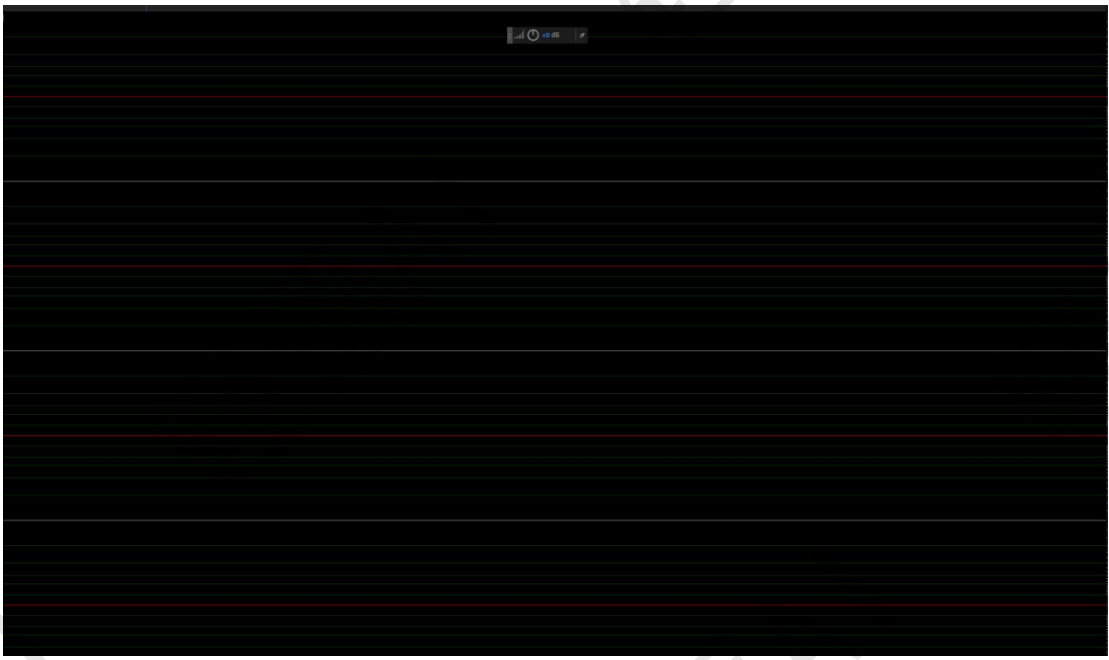


- 打开音频文件之后，适当**调整代码增益值，让 mic0、mic1 和 aec 最高点的平均值在-6 附近，**小于-6，增加增益值；大于-6，减小增益值

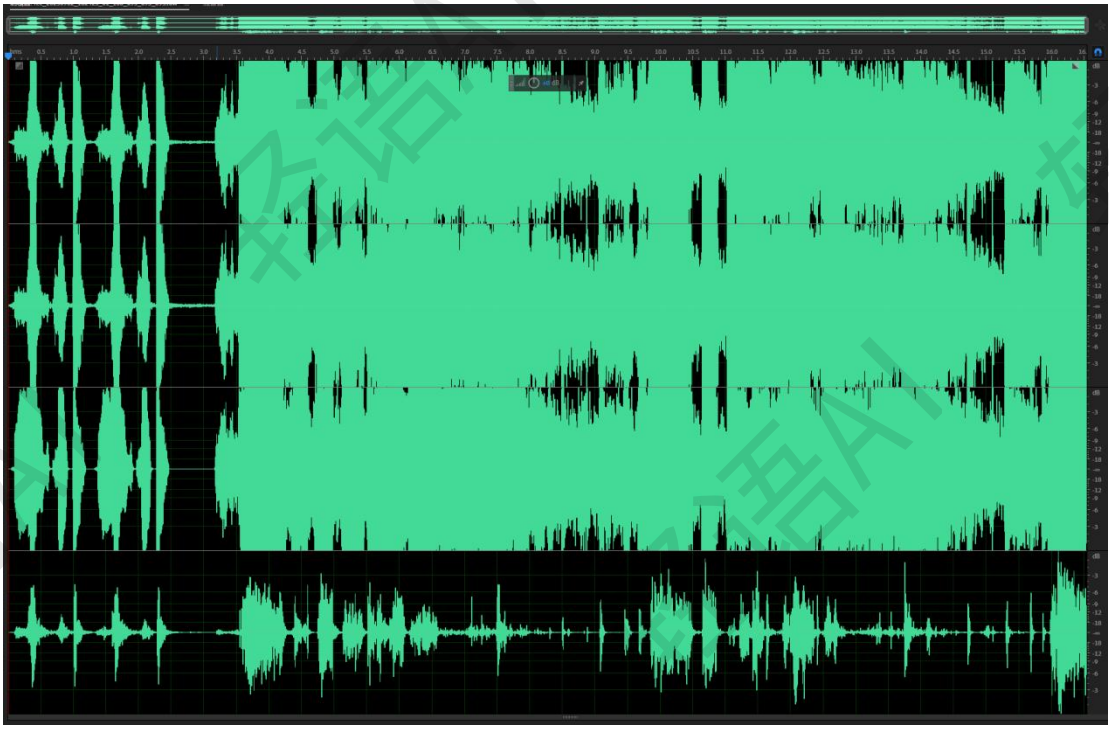


3.2.3 问题实例

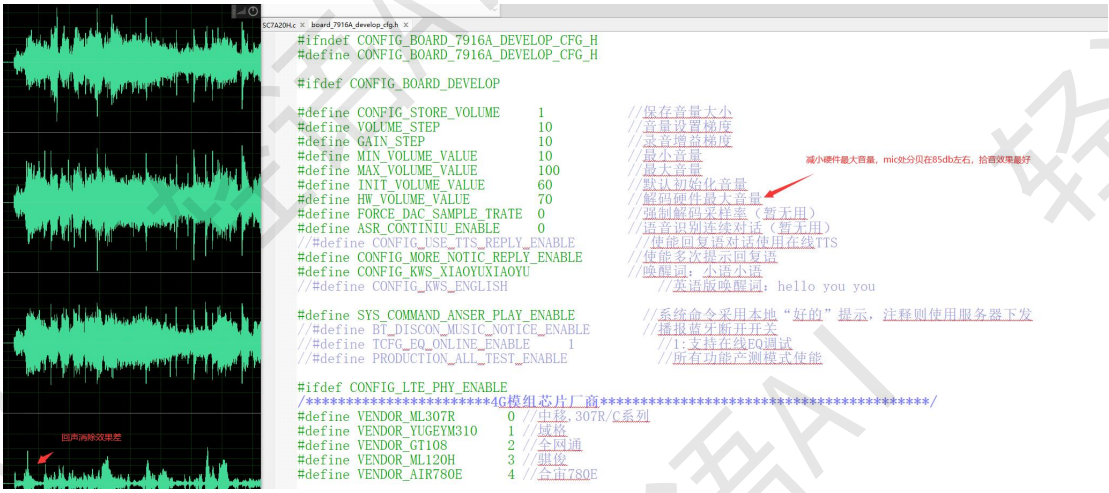
- 无音频数据可能原因：麦克风和喇叭硬件电路连接有问题



- 音频信号出现过载失真可能原因：麦克风密封不好



- 回声消除效果差：分贝仪测试麦克风处的分贝值，大于85db，拾音效果差；此时应该减小硬件音量值



3.3 EQ 调试

3.3.1 工程中打开 EQ 调试宏开关，下载代码到设备

```
*board_7916A.c  *board_7916A_develop_cfg.h  d.c  user_private_cmd.c  ai_speaker.c  lib_driver_config.c  timer.h  dac.h  app_config.h  *lcd_st77912.c  avi_face.c  gif_eye.c
//                                  数芯语-UI-开发板                                                    //
//*****************************************************************************************//
#if SXY_QYAI_UI_BOARD //数芯语-UI-开发板

#define CONFIG_UI_ENABLE                                //开启UI显示
#define CONFIG_UI_ONLY_EYE                              //只开UI的眼睛显示

#ifdef CONFIG_UI_ONLY_EYE
#define CONFIG_UI_TOW_EYE                               //开2个眼睛显示
//#define CONFIG_UI_GIF_EYE                               //开启GIF眼睛显示
#define CONFIG_UI_AVI_EYE                               //开启AVI眼睛显示
#ifdef CONFIG_UI_TOW_EYE
#define CONFIG_UI_MIRROR_EYE                            //开启另外眼睛镜像显示
#endif
#endif

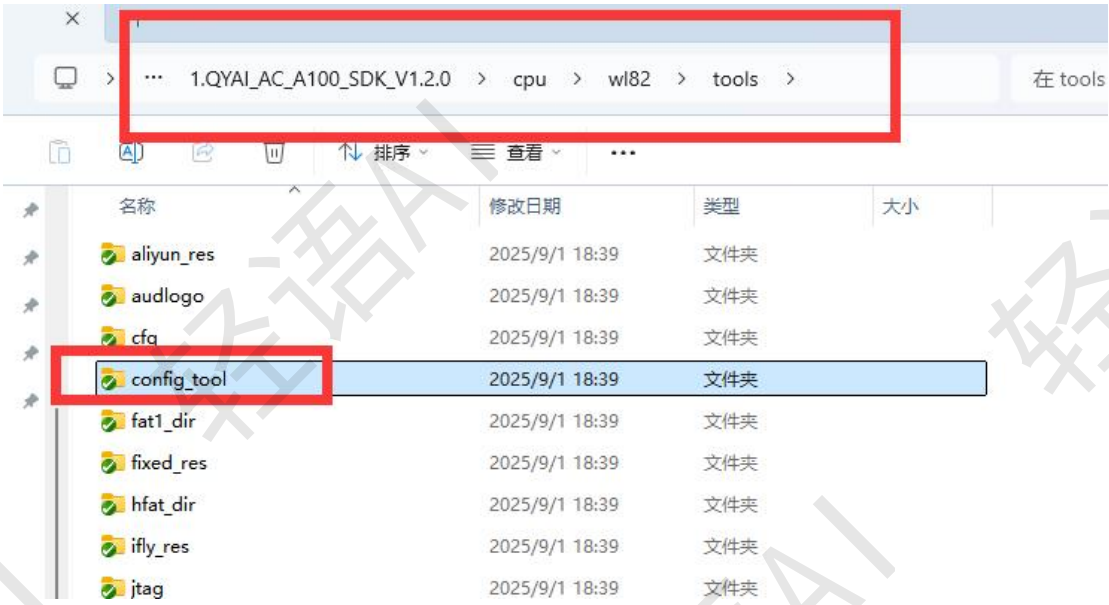
#define TCFG_EQ_ENABLE                  1       //0关闭EQ调试功能

//#define PRODUCTION_UUID_ENABLE                        //开启量产ID
//#define PRODUCTION_TEST_ENABLE                        //进入产测模式使能

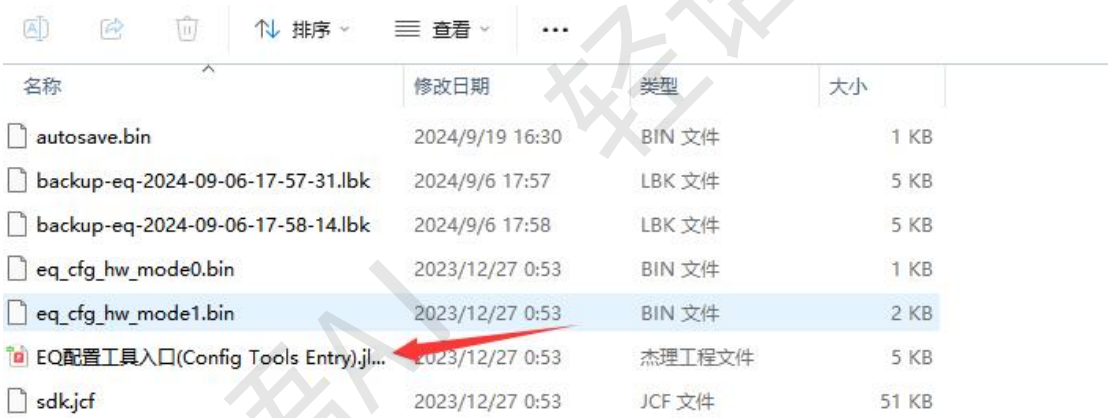
//#define PRODUCTION_ALL_TEST_ENABLE                    //开启量产测试所有功能
```

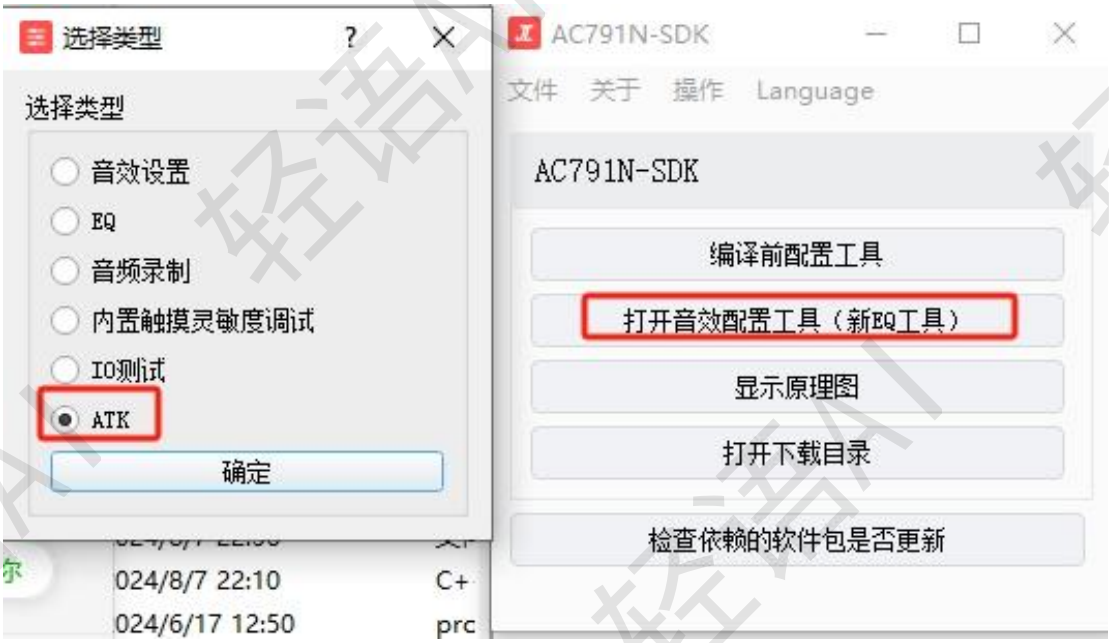
3.3.2 EQ 安装

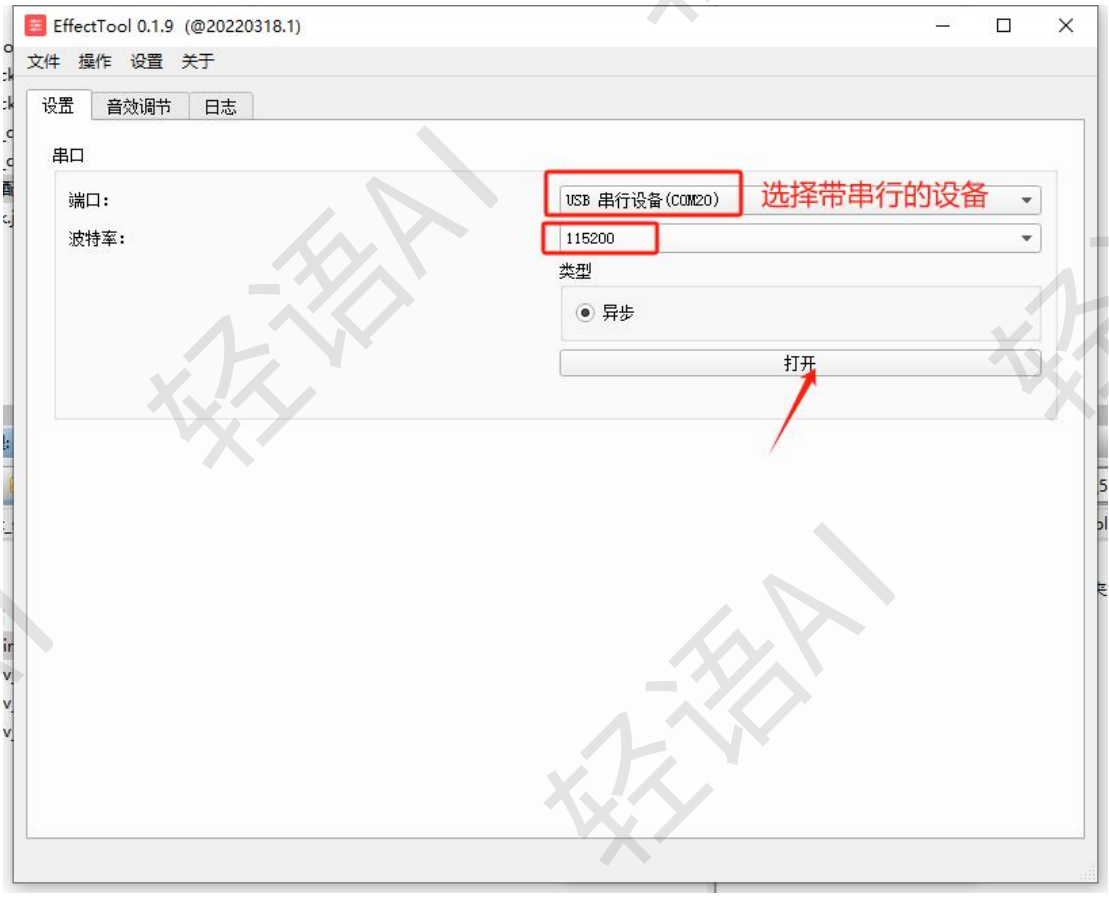
- 打开 config_tool 文件夹



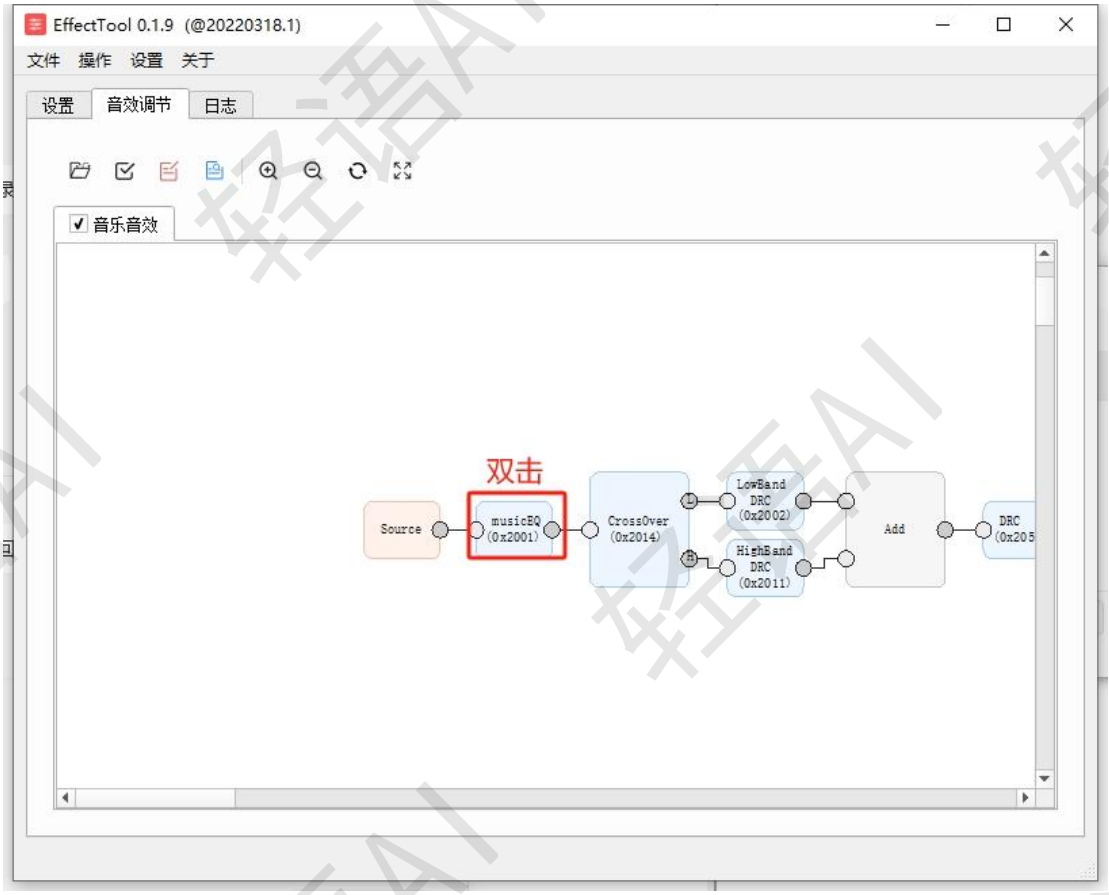
- 运行：EQ 配置工具入口(Config Tools Entry).jlxproj




选择类型
选择类型
音效设置
EQ
音频录制
内置触摸灵敏度调试
IO测试
ATK
确定
AC791N-SDK
文件 关于 操作 Language
AC791N-SDK
编译前配置工具
打开音效配置工具（新EQ工具）
显示原理图
打开下载目录
检查依赖的软件包是否更新
024/8/7 22:10 C+
024/6/17 12:50 prc


EffectTool 0.1.9 (@20220318.1)
文件 操作 设置 关于
设置 音效调节 日志
串口
端口:
USB 串行设备(COM20)
选择带串行的设备
波特率:
115200
类型
异步
打开

轻语AI

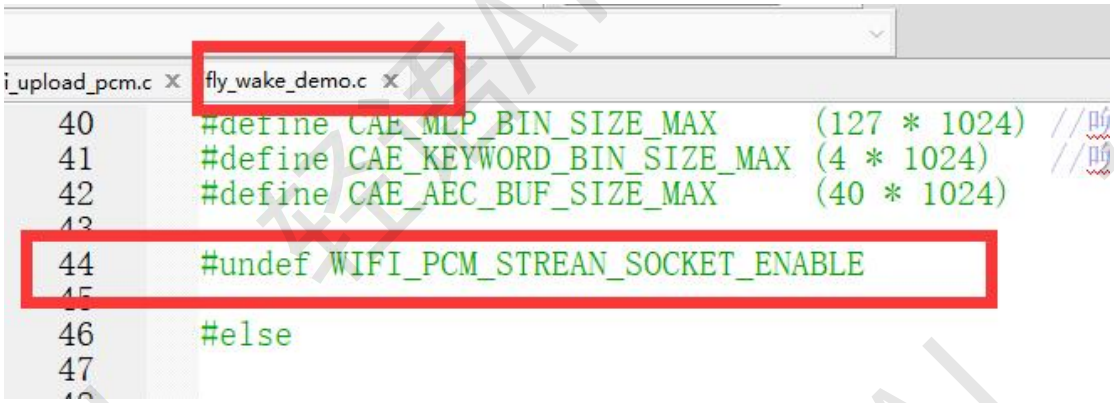


3.3.3 在音乐播放场景中：

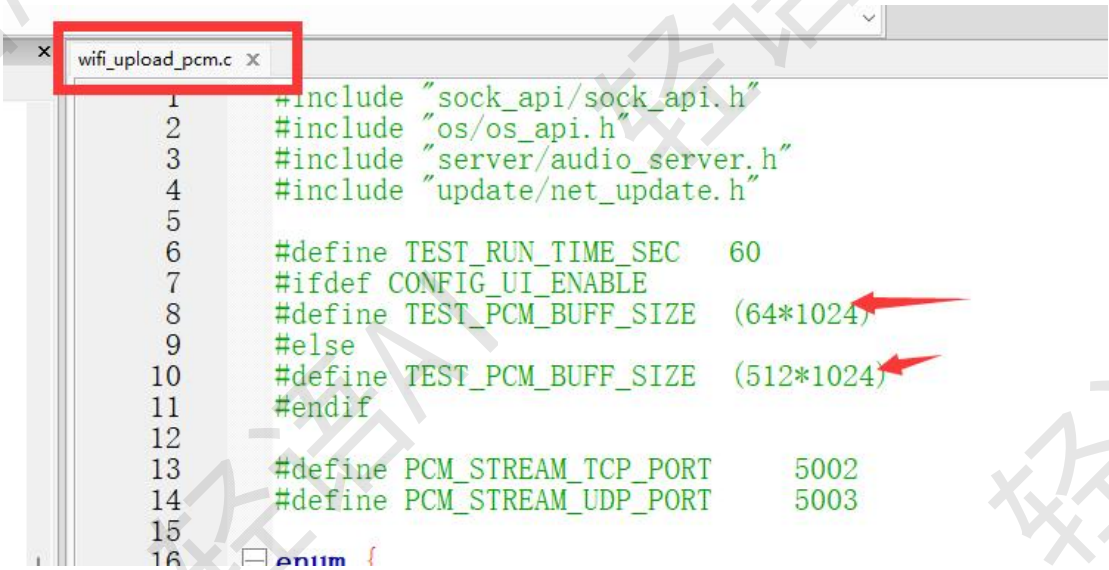
- 提升低频频段的增益，贝斯（bass）音色更浑厚饱满；
- 提升中频频段的增益，人声更清晰突出；
- 提升高频频段的增益，为声音增添细节感与明亮度，让声音整体更通透鲜活。

3.4 测试音频的时候超时

3.4.1 注释这一行



3.4.2 内存改小一点



3.5 麦克风顺序修改配置

- **默认的麦克风配置为：mic0 和 mic1 负责拾音功能，mic2 则用作回采电路。**
- **从代码维护角度考虑，建议保持当前的麦克风排序不变。因为这一顺序在后续代码中已有适配性设计，调整通道顺序，会增加代码维护的复杂度，引发不必要的逻辑问题。**

```
//*************************************************************************//
//                          AUDIO_ADC应用的通道配置                          //
//*************************************************************************//
#define CONFIG_AUDIO_ENC_SAMPLE_SOURCE      AUDIO_ENC_SAMPLE_SOURCE_MIC
#define TCFG_MIC_CHANNEL_MAP                LADC_CH_MIC1_P_N | LADC_CH_MIC2_P_N | LADC_CH_MIC0_P_N

#define MIC0                                1   //mic0通道
#define MIC1                                2   //mic1通道
#define MIC2_AEC                            0   //回采AEC回声消除通道

#define TCFG_MIC_CHANNEL_NUM                3

#ifdef CONFIG_AUX_MUSIC_MODE_ENABLE
#define TCFG_LINEIN_CHANNEL_MAP             LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             2
#else
#define TCFG_LINEIN_CHANNEL_MAP             -1//LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             0
#endif

#define CONFIG_ASR_CLOUD_ADC_CHANNEL        MIC0        //云端识别mic通道
#define CONFIG_VOICE_NET_CFG_ADC_CHANNEL    MIC0        //声波配网mic通道
#define CONFIG_AISP_MIC0_ADC_CHANNEL        MIC0        //本地唤醒左mic通道
#define CONFIG_AISP_MIC1_ADC_CHANNEL        MIC1        //本地唤醒右mic通道
#define CONFIG_REVERB_ADC_CHANNEL           MIC0        //混响mic通道
#define CONFIG_PHONE_CALL_ADC_CHANNEL       MIC0        //通话mic通道
#define CONFIG_UAC_MIC_ADC_CHANNEL          MIC0        //UAC mic通道

#define AUDIO_RECORD_MIC_COUNT          2   //1单麦 2双麦
```

3通道

双麦

调整麦克风配置，将 mic0 作为回采电路，mic1 和 mic2 作为拾音麦克风，可按以下方式修改：

3.5.1 麦克风通道重新配置：

- 拾音麦克风通道：调整为 mic1、mic2
- 回采电路通道：指定为 mic0

```
//*************************************************************************//
//                          AUDIO_ADC应用的通道配置                          //
//*************************************************************************//
#define CONFIG_AUDIO_ENC_SAMPLE_SOURCE      AUDIO_ENC_SAMPLE_SOURCE_MIC
#define TCFG_MIC_CHANNEL_MAP                LADC_CH_MIC1_P_N | LADC_CH_MIC2_P_N | LADC_CH_MIC0_P_N

#define MIC0                                0   //mic0通道
#define MIC1                                1   //mic1通道
#define MIC2_AEC                            2   //回采AEC回声消除通道

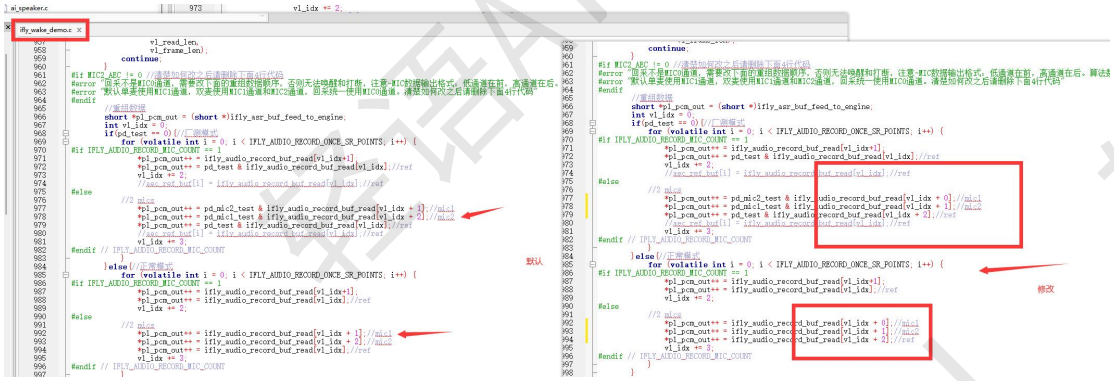
#define TCFG_MIC_CHANNEL_NUM                3

#ifdef CONFIG_AUX_MUSIC_MODE_ENABLE
#define TCFG_LINEIN_CHANNEL_MAP             LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             2
#else
#define TCFG_LINEIN_CHANNEL_MAP             -1//LADC_CH_AUX0 | LADC_CH_AUX2
#define TCFG_LINEIN_CHANNEL_NUM             0
#endif

#define CONFIG_ASR_CLOUD_ADC_CHANNEL        MIC0        //云端识别mic通道
#define CONFIG_VOICE_NET_CFG_ADC_CHANNEL    MIC0        //声波配网mic通道
#define CONFIG_AISP_MIC0_ADC_CHANNEL        MIC0        //本地唤醒左mic通道
#define CONFIG_AISP_MIC1_ADC_CHANNEL        MIC1        //本地唤醒右mic通道
#define CONFIG_REVERB_ADC_CHANNEL           MIC0        //混响mic通道
#define CONFIG_PHONE_CALL_ADC_CHANNEL       MIC0        //通话mic通道
#define CONFIG_UAC_MIC_ADC_CHANNEL          MIC0        //UAC mic通道

#define AUDIO_RECORD_MIC_COUNT          2   //1单麦 2双麦
```

3.5.2 唤醒函数中对麦克风通道的描述修改：原涉及 "mic0、mic1 作为拾音麦克风，mic2 作为回采电路" 的表述，替换为 "mic1、mic2 作为拾音麦克风，mic0 作为回采电路"，确保函数内对麦克风功能的调用逻辑与新配置的通道对应。



4. 语音功能开发与配置

4.1 增加在线指令 && 本地回复语

4.1.1 定义在线指令数组

```
static const char *pwm_light[] = {
    "暗一点", "灯光暗一点", "亮度调暗一点", "亮度调暗一些", "再调暗一点", "灯光调暗一些", "灯光调暗", "调暗灯光", "调暗灯", "再暗点", "亮度增小", "减小亮度", "降低亮度", "调低亮度",
    "亮一点", "灯光亮一点", "亮度调亮一点", "亮度调亮一些", "再调亮一点", "灯光调亮一些", "灯光调亮", "调亮灯光", "调亮灯", "再亮点", "亮度增加", "增加亮度", "提高亮度", "调高亮度",
    NULL,
};
static const char *pwm_light_lowest[] = {
    "调到最暗", "最小亮度", "最暗", "暗到最低", "亮度最低", "最低亮度", "亮度最小", "调到最小亮度", "亮度调到最暗", "亮度调到最小",
    NULL,
};
static const char *pwm_light_highest[] = {
    "调到最亮", "最大亮度", "最亮", "亮到最高", "全亮", "最高亮度", "亮度最高", "调到最高亮度", "调到最大亮度", "亮度最大",
    NULL,
};
static const char *open_light_up[] = {
    "打开上光", "打开上发光", "打开上灯", "上灯打开", "开上光",
    NULL,
};
static const char *close_light_up[] = {
    "关闭上光", "关闭上发光", "关闭上灯", "上灯关闭", "关掉上灯", "关上光",
    NULL,
};
```

定义在线指令数组

4.1.2 定义语音回调函数

```
int light_pwm_instruction_word_callback(char *word) //语音控制灯
{
        char status = 0;
        printf("================================\n");
            // 开灯命令
        for(int i = 0; open_light[i] != NULL; i++) {
            if(strstr(word, open_light[i])) {
                status = ALL_LIGHT_ON;
                printf("识别命令: 打开灯光\n");
                goto exit;
            }
        }

        // 关灯命令
        for(int i = 0; close_light[i] != NULL; i++) {
            if(strstr(word, close_light[i])) {
                status = ALL_LIGHT_OFF;
                printf("识别命令: 关闭灯光\n");
                goto exit;
            }
        }


exit:
    char reida_use = 0;
    if(status){
       music_play_light_anser(0);
    }
    switch(status) {
        case ALL_LIGHT_OFF:
            handleAllLightOff();
            break;

        default:
            handleDefault();
            break;
    }
    return status ? 1 : 0;
}
```

匹配相应指令

匹配成功后，执行对应操作

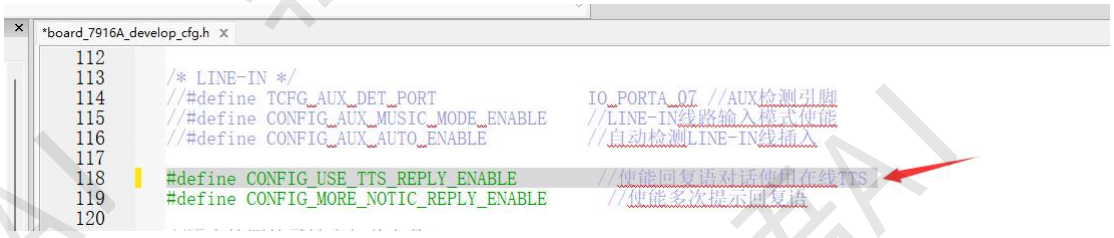
4.1.3 在网络语音识别文本的回调函数里，调用语音回调函数



注意：当语音匹配回调函数返回值为 0 时，会触发“我还没有学会这个功能”的音频。

4.1.4 本地回复语切换在线回复语

头文件中打开在线 tts 回复语的宏定义，即可切换（注意：在线回复语速度比本地回复语慢）



4.1.5 本地回复语切换英文回复语

头文件中打开在线 tts 回复语的宏定义，即可切换（注意：在线回复语速度比本地回复语慢）

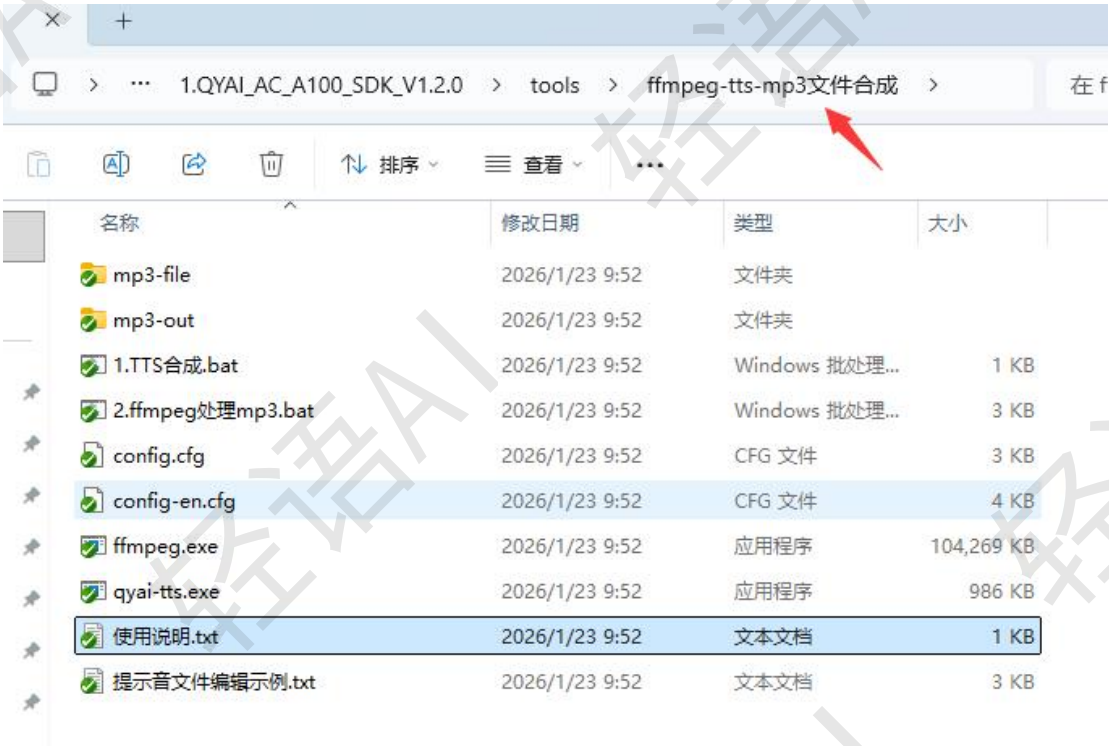
4.2 在线 TTS 语音合成 && 本地 mp3 音频文件合成

4.2.1 在线 tts 语音合成

```
Plain Text
char *vbat_perc_val;
vbat_perc_val = "电池电量为百分之九十";
http_tts_request(vbat_perc_val,strlen(vbat_perc_val));
http_tts_play_wait();
aisp_mic_gain_resum();
```

4.2.2 本地 mp3 音频文件合成

- 打开工程 tools 目录下的 mp3 文件合成工具

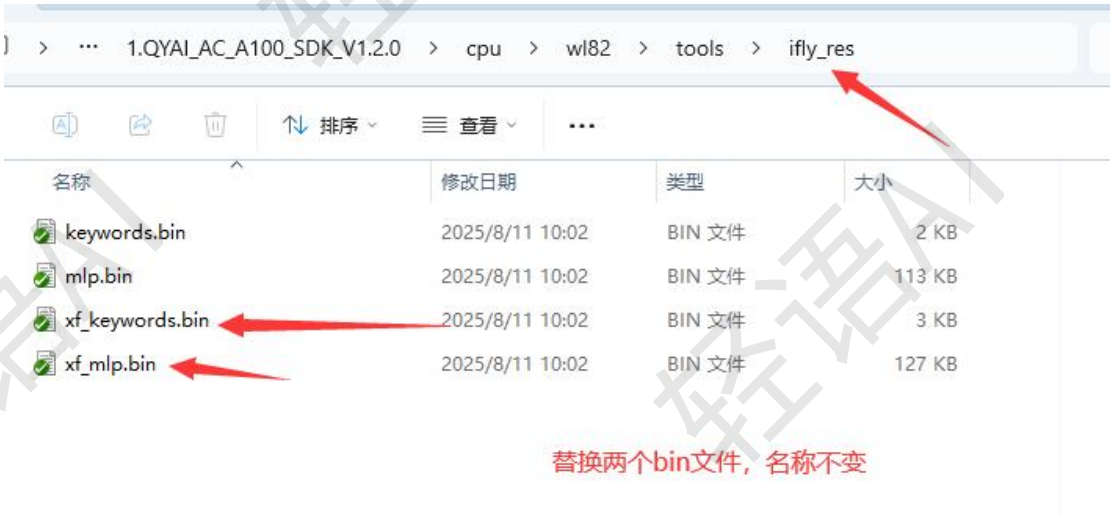


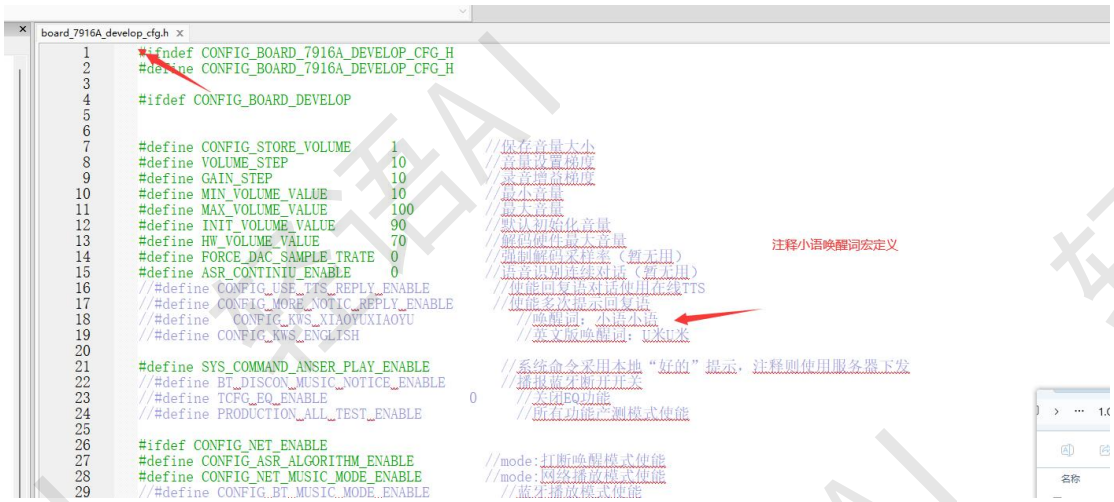
- 使用说明

- 先使用设备进行配网，并拿到：MAC+UUID+批次号+产品名称，启动对话之后在设备的日志可以拿到
- 把"MAC+UUID+批次号+产品名称"放在当前文件夹下配置文件 config.cfg 的 uuid 字段;
- 文件 config-en.cfg 是合成英文范例，如果需要合成英文，把文件名改为 config.cfg[TTS 合成读取的配置文件名是 config.cfg]
- 运行：1.TTS 合成.bat （TTS 合成得到的 mp3 文件在 mp3-file 文件夹目录下）
- 运行：2.ffmpeg 处理 mp3.bat （处理得到的 mp3 文件在 mp3-out 文件夹目录下）

. 复制 mp3-out 文件夹下面所有 mp3 文件到 SDK 的 tools/audlogo/，重新编译下载

4.3 唤醒词替换





4.4 怎么切换 AI 智能体?

4.4.1 切换 AI 智能体的函数接口如下：



[智能体 ID.txt]

注意：仅文档中已列出的智能体支持进行配置，文档中未提及的智能体暂不支持相关设置操作。

4.4.2 语音切换智能体

4.4.3 NFC 切换智能体

4.5 对话等待时间修改

- 在 app_main.c 文件中定义如下函数，调整超时时间即可。

```c
*app_main.c
273     {
274         return NULL; //客户ID
275     }
276
277     //产品批次号
278     int app_user_product_batch(void)
279     {
280         return 1; //批次号
281     }
282
283     //产品批次的数量，单位K
284     int app_user_product_batch_num(void)
285     {
286         return 1; //批次号的数量，单位K
287     }
288
289     /**
290      * @brief 获取AI对话的超时时间（毫秒）
291      * @param timeout: 预留参数（当前未使用）
292      * @return 超时时间，单位为毫秒
293      * @note 如需调整超时时间，修改DIALOGUE_TIMEOUT_SEC宏定义即可
294      */
295     #define DIALOGUE_TIMEOUT_SEC 10  // 对话超时时间（秒），可在此处修改
296
297     int net_ai_dialogue_timeout_get(int timeout) {
298
299         return DIALOGUE_TIMEOUT_SEC * 1000;
300     }
301     /*
302      * 应用程序主函数
303      */
304     void app_main()
```

5. 设备系统配置与模式管理

5.1 怎么配置设备长时间无操作自动退下和自动关机

5.1.1 打开相关使能宏

```c
#define TCFG_AUTO_SLEEP_CHECK_EN            1       //使能特定时间待机检测空闲后进入关机
#define TCFG_AUTO_SLEEP_CHECK_ONLY_SPEECH   1       //使能只检测语音对话作为是否空闲
#define TCFG_AUTO_DEV_LOW_PWER_TIME_SEC     30      //使能30秒进入外设低功耗(关无线，只是把耗电外设降低功耗)
#define AUTO_SLEEP_TIME_MIN                 5       //5分钟自动关机
```

5.1.2 具体实现：

```c
    }else{
#ifdef  USED_TM1629_SHOWN
        tm_1629_time_shown_level_low_power(0);
#endif
        dac_mute_control(0);
    }
#endif
#ifdef AUTO_SLEEP_TIME_MIN
    if(__this->auto_sleep_cnt > AUTO_SLEEP_TIME_MIN * 6){//5分钟后低功耗
        sys_power_poweroff();
    }
#endif
}
static int auto_sleep_init(void)
{
    sys_timer_add_to_task("sys_timer", NULL, auto_sleep_check, 10000);//10秒1次检测
    return 0;
}
late_initcall(auto_sleep_init);
#endif
```

5.2 厂测模式

5.2.1 量产时打开测试网络，网络配置如下：



5.2.2 四连击按键进入产测模式

5.2.3 厂测模式：主要测试 wifi 连接是否正常，麦克风、喇叭是否正常。

1、先设置路由器名称:WIFI-PD-TEST，密码:12345678，频段:2.4GHz

2、四连按按键，进入厂测模式：出货版本会提示“进入厂测模式”

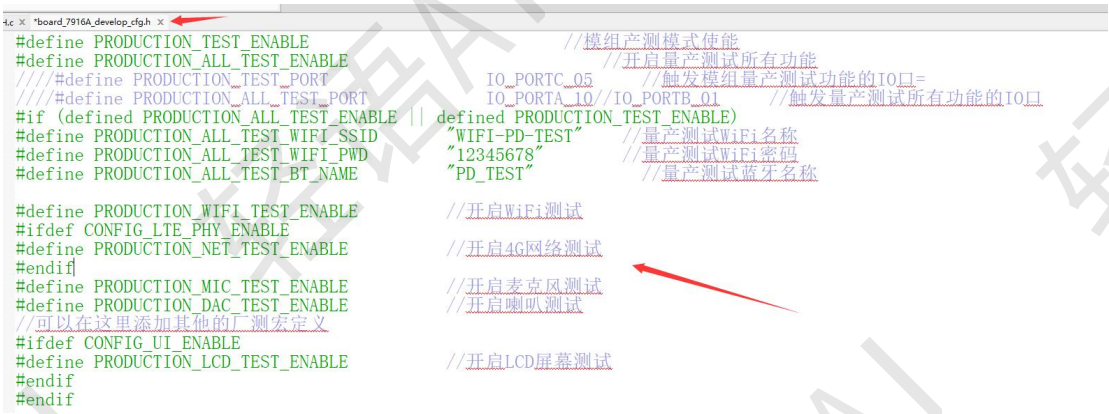
3、等待设备自动测试播报结果。

（1）如果设备测试结束报“麦克风 1”或者“麦克风 2”失败，有可能是设备声音太小导致无法自播字唤醒，则可能需要重启进行测试并在设备自己播放唤醒词时候需要人工含唤醒词。如果人工唤醒还是报麦克风失败，则为硬件麦克风电路有问题。

（2）播报 wifi 测试失败，则无法连接路由，检查路由器名称和密码，并且在 2.4GHz 频段。

（3）测试合格，则代表设备测试项合格。

5.2.4 产测需要开启的宏定义如下（开启产测模式的设备无法查看串口）：



5.3 配网成功之后一直播报未绑定设备？

5.3.1 产品信息确认：

- 确保产品英文名称及客户指定 ID 配置准确无误。

5.3.2 设备操作：

- 执行恢复出厂设置后，重新进行网络配置。

6. 网络通信与 OTA 升级问题

6.1 怎么远程 OTA 升级？

6.1.1 OTA 升级说明：

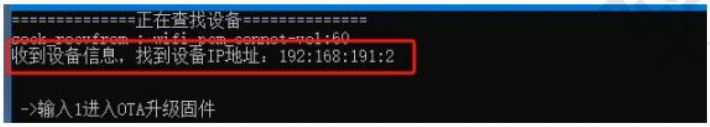
1、电脑和设备连接的 WiFi 在同一个路由器下
2、设备插上电，等待连接 wifi 成功提示，运行解压后的固件升级文件夹下升级工具，目录：固件-升级/QYAI_WIFI_OTA/**QYAI_WIFI_OTA.exe**



下图是固件包名称（如果没有固件包，则需要复制 update-ota.ufw 固件包到这个目录下）



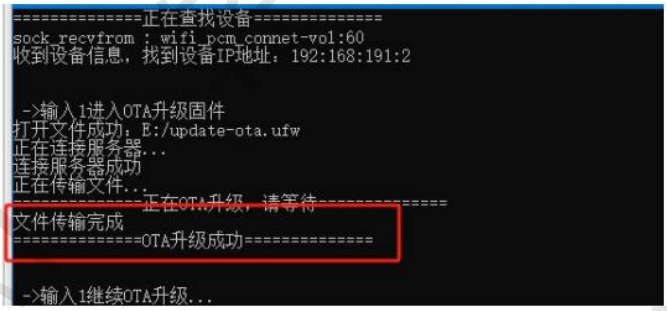
3、查看信息：出现收到设备信息则找到设备，电脑键盘输入 1 进行固件升级



4、升级中

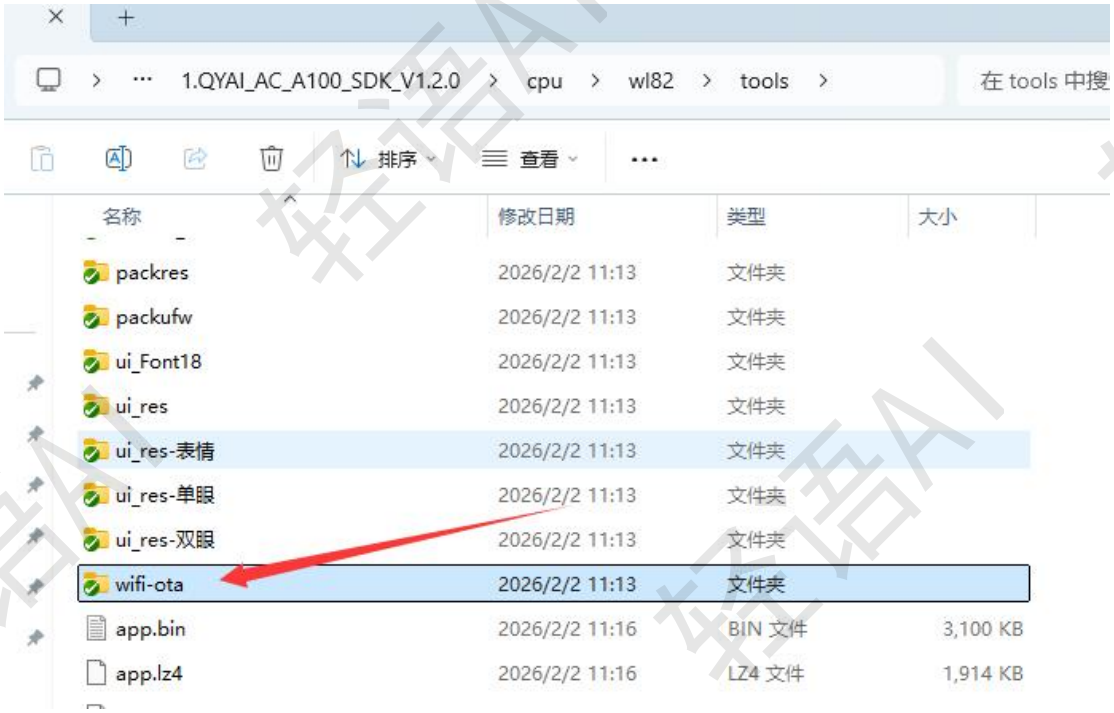


5、等待设备提示升级完成（时间大概 1 分钟），升级工具也提示 OTA 升级成功。

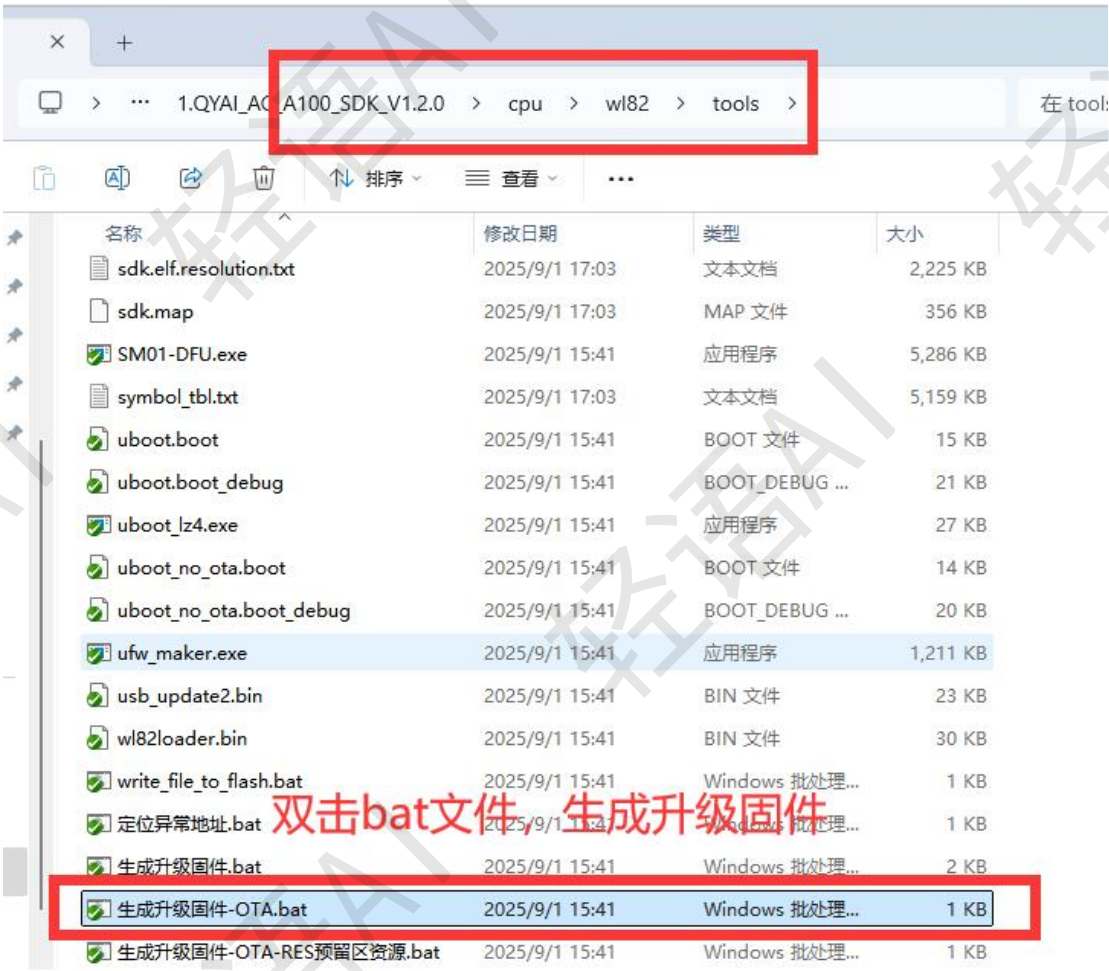


6、如果很久没有升级成功或者升级工具提示升级失败，则重启设备，重启升级工具，重复本文操作。

6.1.2 OTA 升级工具：

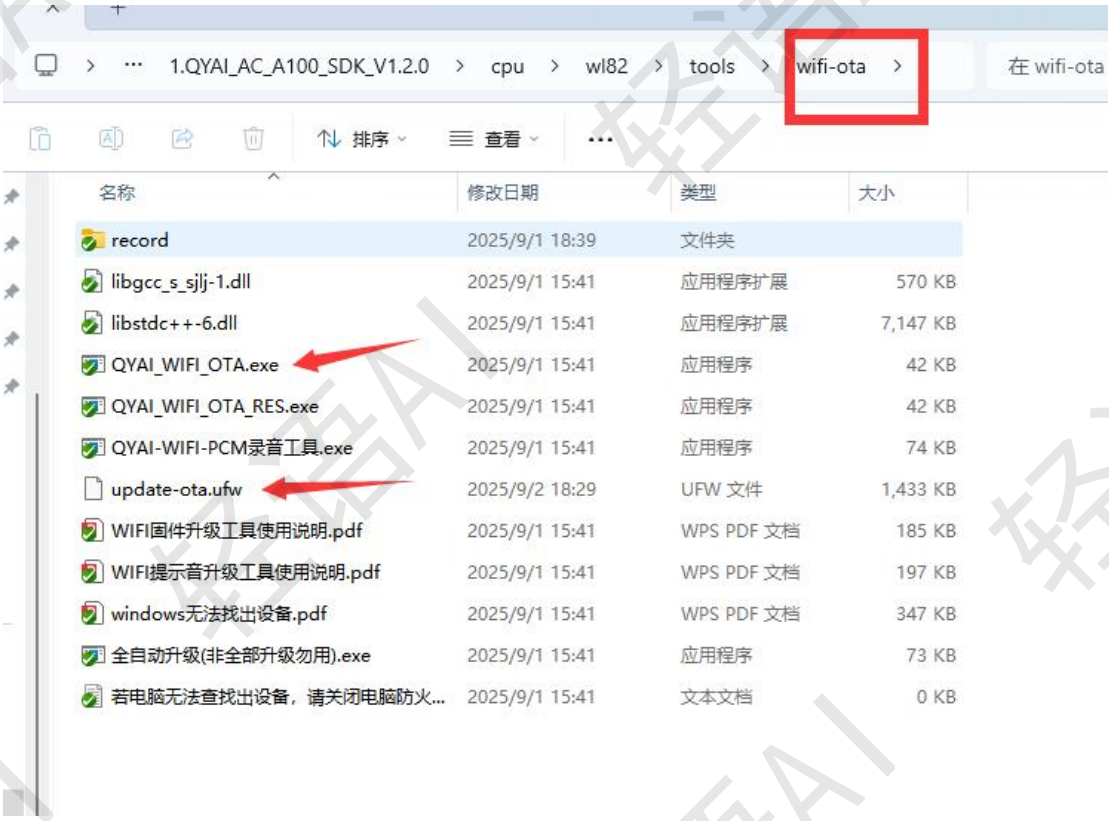


6.1.3 没有 OTA 升级固件：



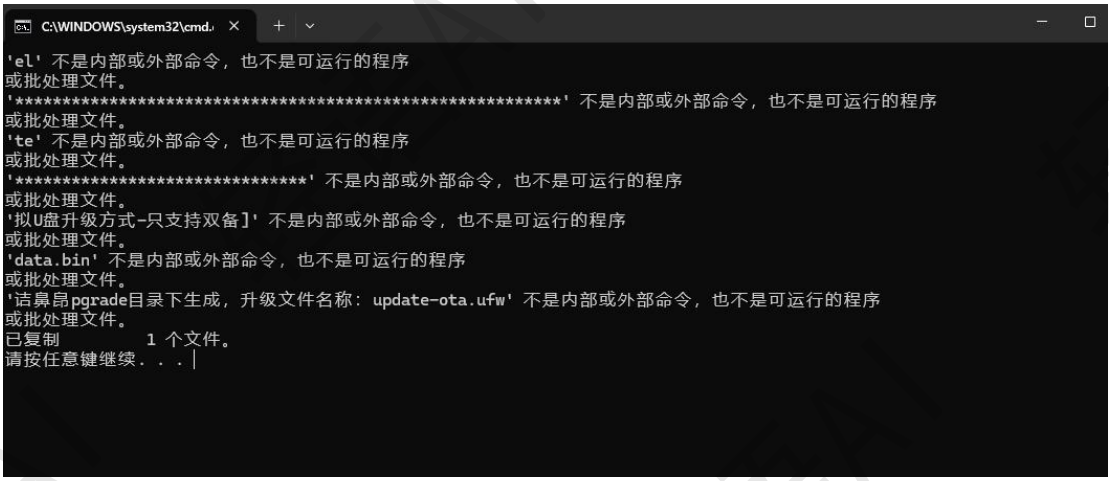


6.1.4 OTA 固件成功生成

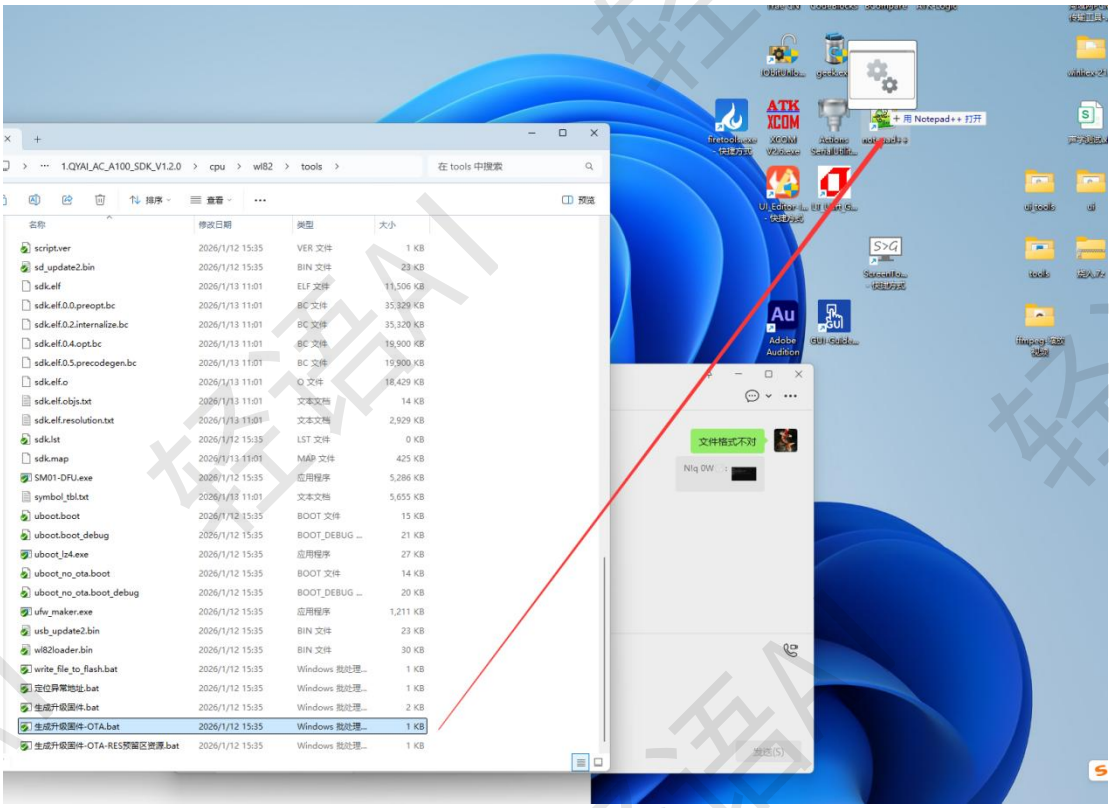


6.2 ota 的 fw 文件生成失败

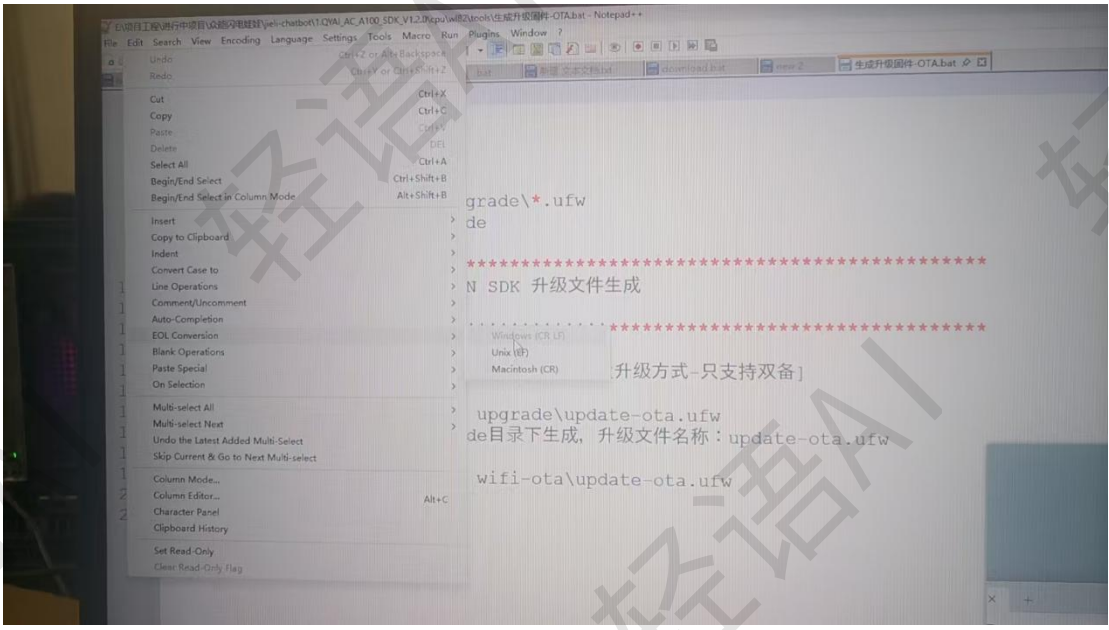
6.2.1 失败页面


C:\WINDOWS\system32\cmd.
'el' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
'**********' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
'te' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
'**********' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
'拟U盘升级方式-只支持双备]' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
'data.bin' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
'洁鼻昂pgrade目录下生成，升级文件名称：update-ota.ufw' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
已复制 1 个文件。
请按任意键继续. . .

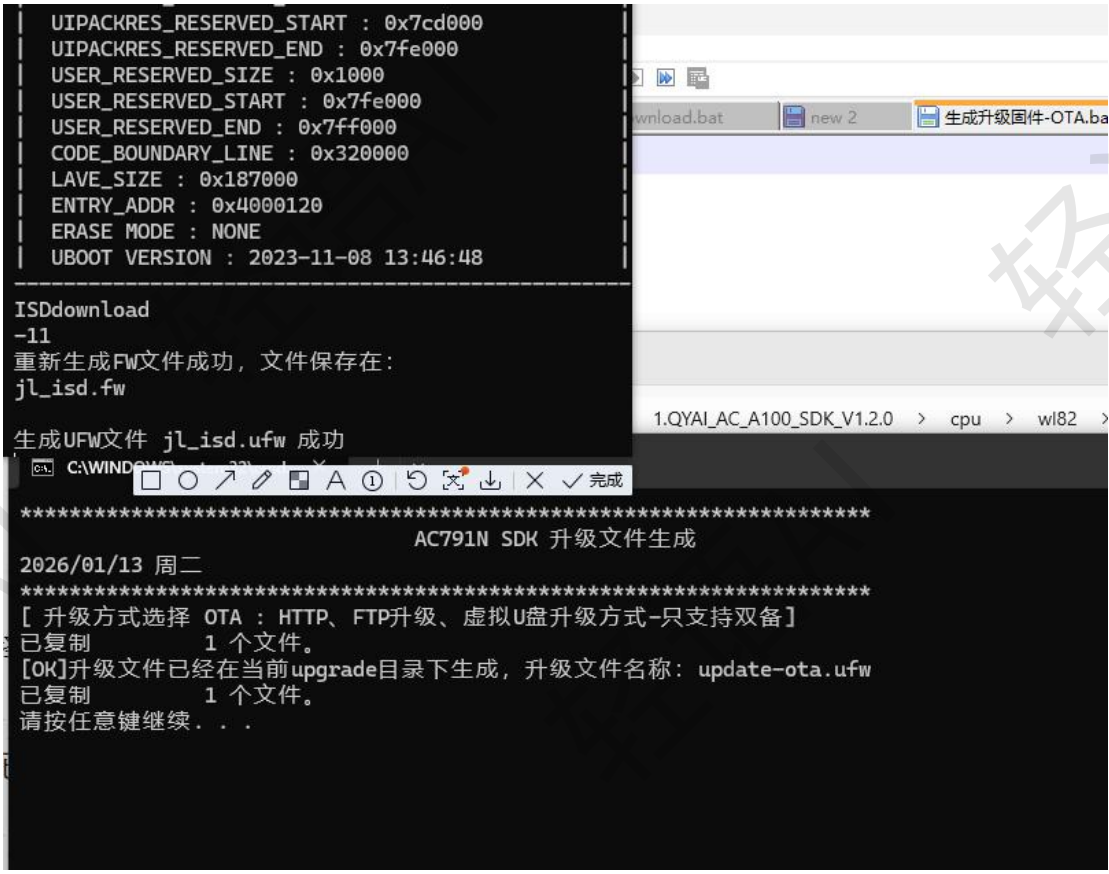
6.2.2 解决


1.QYAI_AC_A100_SDK_V1.2.0 > cpu > wl82 > tools >
在 tools 中搜索
script.ver
sd_update2.bin
sdk.elf
sdk.elf.0.0.preopt.bc
sdk.elf.0.2.internalize.bc
sdk.elf.0.4.opt.bc
sdk.elf.0.5.precodegen.bc
sdk.elf.o
sdk.elf.objs.txt
sdk.elf.resolution.txt
sdk.lst
sdk.map
SM01-DFU.exe
symbol_tbl.txt
uboot.boot
uboot.boot_debug
uboot_lz4.exe
uboot_no_ota.boot
uboot_no_ota.boot_debug
ufw_maker.exe
usb_update2.bin
wl82loader.bin
write_file_to_flash.bat
定位异常地址.bat
生成升级固件.bat
生成升级固件-OTA.bat
生成升级固件-OTA-RES预留区资源.bat
文件格式不对

轻语AI



6.2.3 成功页面



6.3 设备网络不好的原因分析和处理方法

设备网络不好的情况下，下面是原因分析和处理方法（设备有可能是

单个原因，有可能是组合多个原因引起）。

1. 在小房间测试会出现信号差，网络慢问题，路由器穿墙弱就引起网络卡顿网络慢。
    - **解决办法：**需要近距离路由器去联网（距离路由器 1-5 米直线距离，也不能太近）。
2. 低电量也会引起网络慢。
    - **解决办法：**充电再用。
3. 路由器连接太多手机，连接太多用户自己设备（比如手机、摄像头、电脑、平板等设备），也会出现路由器堵塞，设备抢不过网络导致网络慢。
    - **解决办法：**重启路由器，并断开部分手机或设备连接路由器。
4. 路由器或者热点限制设备流量，也会导致网络卡顿。
    - **解决办法：**路由器后台关闭限速设置，手机热点则开启最大限速或者无限速设置。
5. 设备所处的网络环境路由器太多或者手机热点太多导致空中干扰严重，也会导致网络变慢。
    - **解决办法：**可以登录路由器后台，修改路由器选择 wifi 比较少的固定信道，一般 1 信道或者 11 或者 13 信道会好一些；登录路由器后台设置路由器固定为 20MHz 频宽。
6. 路由器是 2.4Hz 和 5GHz 同一个路由器名称，也会导致设备连上部分两个频段同一个名称的路由器，导致网络慢。
    - **解决办法：**登录路由器后台，分开设置 2.4GHz 和 5GHz 频路由名称，设备固定连接 2.4GHz 频段。

7. 资源与存储相关故障

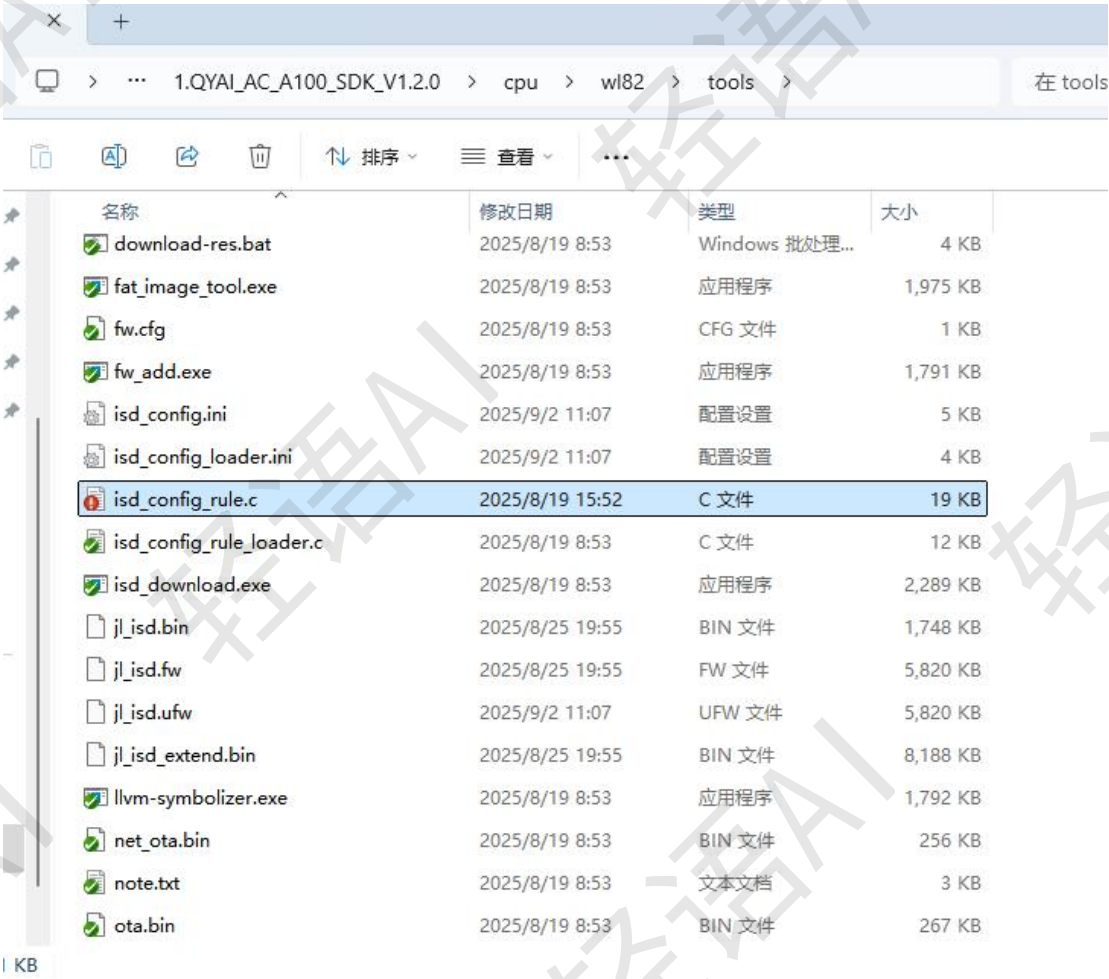
7.1 更新提示音或者 UI 资源后资源地址不够

7.1.1 错误提示：

```
"file saved to packres/AUPACKRES"
compress app.bin (2020944) ----> app.lz4 (1521952) ratio=75.3%  no dict
Device offline, only package the file.
SPI nor flash online
错误：预留配置AUPACKRES中指定的文件的大小比当前配置区域的大小要大。
-1
错误：fw文件中已经存在要添加的文件.
生成UFW文件 jl_isd.ufw 成功
```

7.1.2 重新生成资源地址：

- 打开 isd_config_rule.c 文件



```
#
#操作符说明 OPT:
#    0: 下载代码时擦除指定区域
#    1: 下载代码时不操作指定区域
#    2: 下载代码时给指定区域加上保护
################################################################################
*/
[RESERVED_CONFIG]
//#升级之后需要保留VM数据，在生成升级文件时需要设置VM_OPT=1
VM_ADR=AUTO; [设置VM]
VM_LEN=32K;
#if CONFIG_DOUBLE_BANK_ENABLE
VM_OPT=1;
#else
VM_OPT=1;//单备份升级VM在升级时候默认VM不需要擦除,选择擦除会在ota_loader第二阶段擦除比较长，而且可能会造成VM丢失
#endif

BTIF_ADR=AUTO; [设置资源]
BTIF_LEN=0x1000;
BTIF_OPT=1;

#if defined CONFIG_AUDIO_ENABLE && defined CONFIG_VOICE_PROMPT_FILE_SAVE_IN_RESERVED_ZONE
AUPACKRES_FILE=packres/AUPACKRES;
AUPACKRES_ADR=0x374000; [请根据编译后FLASH INFO打印的实际地址填写,比如0x59a000]
AUPACKRES_LEN=0xe5000; [更新提示音资源打包后必须更新此实际长度,比如0x141000,建议根据后续资源升级的需求预留好空间]
AUPACKRES_OPT=1;
#endif

#if defined CONFIG_UI_ENABLE && defined CONFIG_UI_FILE_SAVE_IN_RESERVED_ZONE
UIPACKRES_FILE=packres/UIPACKRES;
UIPACKRES_ADR=0x459000; [请根据编译后FLASH INFO打印的实际地址填写,比如0x6db000]
UIPACKRES_LEN=0x3a5000; [更新UI资源打包后必须更新此实际长度,比如0x123000, 建议根据后续资源升级的需求预留好空间]
UIPACKRES_OPT=1;
#endif
```

重新生成资源地址

- 把上图中原有的资源地址，赋值为 AUTO 后保存文件，重新编译工程

```
#
#操作符说明 OPT:
#    0: 下载代码时擦除指定区域
#    1: 下载代码时不操作指定区域
#    2: 下载代码时给指定区域加上保护
################################################################################
*/
[RESERVED_CONFIG]
//#升级之后需要保留VM数据，在生成升级文件时需要设置VM_OPT=1
VM_ADR=AUTO; [设置VM]
VM_LEN=32K;
#if CONFIG_DOUBLE_BANK_ENABLE
VM_OPT=1;
#else
VM_OPT=1;//单备份升级VM在升级时候默认VM不需要擦除,选择擦除会在ota_loader第二阶段擦除比较长，而且可能会造成VM丢失
#endif

BTIF_ADR=AUTO; [设置资源]
BTIF_LEN=0x1000;
BTIF_OPT=1;

#if defined CONFIG_AUDIO_ENABLE && defined CONFIG_VOICE_PROMPT_FILE_SAVE_IN_RESERVED_ZONE
AUPACKRES_FILE=packres/AUPACKRES;
AUPACKRES_ADR=AUTO; [请根据编译后FLASH INFO打印的实际地址填写,比如0x59a000]
AUPACKRES_LEN=AUTO; [更新提示音资源打包后必须更新此实际长度,比如0x141000,建议根据后续资源升级的需求预留好空间]
AUPACKRES_OPT=1;
#endif

#if defined CONFIG_UI_ENABLE && defined CONFIG_UI_FILE_SAVE_IN_RESERVED_ZONE
UIPACKRES_FILE=packres/UIPACKRES;
UIPACKRES_ADR=AUTO; [请根据编译后FLASH INFO打印的实际地址填写,比如0x6db000]
UIPACKRES_LEN=AUTO; [更新UI资源打包后必须更新此实际长度,比如0x123000, 建议根据后续资源升级的需求预留好空间]
UIPACKRES_OPT=1;
#endif
```

地址赋值为AUTO后保存，重新编译工程

- 重新生成资源地址
- 找到 isd_config_rule.c 文件并打开
- 在该文件中找到原来设置为 AUTO 的资源地址配置项
- 用新生成的具体地址值替换掉 AUTO

- 保存 isd_config_rule.c 文件



- 重新编译工程

7.2 内存不够

- 报错 wifi_pcm...，在 app_main.c 文件中加入一下代码

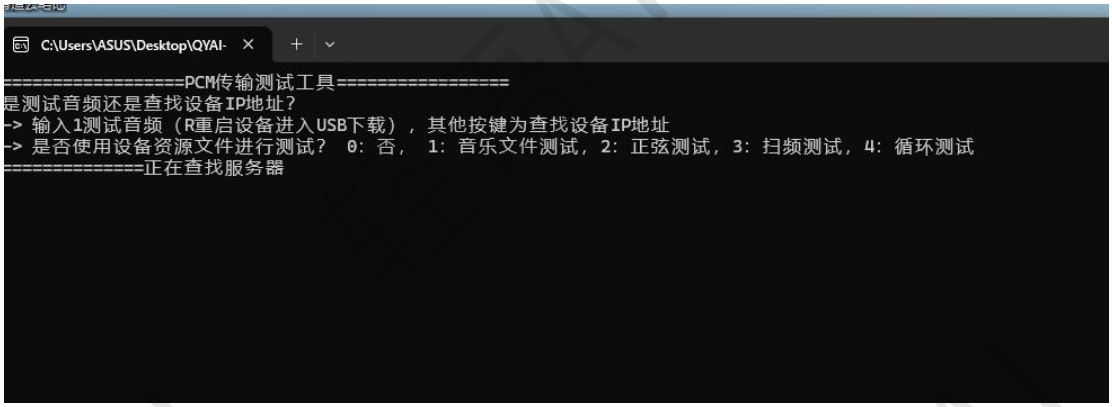
```
Plain Text
int ai_audio_play_buf_size_set(void)
{
return 150*1024;
}
```

7.3 双备份方式改单备份



8. 工程实操与程序更新

8.1 为什么双击 exe 文件一直在查找服务器？



- 可能存在多个该 exe 文件同时运行的情况，需先关闭所有相关 exe，再重新启动 exe 文件。
- 如果不行，设备恢复出厂设置之后重复操作。

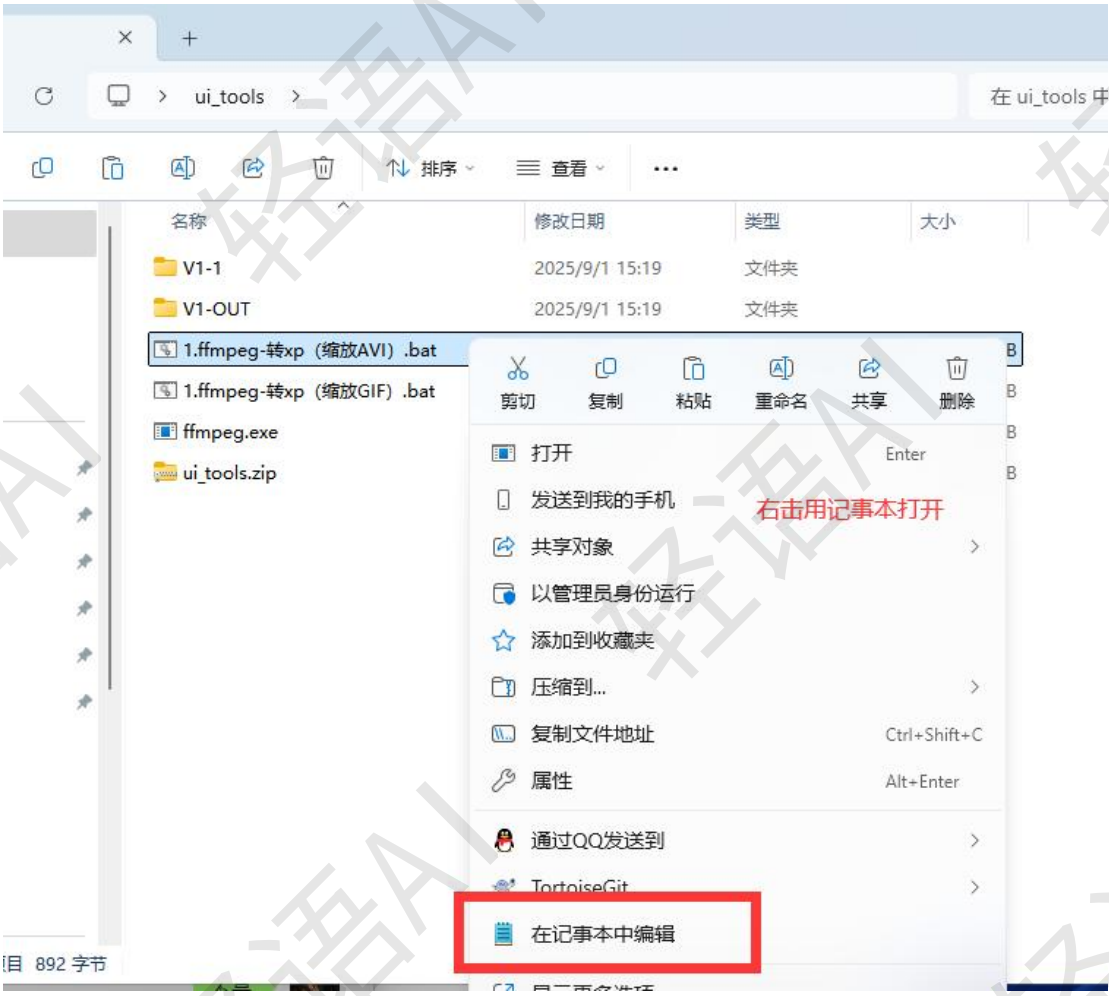
9. 专项功能开发与问题排查

9.1 切换怎么压缩视频?

使用 ffmpeg 处理视频及其分辨率的工具，使用方法如下:

提醒：该 SDK 仅能对分辨率为 16 倍数的视频进行解码。

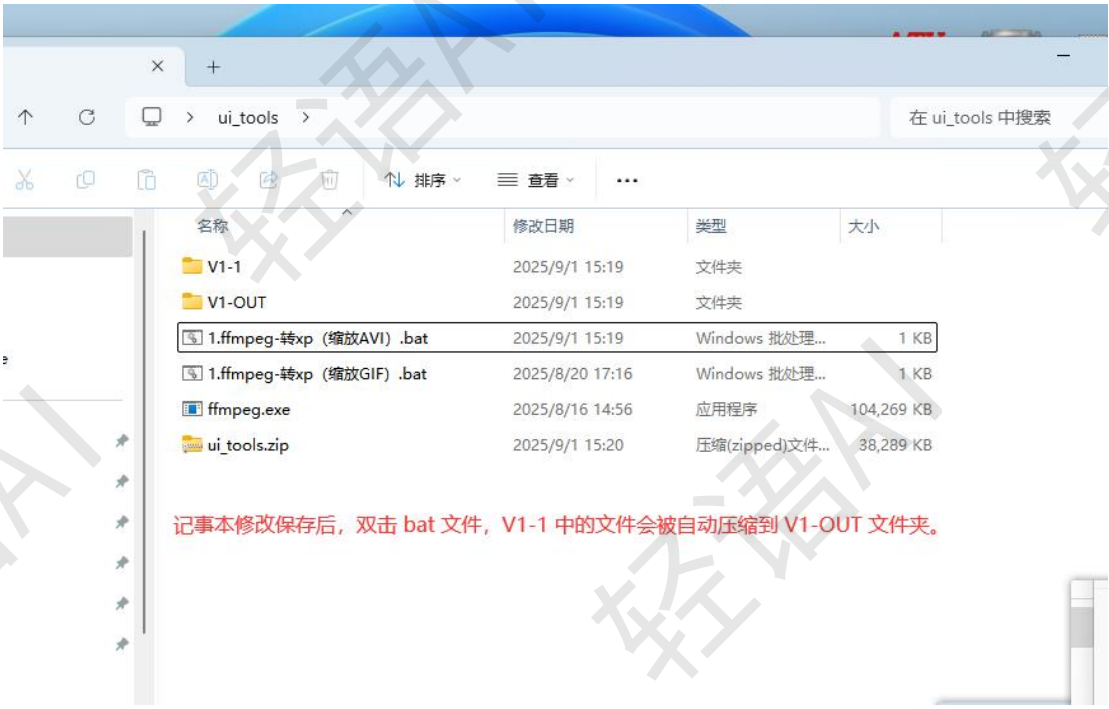
9.1.1 第一步



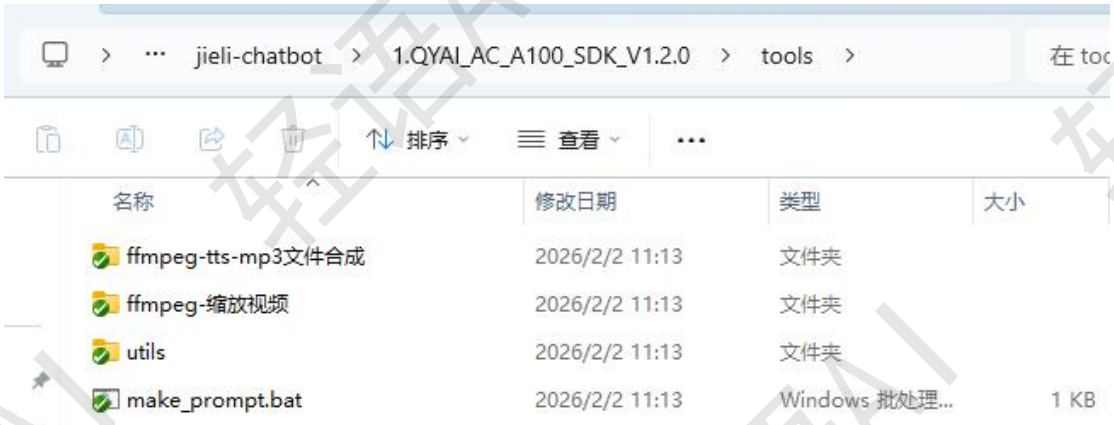
9.1.2 第二步



9.1.3 第三步


ui_tools
在 ui_tools 中搜索
排序
查看
名称
修改日期
类型
大小
V1-1
2025/9/1 15:19
文件夹
V1-OUT
2025/9/1 15:19
文件夹
1.ffmpeg-转xp（缩放AVI）.bat
2025/9/1 15:19
Windows 批处理...
1 KB
1.ffmpeg-转xp（缩放GIF）.bat
2025/8/20 17:16
Windows 批处理...
1 KB
ffmpeg.exe
2025/8/16 14:56
应用程序
104,269 KB
ui_tools.zip
2025/9/1 15:20
压缩(zipped)文件...
38,289 KB
记事本修改保存后，双击 bat 文件，V1-1 中的文件会被自动压缩到 V1-OUT 文件夹。

9.1.4 工具目录


jieli-chatbot
1.QYAI_AC_A100_SDK_V1.2.0
tools
排序
查看
名称
修改日期
类型
大小
ffmpeg-tts-mp3文件合成
2026/2/2 11:13
文件夹
ffmpeg-缩放视频
2026/2/2 11:13
文件夹
utils
2026/2/2 11:13
文件夹
make_prompt.bat
2026/2/2 11:13
Windows 批处理...
1 KB